Laporan Teknis Paket fastsae:

Estimasi Area Kecil Frekuentis dan Bayes Berperforma Tinggi di R

Laporan otomatis dari pembacaan kode sumber fastsae
`https://github.com/ridsonap/fastsae`

Oktober 2026

Ringkasan

Laporan ini mendokumentasikan paket R fastsae (v1.0.0) langsung dari kodenya: setiap persamaan, algoritma, dan klaim implementasi dibaca dari `R/`, `src/*.cpp`, `man/`, `vignettes/`, `tests/`, `DESCRIPTION` dan `README.Rmd`. Bagian pendahuluan menjelaskan masalah estimasi area kecil (SAE) dan arsitektur paket; bagian 2 mendeskripsikan inti C++ (Fisher scoring, identitas Woodbury, bootstrap OpenMP); bagian 3 melaporkan benchmark yang seluruh angkanya dihitung ulang dari `inst/extdata/*.rds`; bagian 4 menyajikan latar statistik; bagian 5 membedah sembilan fungsi model (`eblup_fh()`, `eblup_sfh()`, `eblup_stfh()`, `eblup_bhf()`, `eblup_twofold()`, `hb_area()`, `hb_unit()`, `hb_twofold()` beserta utilitasnya) lengkap dengan pseudo-code yang dicocokkan baris demi baris terhadap sumber; bagian 6 menjelaskan kerangka Bayes–INLA (GMRF, CAR/ICAR, BYM/BYM2, prior PC); bagian 7 membahas utilitas diagnostik dan pemetaan; bagian 8 berisi derivasi dan reproduktibilitas. Seluruh perbedaan antara implementasi dan rumus yang lazim dikutip didaftarkan dalam `DISCREPANCIES.md`, dan referensi yang tidak dapat diverifikasi didaftarkan dalam `TODO_REFERENCES.md`.

Daftar Isi

1 Pendahuluan **9**

1.1 Masalah estimasi area kecil . . . 9
1.2 Estimasi langsung, sintetis, dan EBLUP . . . 9
1.3 Apa yang ditawarkan fastsae . . . 9
1.4 Arsitektur paket . . . 11
1.5 Instalasi, termasuk INLA . . . 12
1.6 Struktur repositori dan cara membaca laporan ini . . . 12

2 Desain perangkat lunak **12**

2.1 Pembagian kerja R dan C++ . . . 13
2.2 Struktur iterasi: Fisher scoring . . . 13
2.3 Identitas matriks: di mana Woodbury dipakai . . . 14
2.4 Strategi memori . . . 15
2.5 Bootstrap paralel . . . 15
2.6 Objek keluaran dan kelas S3 . . . 16
2.7 Ringkasan desain . . . 16

3 Benchmark **17**
3.1 Metodologi . . . 17
3.2 Waktu komputasi . . . 19
3.3 Penggunaan memori . . . 21
3.4 Skala terhadap jumlah area . . . 21
3.5 Ekuivalensi numerik terhadap pembanding . . . 25
3.6 Perbandingan engine `hb_area()` pada model spasial . . . 27
3.7 Kaveat dan keadilan . . . 28
3.8 Kontradiksi klaim dokumentasi terhadap data . . . 29

4 Latar statistik **29**
4.1 Tabel notasi . . . 29
4.2 Model campuran linear, GLS, dan BLUP . . . 29
4.3 EBLUP . . . 31
4.4 ML, REML, dan skor . . . 31
4.5 Fisher scoring . . . 32
4.6 Dekomposisi MSE . . . 32
4.7 MSE bootstrap . . . 32
4.8 Domain tanpa sampel dan prediksi sintetis . . . 33
4.9 Benchmarking: pengertian internal dan eksternal . . . 33

5 Model Fay–Herriot area-level: `eblup_fh` **34**
5.1 Setting data dan kapan dipakai . . . 34
5.2 Persamaan model . . . 34
5.3 Prediktor titik . . . 34
5.4 Estimasi komponen varians . . . 35
5.5 Estimasi MSE . . . 35
5.6 Pseudo-code . . . 36
5.7 Signature, argumen, objek keluaran, dan contoh . . . 36
5.8 Referensi kunci . . . 36

6 Model Fay–Herriot spasial: `eblup_sfh` **38**
6.1 Setting data dan kapan dipakai . . . 38
6.2 Persamaan model . . . 38
6.3 Prediktor titik . . . 38
6.4 Estimasi komponen varians . . . 39
6.5 Estimasi MSE . . . 39
6.6 Pseudo-code . . . 40
6.7 Signature, argumen, objek keluaran, dan contoh . . . 40

6.8 Referensi kunci 42

7 Model Fay–Herriot spasio-temporal: `eblup_stfh` 42

7.1 Setting data dan kapan dipakai 42
7.2 Persamaan model 42
7.3 Prediktor titik 43
7.4 Estimasi komponen varians 43
7.5 Estimasi MSE 44
7.6 Pseudo-code 45
7.7 Signature, argumen, objek keluaran, dan contoh 45
7.8 Referensi kunci 47

8 Model unit-level Battese–Harter–Fuller: `eblup_bhf` 48

8.1 Setting data dan kapan dipakai 48
8.2 Persamaan model 48
8.3 Prediktor titik 48
8.4 Estimasi komponen varians 49
8.5 Estimasi MSE 49
8.6 Pseudo-code 50
8.7 Signature, argumen, objek keluaran, dan contoh 50
8.8 Referensi kunci 51

9 Model sub-area dua-lipat: `eblup_twofold` 51

9.1 Setting data dan kapan dipakai 51
9.2 Persamaan model 52
9.3 Prediktor titik 52
9.4 Estimasi komponen varians 52
9.5 Estimasi MSE 53
9.6 Pseudo-code 55
9.7 Signature, argumen, objek keluaran, dan contoh 55
9.8 Referensi kunci 55

10 Model hierarki Bayes area-level: `hb_area` 55

10.1 Setting data dan kapan dipakai 55
10.2 Persamaan model 56
10.3 Prediktor titik 57
10.4 Estimasi komponen varians 57
10.5 Estimasi MSE 58
10.6 Mode INLA dan validasi terhadap Stan 58

10.7 Pseudo-code . . . 59
10.8 Signature, argumen, objek keluaran, dan contoh . . . 59
10.9 Referensi kunci . . . 61

11 Model hierarki Bayes unit-level: `hb_unit` 61
11.1 Setting data dan kapan dipakai . . . 61
11.2 Persamaan model . . . 62
11.3 Prediktor titik . . . 62
11.4 Estimasi komponen varians . . . 62
11.5 Estimasi MSE . . . 63
11.6 Pseudo-code . . . 63
11.7 Signature, argumen, objek keluaran, dan contoh . . . 63
11.8 Referensi kunci . . . 65

12 Model hierarki Bayes sub-area dua-lipat: `hb_twofold` 65
12.1 Setting data dan kapan dipakai . . . 65
12.2 Persamaan model . . . 65
12.3 Prediktor titik . . . 66
12.4 Estimasi komponen varians . . . 66
12.5 Estimasi MSE . . . 66
12.6 Pseudo-code . . . 67
12.7 Signature, argumen, objek keluaran, dan contoh . . . 67
12.8 Referensi kunci . . . 68

13 Kerangka Bayes: INLA, GMRF, struktur spasial, dan prior 68
13.1 Model latent Gaussian dan aproksimasi INLA . . . 68
13.2 Gaussian Markov Random Field . . . 68
13.3 Struktur presisi: CAR, ICAR, BYM, dan varian lain . . . 69
13.4 Prior PC dan log-gamma . . . 70
13.5 Likelihood, link, dan parameterisasi `scale` . . . 70

14 Lapisan utilitas: benchmarking, diagnostik, perbandingan, ekspor, peta, pra-pemodelan 72
14.1 Benchmarking: `benchmark_sae` . . . 72
14.2 Diagnostik: `diagnose` . . . 74
14.3 Perbandingan dua model: `compare_sae` . . . 75
14.4 Flag reliabilitas dan ekspor: `export_sae` . . . 76
14.5 Peta choropleth: `map_sae` . . . 76
14.6 Pra-pemodelan: `create_weights`, `spatial_test`, `gvf_smooth` . . . 76

14.7 Metode S3 dan grafik 78
14.8 Data simulasi 78
14.9 Rujukan lapisan utilitas 79

A Derivasi identitas Woodbury dan penerapannya **79**

B Derivasi skor dan informasi Fisher (ML dan REML) **80**

C Derivasi suku-suku MSE (g1, g2, g3, g4) **80**

D Glosarium **82**

E Reproduksi angka dan gambar dalam laporan ini **82**

Daftar Gambar

1 Waktu median per iterasi terhadap jumlah area n (skala log pada kedua sumbu). Sumber: `inst/extdata/eblup_benchmark.rds`, `seblup_benchmark.rds`, `steblup_benchmark.rds`, `hb_benchmark.rds`; digambar oleh `scripts/02_figures.R`. 23

2 Waktu median untuk tiga varian model beta (skala log–log). Sumber: `inst/extdata/beta_benchmark.rds`, `spatial_beta_benchmark.rds`, `spatio_temporal_beta_benchmark.rds`; `scripts/02_figures.R`. Panel (c) hanya memiliki $n = 30$ dan $n = 50$. 24

3 Alokasi memori total per iterasi terhadap n (skala log–log). Sumber: file `inst/extdata/*_benchmark.rds` sebagaimana tercantum pada tiap panel; `scripts/02_figures.R`. 24

4 Rasio waktu (pembanding/tes) terhadap n; garis putus-putus menandai rasio $= 1$. Sumber: file `inst/extdata/*_benchmark.rds`; `scripts/02_figures.R`. . 25

5 Ekuivalensi numerik `fastsae` vs `tipsae` untuk $n = 1000$ area: (a) prediktor area terhadap garis identitas; (b) MSE pada skala log. Sumber: `inst/extdata/beta_estimates_n1000.rds`; `scripts/02_figures.R`. 26

6 Waktu *warm* engine `hb_area()` terhadap jumlah area D (skala log) untuk model ICAR: (a) Gaussian, (b) Beta. Sumber: `inst/extdata/hb_engine_benchmark*.rds`; `scripts/02_figures.R`. 28

7 Graf tetangga enam domain beserta matriks yang diturunkannya. Derajat dibaca dari graf: d_1 dan d_3 bertiga, sisanya bertua. Matriks bobot apa pun (matrix, `nb`, `listw`) dikonversi ke graf biner simetris oleh `.convert_spatial_weights` sebelum masuk `inla.graph`. Sumber: `R/inla_utils.R:48-109` dan pemakaian Q di `R/hb_area.R:570-649`. 71

8 Struktur ketergantungan pada field laten GMRF: hanya pasangan dengan $Q_{ij} \neq 0$ yang bersambung (markov blanket), sementara y_i bergantung pada x_i lewat link. Struktur Q inilah yang diisi oleh CAR/ICAR/BYM/BYM2 pada Tabel 28. Sumber: (74) dan `R/hb_area.R:570-649`. 72

Daftar Tabel

1 Fungsi yang diekspor fastsae dan perannya. Sumber: `NAMESPACE` dan kode masing-masing fungsi di `R/`. 10

2 Fungsi Rcpp dan OpenMP. Sumber: `src/RcppExports.cpp`, `src/Makevars`, dan grep `#pragma omp` pada `src/*.cpp`. 11

3 Pembagian kerja lapis R dan lapis C++ per fungsi model. Sumber: pemanggilan `.eblup_core`/`.seblup_core`/`.eblup_stfh_core`/`.eblup_twofold_core`/`.bhf_fit_cpp`/`.eblup_bhf_cpp` beserta berkas pemanggilnya. 13

4 Pengaturan iterasi pada inti Fisher scoring. Sumber: `src/eblup_fh.cpp`, `src/eblup_twofold.cpp`, `src/eblup_sfh_core.cpp`, `src/eblup_stfh_core.cpp`. 14

5 Kelas dan elemen khusus objek keluaran. Sumber: kode perakitan output masing-masing fungsi. 16

6 Skrip benchmark: pembanding, ukuran masalah, iterasi, dan seed. Sumber: pembacaan `benchmarks/*.R` (lihat `_notes/bench_methodology.md`). 18

7 Waktu median per iterasi (detik) menurut jumlah area n. Sumber: file `eblup_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan. 19

8 Waktu median per iterasi (detik) menurut jumlah area n. Sumber: file `seblup_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan. 20

9 Waktu median per iterasi (detik) menurut jumlah area n. Model spasio-temporal dengan T periode waktu; waktu termasuk estimasi komponen varian. Sumber: file `steblup_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan. 20

10 Waktu median per iterasi (detik) menurut jumlah area n. Model area-level beta (`fastsae::hb_area` vs `tipsae::fit_sae`). Sumber: file `beta_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan. 20

11 Waktu median per iterasi (detik) menurut jumlah area n. Model area-level beta dengan efek spasial. Sumber: file `spatial_beta_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan. 20

12 Waktu median per iterasi (detik) menurut jumlah area n. Model beta spasio-temporal ($T = 5$); data tersedia hanya untuk $n = 30$ dan $n = 50$. Sumber: file `spatio_temporal_beta_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan. 21

13 Waktu median per iterasi (detik) menurut jumlah area n. Model hierarki Bayes area-level Gaussian (`hb_area`) vs `hbsae::fSAE.Area`. Sumber: file `hb_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan. 21

14 Alokasi memori total per iterasi (MB) menurut jumlah area n. Sumber: file `eblup_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan. 21

15 Alokasi memori total per iterasi (MB) menurut jumlah area n. Sumber: file `seblup_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan. 22

16 Alokasi memori total per iterasi (MB) menurut jumlah area n. Sumber: file `steblup_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan. 22

17 Alokasi memori total per iterasi (MB) menurut jumlah area n. Model hierarki Bayes area-level Gaussian. Sumber: file `hb_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan. 22

18 Ringkasan alokasi memori total per iterasi (MB): rerata, minimum dan maksimum lintas seluruh n pada tiap file sumber. Kolom "maks" adalah nilai pada n terbesar pada file tersebut. 23

19 Rasio kecepatan = median waktu pembanding / median waktu `fastsae` (semakin besar semakin cepat `fastsae`; nilai < 1 berarti pembanding lebih cepat). Rasio dihitung per n lalu diringkas. 25

20 Ekuivalensi numerik antara `fastsae` dan `tipsae` pada $n = 1000$ area (indikator beta). Semua metrik dihitung dari pasangan kolom pada file sumber. 26

21 Waktu komputasi (detik, run *warm*) `hb_area()` dengan efek ICAR (Besag) pada grid beraturan: INLA, JAX/NumPyro NUTS di CPU dan di GPU, serta `tipsae` (Stan, hanya model Beta). JAX: 2 rantai $\times$ 1000 draw setelah 500 warmup. Sumber: file `hb_engine_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan. 27

22 Kesesuaian tiap engine terhadap INLA pada mesin dan data yang sama, lintas seluruh D: korelasi minimum rerata posterior, rentang median rasio sd posterior (engine/INLA), dan $\hat{R}$ terpisah (*split*) terbesar. Sumber: file `hb_engine_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan. 27

23 Klaim dokumentasi yang bertolak belakang dengan file data. Sumber: `README.Rmd`, `vignettes/benchmarks.Rmd`, `jssjournal/article.tex` vs `inst/extdata/*.rds` (butir DOC-1 – DOC-11 di `DISCREPANCIES.md`). 30

24 Notasi utama dan padanannya dalam kode. Sumber: simbol pada `R/*.R` dan `src/*.cpp`. 30

25 Penanganan domain tanpa sampel. Sumber: cabang kode pada kolom terakhir. 33

26 Simulasi Monte Carlo berbasis model untuk `eblup_stfh` dengan sel tanpa estimasi langsung: $D = 50$, $T = 5$, 200 replikasi (bootstrap $B = 100$ pada 50 replikasi pertama), $(\sigma_1^2, \rho_1, \sigma_2^2, \rho_2) = (0.5, 0.5, 0.2, 0.5)$. MSE empiris EBLUP dan estimator sintetis $x'\hat{\beta}$; bias relatif (%) estimator MSE analitik dan bootstrap; cakupan (%) interval $\hat{\theta} \pm 1{,}96\sqrt{\widehat{\mathrm{MSE}}}$ analitik, dirata-ratakan per jenis sel. Sumber: `benchmarks/output/sim_stfh_unsampled.rds` (`benchmarks/sim_stfh_unsampled.R`); angka dihitung ulang oleh `scripts/01_tables.R`. 45

27 Pemetaan `st_interaction` ke bentuk interaksi Knorr–Held. Sumber: penyusunan string `f(...)` di `R/hb_area.R:651-728`. 56

28 Struktur presisi spasial dan pemakainya di fastsae. Sumber: penyusunan `f(...)` di `R/hb_area.R:570-649` dan `R/hb_unit.R:280-294`, `R/hb_twofold.R:250-266`. 69

29 Prior default di tiga fungsi Bayes (baca: "argumen bawaan yang didefinisikan dalam signature"). Sumber: signature fungsi dan pemakaian `hyper` di `R/hb_area.R:559-649`, `R/hb_unit.R:137-144`, `R/hb_twofold.R:155-171`. . . 70

30 Family likelihood dan parameterisasi yang dikirim fastsae ke INLA. Sumber: `R/hb_area.R:785-841`, `R/hb_unit.R:303-311`, `R/hb_twofold.R:286-296`. . . 71

31 Metode S3 utama beserta file sumbernya. 78

32 Pemakaian identitas Woodbury di dalam paket. Sumber: baris kode yang dikutip pada kolom terakhir. 80

33 Glosarium tiga arah: istilah — simbol — nama di kode. 82

1 Pendahuluan

1.1 Masalah estimasi area kecil

Banyak keputusan publik membutuhkan indikator pada unit geografis yang kecil: kabupaten, kecamatan, desa, sekolah, atau rumah sakit. Survei nasional yang dirancang untuk tingkat nasional atau provinsi biasanya menghasilkan sampel yang terlalu kecil pada unit-unit tersebut, sehingga *estimasi langsung* (*direct estimator*) menjadi tidak stabil atau bahkan tidak terdefinisi. Estimasi langsung

$$\hat{y}_d^{\text{dir}} = \frac{1}{n_d} \sum_{i \in s_d} y_{id}, \qquad \text{Var}\left(\hat{y}_d^{\text{dir}}\right) = \psi_d = \frac{S_d^2}{n_d}, \tag{1}$$

hanya memakai informasi dari domain d itu sendiri; ketika n_d kecil, galat baku $\sqrt{\psi_d}$ melebihi toleransi keandalan yang lazim (misalnya kamuflase RSE di atas 25%; lihat bagian 14.2). Beberapa domain bahkan tidak tersampel sama sekali ($n_d = 0$), dan (1) tidak terdefinisi di sana.

Estimasi berbasis model (*model-based estimation*) menyiasati hal tersebut dengan memodelkan domain kecil sebagai bagian dari satu himpunan data bersama. Informasi dari domain lain "dipinjamkan" melalui model, sehingga estimasi menjadi lebih halus (*shrinkage*) dan domain tanpa sampel tetap dapat diprediksi. Biayanya adalah asumsi model. Pendahuluan metode ini dalam literatur SAE dibahas oleh Pfeffermann [2013] dan Rao and Molina [2015].

1.2 Estimasi langsung, sintetis, dan EBLUP

Tiga kelas prediktor membedah literatur SAE [Rao and Molina, 2015]:

(a) **Prediktor sintetis** memakai hanya kovariat domain: $\hat{\theta}_d^{\text{syn}} = \bar{x}_d' \hat{\beta}$. Kuat pada domain tanpa sampel, tetapi mengabaikan data domain itu sendiri bila ada.

(b) **EBLUP** (*empirical best linear unbiased predictor*) menggabungkan kovariat dan data domain melalui model campuran linear: $\hat{\theta}_d = x_d' \hat{\beta} + \hat{u}_d$. Perkiraan komponen varians σ^2 dari data membuatnya "empiris". Model Fay–Herriot [Fay and Herriot, 1979] adalah contoh kanoniknya.

(c) **EBP** (*empirical best predictor*) dan pendekatan Bayes memperluas gagasan yang sama ke indikator non-Gaussian (rasio, proporsi, jumlah) dan ke pemodelan penuh atas komponen varians [Molina and Rao, 2010, Molina and Marhuenda, 2015].

1.3 Apa yang ditawarkan fastsae

fastsae v1.0.0 (deskripsi diambil dari `DESCRIPTION`) adalah paket R untuk estimasi area kecil yang membagi dua jalur: model frekuentis EBLUP yang dikompilasi ke C++ lewat Rcpp/RcppArmadillo, dan model Bayes hierarki yang diserahkan ke INLA. Tabel 1 meringkas fungsi yang diekspor (daftar lengkap menurut `NAMESPACE`; peta file, metode estimasi, dan metode MSE ada di `CODE_MAP.md`).

Empat pilar yang membedakan paket ini, semuanya terverifikasi dari kode:

1. **Inti numerik di C++/Armadillo.** Fisher scoring untuk model area-level, sub-area, spasial, dan spasio-temporal ditulis dalam `src/*.cpp` (bagian 2).

Tabel 1: Fungsi yang diekspor fastsae dan perannya. Sumber: `NAMESPACE` dan kode masing-masing fungsi di `R/`.

Fungsi	Jenis	Peran
`eblup_fh`	frekuentis	Fay–Herriot area-level
`eblup_sfh`	frekuentis	Fay–Herriot spasial (SAR/error-components)
`eblup_stfh`	frekuentis	Fay–Herriot spasio-temporal
`eblup_bhf`	frekuentis	Battese–Harter–Fuller unit-level
`eblup_twofold`	frekuentis	model sub-area *two-fold*
`hb_area`	Bayes	hierarki Bayes area-level, 6 family likelihood
`hb_unit`	Bayes	hierarki Bayes unit-level
`hb_twofold`	Bayes	hierarki Bayes sub-area
`inla_engine`, `jax_engine`	Bayes	objek *engine* untuk `hb_*`: INLA (default) atau MCMC JAX/NumPyro (hanya `hb_area`)
`setup_numpyro_env`, `check_numpyro_available`	Bayes	menyiapkan/memeriksa lingkungan Python untuk `jax_engine`
`benchmark_sae`	utilitas	kalibrasi ke agregat target
`diagnose`	utilitas	uji kecocokan, uji bias, Moran's I, flag keandalan
`compare_sae`	utilitas	perbandingan dua model
`export_sae`, `map_sae`	utilitas	ekspor tabel dan peta choropleth
`sim_spatial_weights`, `sim_area_data`, `sim_series_data`	utilitas	pembangkit data dan matriks tetangga
`add_reliability_flags`	utilitas	flag RSE
`create_weights`, `spatial_test`	pra-pemodelan	matriks bobot spasial dari `sf`/koordinat; uji Moran/Geary/LM sebelum memilih model spasial
`gvf_smooth`	pra-pemodelan	penghalusan `vardir` dengan *generalized variance function*
`autoplot` (S3)	utilitas	grafik model, benchmark, diagnosis

2. **Bootstrap paralel OpenMP.** Lima berkas sumber memakai `#pragma omp parallel for` (`eblup_twofold.cpp:558`, `eblup_sfh_pbmse.cpp:116`, `eblup_sfh_npbmse.cpp:188`, `eblup_stfh_pbmse.cpp:182`, `eblup_bhf_core.cpp:283`); bilangan acak diambil berurutan dari RNG R sehingga hasil hanya bergantung pada `seed`.

3. **Lapisan INLA untuk model Bayes.** Enam family likelihood (Gaussian, Beta, Binomial, Poisson, Negative Binomial, Gamma), enam struktur spasial, lima struktur temporal, dan interaksi spatio-temporal (bagian 13).

4. **Domain tanpa sampel ditangani eksplisit** di setiap fungsi model: FH memprediksi $x'_{ns}\hat{\beta}$, BHF menghasilkan prediktor sintetis, dan `hb_unit()` menambahkan baris `y = NA` per domain (lihat tiap bagian model).

1.4 Arsitektur paket

Secara arsitektur fastsae terdiri atas lima lapis:

1. **Lapis pemanggilan R** — parsing formula, validasi argumen, pembangunan `model.frame`/`model.matrix`, pelabelan domain, dan perakitan objek keluaran berkelas S3. Semua file berada di `R/` mengikuti urutan `Collate` pada `DESCRIPTION`.

2. **Lapis C++** — sembilan berkas `src/*.cpp` yang mengekspor sepuluh fungsi Rcpp (Tabel 2), ditambah `sar_engine.h` (faktorisasi Cholesky sparse untuk model SAR); semua dikompilasi dengan `CXX17` dan tautan LAPACK/BLAS serta OpenMP melalui `src/Makevars`.

3. **Lapis INLA** — `R/inla_utils.R` merangkai formula `f(...)` dan mengekstrak hasil; tidak diekspor (semua `@noRd`).

4. **Lapis *engine* Bayes** — `inla_engine()` dan `jax_engine()` (`R/engine.R`) memisahkan spesifikasi model dari cara estimasinya; `jax_engine` menjalankan MCMC NUTS di Python (JAX/NumPyro) lewat reticulate (`R/hb_area_jax.R`, `inst/python/`). lme4 tidak lagi dipakai oleh kode paket.

5. **Metode S3** — kelas `fastsae` beserta turunannya (`fastsae_unit`, `fastsae_hb*`, `fastsae_diagnose`, `fastsae_benchmark`, `fastsae_comparison`) memiliki method `print`, `summary`, `plot`, `autoplot`, `coef`, `fitted`, `residuals`.

Tabel 2: Fungsi Rcpp dan OpenMP. Sumber: `src/RcppExports.cpp`, `src/Makevars`, dan grep `#pragma omp` pada `src/*.cpp`.

Berkas sumber	Fungsi Rcpp	OpenMP
`eblup_fh.cpp`	`.eblup_core`	—
`eblup_unit.cpp`	`.eblup_bhf_cpp`	—
`eblup_bhf_core.cpp`	`.bhf_fit_cpp`, `.pbmse_bhf_cpp`	baris 283
`eblup_twofold.cpp`	`.eblup_twofold_core`	baris 558
`eblup_sfh_core.cpp`	`.seblup_core`	—
`eblup_sfh_pbmse.cpp`	`.seblup_pbmse`	baris 116
`eblup_sfh_npbmse.cpp`	`.seblup_npbmse`	baris 188
`eblup_stfh_core.cpp`	`.eblup_stfh_core`	—
`eblup_stfh_pbmse.cpp`	`.pbmse_stfh`	baris 182

1.5 Instalasi, termasuk INLA

Paket berada di CRAN dan dapat dipasang bersama versi pengembangannya (petunjuk di `README.Rmd:85-108`):

Listing 1: Perintah instalasi sebagaimana terdokumentasi di `README.Rmd`.

```
install.packages("fastsae")                       # rilis stabil
# install.packages("remotes")
remotes::install_github("ridsonap/fastsae")      # versi pengembang

# model Bayes memerlukan INLA (tidak ada di CRAN)
install.packages("INLA",
  repos = c(getOption("repos"),
            INLA = "https://inla.r-inla-download.org/R/stable"),
  dep = TRUE)
```

Repo INLA pada baris terakhir identik dengan `Additional_repositories` di `DESCRIPTION`. INLA berada pada `Suggests`, sehingga fungsi frekuentis tetap dapat dipakai tanpanya; `hb_area()` memeriksanya lebih dulu melalui `.check_inla_installed()` (`R/inla_utils.R:11-17`) dan melempar `cli_abort` dengan instruksi repo bila tidak ada.

1.6 Struktur repositori dan cara membaca laporan ini

Listing 2: Struktur repositori (pilihan relevan).

```
R/           28 berkas R     : lapis pemanggilan, engine, utilitas, data
src/         10 berkas C++   : inti numerik + RcppExports (+3 header)
inst/python/                  : model NumPyro untuk jax_engine()
man/         40 berkas Rd    : dokumentasi (sumber sitasi roxygen)
vignettes/    7 Rmd           : pengantar, benchmark, diagnosis, model Bayes
tests/        testthat        : >1.500 ekspektasi lulus (lihat bagian 8.3)
inst/extdata/ 9 file .rds     : hasil benchmark (satu-satunya sumber angka)
benchmarks/ 14 skrip          : pembangkit hasil benchmark
```

Laporan ini disusun agar setiap klaim dapat ditelusuri: nomor baris dalam teks mengacu ke berkas sumber, seluruh angka berasal dari `inst/extdata/*.rds` lewat skrip di `scripts/`, dan seluruh perbedaan antara kode dan rumus publik dikumpulkan di `DISCREPANCIES.md`. Referensi dikutip dengan penulis–tahun (natbib) dan hanya entri yang lolos verifikasi DOI yang masuk ke `references.bib` (lihat `TODO_REFERENCES.md` untuk yang tidak lolos).

Referensi utama pendahuluan: Fay and Herriot [1979] untuk model area-level, Rao and Molina [2015] untuk kerangka umum, Pfeffermann [2013] untuk perkembangan mutakhir, Rue et al. [2009] untuk INLA, serta tiga paket pembanding pada benchmark: sae [Molina and Marhuenda, 2015], emdi [Kreutzmann et al., 2019], dan tipsae [De Nicolò and Gardini, 2024].

2 Desain perangkat lunak

Bagian ini menjelaskan apa yang sebenarnya dilakukan inti kode: bagian mana yang berjalan di C++, identitas matriks mana yang dipakai, bagaimana memori diatur, dan bagaimana bootstrap diparalelkan. Rumus statistik yang muncul di sini dibahas lebih rinci pada bagian 4 dan tiap bagian model.

2.1 Pembagian kerja R dan C++

Prinsipnya: R menangani *parsing* dan *perakitan*, C++ menangani *iterasi numerik*. Tabel 3 memberi pembagian persisnya.

Tabel 3: Pembagian kerja lapis R dan lapis C++ per fungsi model. Sumber: pemanggilan `.eblup_core/.seblup_core/.eblup_stfh_core/.eblup_twofold_core/ .bhf_fit_cpp/.eblup_bhf_cpp` beserta berkas pemanggilnya.

Fungsi	Di R	Di C++
`eblup_fh`	`model.frame(na.pass)`, validasi `vardir`, pelabelan `domain`, kelas	pemisahan sampel/tak-sampel, Fisher scoring, EBLUP, MSE $g_1 g_2 g_3$, loglik
`eblup_sfh`	validasi W ($D \times D$, atau dibangun dari `sf`), pemilihan jalur MSE	faktorisasi SAR sparse, Fisher scoring 2-parameter + fallback terkendala, MSE $g_1 \dots g_4$, kriging, bootstrap
`eblup_stfh`	pengurutan domain-major, validasi panel seimbang, W, nilai awal	Fisher scoring REML 3/4-parameter terstruktur (Woodbury), fallback terkendala, MSE analitik $g_1 \dots g_4$, bootstrap
`eblup_twofold`	pembuatan `area` 0-based, `set.seed`	Fisher scoring 2-parameter per blok, MSE analitik, bootstrap
`eblup_bhf`	`model.frame`, pengurutan `Xpop`, rata-rata populasi, perakitan	REML/ML terprofil atas $\gamma = \sigma_u^2/\sigma_e^2$, prediktor per domain, bootstrap
`hb_*`	seluruhnya: merangkai formula INLA, prior, kontrol, ekstraksi hasil	— (INLA sendiri adalah biner eksternal; `jax_engine` memanggil Python)

Konsekuensinya, model Bayes tidak memakai C++ buatan paket sama sekali: seluruh pekerjaan numerik berada di dalam INLA atau, untuk `hb_area(engine = jax_engine())`, di JAX/NumPyro, sedangkan paket hanya merangkai model dan membaca hasilnya (bagian 13).

2.2 Struktur iterasi: Fisher scoring

Model frekuentis area-level dan sub-area memakai algoritme yang sama: *Fisher scoring* atas vektor parameter komponen varians θ. Pola yang sama berulang di empat inti C++ (lihat bagian 4 untuk turunannya):

1. hitung $\mathbf{V}(\theta)$ (dalam bentuk terfaktor), $\mathbf{Q} = (\mathbf{X}'\mathbf{V}^{-1}\mathbf{X})^{-1}$, dan $P = \mathbf{V}^{-1} - \mathbf{V}^{-1}\mathbf{X}\mathbf{Q}\mathbf{X}'\mathbf{V}^{-1}$;
2. hitung skor $s(\theta)$ dan informasi Fisher $\mathcal{I}(\theta)$;
3. update Newton–Fisher $\theta^{(k+1)} = \theta^{(k)} + \mathcal{I}^{-1}s$;
4. uji batas (*boundary*) dan kriteria konvergensi;
5. pass akhir: hitung ulang $\mathbf{Q}, \hat{\beta}$, galat baku, prediktor, dan MSE pada $\hat{\theta}$ *setelah* update terakhir.

SFH dan STFH menambahkan *fallback* terkendala: bila iterasi klasik (yang sengaja dibuat sama dengan `sae`) gagal atau berakhir pada varians negatif, likelihood dimaksimumkan ulang dengan Fisher scoring terproyeksi dan *step halving*; solusi interior iterasi klasik tidak berubah. BHF tidak memakai Fisher scoring: β dan σ_e^2 diprofilkan secara analitik dan sisa masalah satu dimensi dalam γ diselesaikan dengan pencarian grid dan metode Brent (bagian 8).

Tabel 4 merangkum pengaturan iterasi tiap inti. Semua nilai dibaca dari kode, bukan dari dokumentasi.

Tabel 4: Pengaturan iterasi pada inti Fisher scoring. Sumber: `src/eblup_fh.cpp`, `src/eblup_twofold.cpp`, `src/eblup_sfh_core.cpp`, `src/eblup_stfh_core.cpp`.

Inti	Parameter	Nilai awal	Toleransi	Penanganan batas
`.eblup_core`	σ^2	$\max(\text{median}(\psi), 0)$	relatif $\left\lvert\Delta\sigma^2 / \max(\sigma^2, 10^{-12})\right\rvert \le 10^{-4}$, `maxiter` $= 100$	update non-finite/negatif dipaksa 0
`.eblup_twofold_core`	(σ_v^2, σ_u^2)	$0.5\,\text{median}(\psi)$ keduanya	maksimum selisih relatif per parameter $\le 10^{-4}$, `maxiter` $= 100$	negatif $\to 0$; informasi singular $\to$ `pinv`, lalu skor diagonal
`.seblup_core`	(σ^2, ρ)	$\text{median}(\psi)$, 0.5	$\max\left\lvert\Delta\theta/(\theta + 10^{-12})\right\rvert \le 10^{-4}$, `maxiter` $= 100$	ρ di-clamp ± 0.999; gagal atau $\sigma^2 < 0$ $\to$ fallback terkendala
`.eblup_stfh_core`	$(\sigma_1^2, \rho_1, \sigma_2^2, \rho_2)$	$0.5\,\text{median}(\psi)$ dan 0.5	relatif $\le 10^{-4}$, `maxiter` $= 100$	$\lvert\rho\rvert \ge 1 \to \pm 0.999$; gagal atau varians $< 0 \to$ fallback REML terkendala

Dua hal patut dicatat. *Pertama*, pada SFH batas bawah $\rho = -0.999$ dipetakan ke -1 seperti `sae::eblupSFH` hanya bila $(\mathbf{I} + W')(\mathbf{I} + W)$ tidak singular; pada graf bipartit (mis. grid rook) ρ tetap -0.999, dan batas atas tidak pernah dipetakan ke $+1$. *Kedua*, `eblup_stfh` selalu memakai REML: `method` di-hardcode `"REML"` di `R/eblup_stfh.R:325`.

2.3 Identitas matriks: di mana Woodbury dipakai

Tidak satu pun inti frekuentis membentuk invers matriks kovarians penuh. Semua memakai struktur $\mathbf{V}$:

Two-fold (per blok, Sherman–Morrison). Untuk tiap area d dengan n_d sub-area, kovarians $\mathbf{V}_d = \sigma_v^2 \mathbf{1}\mathbf{1}' + \text{diag}(\sigma_u^2 + \psi_d)$. Karena suku $\sigma_v^2 \mathbf{1}\mathbf{1}'$ berpangkat satu, inversi dilakukan lewat

$$\mathbf{V}_d^{-1} a = \text{dd} \odot a - c\,\text{dd}\,(\text{dd}' a), \qquad \text{dd} = \frac{1}{\sigma_u^2 + \psi_d}, \quad c = \frac{\sigma_v^2}{1 + \sigma_v^2 \sum_j \text{dd}_j}, \tag{2}$$

persis kode `eblup_twofold.cpp:102` dan `:375-377`. Biaya per blok $O(n_d)$, bukan $O(n_d^3)$.

FH spasial (Woodbury + faktorisasi sparse). Dengan $A = (\mathbf{I} - \rho W')(\mathbf{I} - \rho W)$ pada semua D domain dan $M = A + \sigma^2 E' \Psi^{-1} E$ (`src/sar_engine.h`),

$$\mathbf{V}^{-1} = \Psi^{-1} - \sigma^2 \Psi^{-1} E M^{-1} E' \Psi^{-1}, \qquad \log|\mathbf{V}| = \log|\Psi| + \log|M| - \log|A|\,. \tag{3}$$

A dan M difaktorkan dengan LDL$'$ sparse Eigen (urutan AMD, analisis simbolik sekali pada pola $\mathbf{I} + W + W' + W'W$) bila pola itu jarang ($\le 25\%$ non-nol), selain itu dengan faktorisasi padat. Semua besaran fit dan MSE — termasuk g_3, g_4, dan MSE kriging — diperoleh dari *solve* dengan A dan M; tidak ada invers $D \times D$ maupun perkalian berordo D^3.

Spasio-temporal (Woodbury terstruktur). Dengan $A = \text{blockdiag}_d(\sigma_2^2 \Omega_2 + \text{diag}(\psi_d))$ dan $Z = \mathbf{I}_D \otimes \mathbf{1}_T$ (`eblup_stfh_core.cpp:71-88`),

$$\mathbf{C} = (\sigma_1^2 \Omega_1)^{-1} + Z' A^{-1} Z, \tag{4}$$

$$\mathbf{V}^{-1} = A^{-1} - A^{-1} Z\, \mathbf{C}^{-1} Z' A^{-1}, \qquad P = A^{-1} - U\,\text{blockdiag}(\mathbf{C}^{-1}, \mathbf{Q})\, U', \tag{5}$$

$U = [A^{-1}Z,\ \mathbf{V}^{-1}\mathbf{X}]$. Inversi A hanya $T \times T$ per domain dan $\mathbf{C}$ hanya $D \times D$; karena turunan $\mathbf{V}_a$ berbentuk ZB_aZ' atau $\mathbf{I}_D \otimes E_a$, semua jejak skor, informasi, dan MSE direduksi ke blok kecil. Biaya $O(D^3 + MT(D+p))$ per iterasi dan memori $O(M(D+p))$, alih-alih $O(M^3)$ dan $O(M^2)$.

FH dan BHF. `.eblup_core` (FH) bekerja pada skalar per-observasi: $w_d = 1/(\psi_d + \sigma^2)$ dan matriks $\mathbf{Q}$ hanya $p \times p$. BHF memakai statistik cukup per domain (*cross-product* dalam-domain dan rata-rata domain), sehingga setiap evaluasi likelihood berbiaya $O(Dp^2 + p^3)$, tidak bergantung pada jumlah unit. Implikasi terhadap skala dibahas pada bagian 3.

2.4 Strategi memori

Tiga kebiasaan kode menjelaskan mengapa jejak memori fastsae kecil:

1. **Tidak ada matriks kovarians penuh.** FH, two-fold, dan BHF tidak pernah membuat matriks $m \times m$; SFH menyimpan faktor sparse A dan M serta blok $D \times m$ hasil *solve*; STFH hanya blok $T \times T$, $D \times D$, dan faktor U berukuran $M \times (D+p)$.
2. **Pola tetap dihitung sekali.** Pola sparse dan analisis simbolik SFH dibangun sekali sebelum iterasi; setiap iterasi hanya mengisi ulang nilai numerik dan memfaktorkan ulang.
3. **Bootstrap menulis per-kolom.** Replikat disimpan pada matriks $N \times B$ (two-fold) atau lewat akumulator kuadrat (SFH/STFH/BHF), bukan menyimpan seluruh hasil per replikat.

Bandingkan `sae`, yang pada benchmark membangun matriks penuh per iterasi untuk masalah $n = 1000$ (Tabel 14 dan Tabel 18).

2.5 Bootstrap paralel

Lima titik memakai `#pragma omp parallel for`: bootstrap SFH parametrik dan nonparametrik, STFH, two-fold, dan BHF. SFH dan STFH memanggil `omp_set_num_threads` bila `n_threads > 0` (`eblup_sfh_pbmse.cpp:82`, `eblup_sfh_npbmse.cpp:155`, `eblup_stfh_pbmse.cpp:141`); `n_threads <= 0` berarti memakai semua core. BHF meneruskan `n_threads` ke klausa `num_threads`. Skema umumnya sama:

Dua detail penting:

- **RNG dibangkitkan sebelum pragma** karena generator R tidak aman thread. Konsekuensinya hasil bootstrap *reproducible terlepas dari jumlah thread*, selama `seed` diberikan. Panggilan di sisi R dibungkus `.with_seed()`, sehingga seed tetap tidak mengubah aliran RNG sesi pengguna.
- **Jumlah thread** diatur lewat argumen `n_threads` (SFH/STFH/BHF) atau `OMP_NUM_THREADS` (two-fold, yang tidak punya argumen itu).

Penanganan replikat gagal kini seragam untuk model yang mengestimasi ulang θ per replikat: SFH dan STFH mengganti replikat gagal dengan draw baru sampai tepat B replikat valid (STFH maksimum $5B$ percobaan), sedangkan BHF membagi dengan jumlah replikat yang berhasil.

Algorithm 1: Akumulasi: $\widehat{\mathrm{MSE}} = B^{-1}\sum_b(\hat{\theta}_b^* - \theta_b^*)^2$; versi bias-koreksi pada teks SFH.

Masukan *data* $(\mathbf{X}, y, \psi, W)$, B, `n_threads`, `seed` **Keluaran** *vektor MSE per area*
  Data *bilangan acak dibangkitkan* sekuensial *sebelum region paralel* **if** `seed` ≥ 0 **then**
    `set.seed(seed)` ; `// baris 29`
$\hat{\theta} \leftarrow$ fit awal;
while *replikat valid* $< B$ **do**
  for *replikat yang masih dibutuhkan, sekuensial* **do**
    bangkitkan $u_b^* \sim N(0, \hat{\sigma}^2)$ dan $e_b^* \sim N(0, \psi)$;
  #pragma omp parallel for schedule(dynamic);
  for *replikat paralel* **do**
    $v_b^* \leftarrow (\mathbf{I} - \hat{\rho}W)^{-1}u_b^*$; $\theta_b^* = \mathbf{X}\hat{\beta} + v_b^*$; $y_b^* = \theta_b^* + e_b^*$;
    `refit` Fisher scoring penuh pada y_b^*;
    if *tidak konvergen atau tidak valid* **then**
      tandai tidak valid (diganti draw baru pada putaran berikutnya)
    kembalikan $(\hat{\theta}_b^* - \theta_b^*)^2$

2.6 Objek keluaran dan kelas S3

Semua fungsi model mengembalikan `list` berkelas `fastsae` (atau turunannya) dengan elemen inti yang seragam: `estcoef`, `df_eblup`, `random_effect_var`, `goodness`, `n_iter`, `convergence`, `formula`, `model`, `call`, `data`. Tabel 5 mencatat variasinya.

Tabel 5: Kelas dan elemen khusus objek keluaran. Sumber: kode perakitan output masing-masing fungsi.

Fungsi	`class()`	Elemen khusus
`eblup_fh`	`fastsae`	`goodness` (loglik penuh, AIC, BIC; juga untuk REML)
`eblup_sfh`	`fastsae`	`rho`, `estvarcomp`, `W`
`eblup_stfh`	`fastsae`	`estvarcomp` 4 baris; `g1`–`g4`; `mse/rse = NA` bila `mse_type = "none"`
`eblup_bhf`	`fastsae`, `fastsae_unit`	`fit` (estimasi dan residual), `samp_size`; tanpa `goodness`
`eblup_twofold`	`fastsae`	`level = "subarea"`, `sigma2_v`/`sigma2_u`
`hb_area`	`fastsae_hb_area`, `fastsae_hb`, `fastsae`	`fit` (objek `inla`, atau hasil NumPyro untuk `jax_engine`), `engine`, `hyperpar`
`hb_unit`, `hb_twofold`	`fastsae_hb*`, `fastsae_hb`, `fastsae`	`goodness` berupa `list` (DIC, WAIC, marginal likelihood)

Metode `print`/`summary` (bagian 14) memakai elemen ini. Sejak 1.0.0 ketiga fungsi Bayes berbagi kelas `fastsae_hb` (sebelumnya `hb_area` tidak, HB-16).

2.7 Ringkasan desain

- Inti numerik ada di sepuluh fungsi C++; model Bayes diserahkan ke INLA (atau JAX/NumPyro) tanpa kode C++ tambahan.

- Tidak ada inti yang membentuk invers kovarians penuh: two-fold memakai Sherman–Morrison per blok, SFH Woodbury dengan faktorisasi sparse, STFH Woodbury terstruktur, FH dan BHF statistik per domain.
- Bootstrap diparalelkan dengan OpenMP pada lima titik; RNG dibangkitkan sekuensial agar hasil tidak bergantung thread.
- Kriteria konvergensi seragam (10^{-4}, 100 iterasi); SFH dan STFH memiliki fallback terkendala untuk kasus di batas ruang parameter.

3 Benchmark

Seluruh angka pada bagian ini dihitung ulang oleh `scripts/01_tables.R` dan `scripts/02_figures.R` langsung dari sembilan berkas `inst/extdata/*.rds`. Tidak ada angka yang diketik tangan, dan setiap tabel/figur mencantumkan file sumbernya pada caption. Bila klaim di `README.Rmd` atau vignette bertolak belakang dengan file data, laporan ini memakai data dan mencatat kontradiksinya pada bagian 3.8 (daftar lengkap: `DISCREPANCIES.md` bagian H).

3.1 Metodologi

Apa yang diukur. Semua file benchmark berformat sama: kolom `Method`, `Min (s)`, `Median (s)`, `Iterations/sec`, `Memory (MB)`, `n_iter`, `n`. Dua kolom waktu berasal dari `bench::mark()$min` dan `$median`; `Memory (MB)` = `as.numeric(res$mem_alloc)/1024^2`, yaitu *total alokasi memori selama run*, bukan puncak RSS (lihat `benchmarks/bench_beta_tipsae.R:92-98`). `n_iter` mencatat jumlah iterasi aktual yang benar-benar dijalankan.

Skrip, pembanding, ukuran, iterasi, seed. Tabel 6 disusun dari pembacaan langsung skrip di `benchmarks/`. Baris fastsae pada semua file benchmark diukur ulang dengan fastsae 1.0.0 oleh dua skrip pengukur ulang (`bench_frequentist_fastsae.R`, 13 Okt 2026, dan `bench_bayes_fastsae.R`, 14 Okt 2026); baris pembanding berasal dari run sebelumnya dan dipertahankan.

Apa yang tidak tercatat. Model CPU, jumlah inti, RAM, sistem operasi, tanggal run, versi R pada saat run, dan versi paket pembanding *tidak ada* di dalam file `.rds`. Pengukuran ulang baris fastsae dilakukan pada Apple M4 (10 core), macOS 26.6, R 4.6.1 (`scripts/sessionInfo.txt`); untuk baris pembanding hal itu tidak tercatat. Tidak ada skrip yang mengatur `OMP_NUM_THREADS`. Baris sae/emdi pada tiga file FH dan file `beta_estimates_n1000.rds` *tidak punya skrip pembuat* di repo (terverifikasi dengan `git log -S`); baris fastsae pada ketiga file FH kini dapat direproduksi dengan `bench_frequentist_fastsae.R`, tetapi pada desain yang sama dengan bilangan acak yang *berbeda* dari baris pembanding. Kolom fastsae pada `beta_estimates_n1000.rds` terbukti dapat direproduksi persis (selisih maksimum 1.4×10^{-10}, 14 Okt 2026) dengan `sim_area_data(D = 1000, spatial_type = "knn", seed = 1042)` dan `hb_area(y_beta ~ x1 + x2, family = "beta")`, sehingga konkordansi pada Tabel 20 juga berlaku untuk versi 1.0.0.

Keadilan perbandingan. Pada skrip yang ada, kedua metode menerima *data simulasi yang sama* (`sim_area_data(D = n_val, seed = 42 + n_val)`), formula yang sama, dan ko-

Tabel 6: Skrip benchmark: pembanding, ukuran masalah, iterasi, dan seed. Sumber: pembacaan `benchmarks/*.R` (lihat `_notes/bench_methodology.md`).

Skrip	Pembanding (dari kode)	n	Iterasi / seed
`bench_frequentist_fastsae.R`	hanya baris fastsae (FH, SFH, STFH, MSE analitik); baris sae/emdi dipertahankan	30 – 1000	20; data seed tetap, 5 kovariat, $T = 5$
`bench_bayes_fastsae.R`	hanya baris fastsae (`hb_area`, API `engine`); baris tipsae/hbsae dipertahankan	sama dengan file	10 + 1 pemanasan; data dan seed sama dengan skrip asal
`bench_beta_tipsae.R`	`fastsae::hb_area` vs `tipsae::fit_sae`	250, 500	20; data `seed = 42+n`; tipsae `seed = 42`
`bench_spatial_beta.R`	`fastsae` vs `tipsae` (+ `sf`, `spdep`)	30, 50, 100	20; data `42+n`
`bench_spatio_temporal_beta.R`	`fastsae` vs `tipsae` ($T = 5$)	30 – 1000	5; data `42+n`
`bench_hb_area_engines.R`	`hb_area` dengan `inla_engine()`, `jax_engine()` di CPU dan GPU (Metal/CUDA), dan `tipsae::fit_sae` (Beta); model ICAR	100 – 2500	2 run per fit (pertama + *warm*); data `42+n`; satu proses R per engine
`bench_hb_normal.R`	`fastsae::hb_area` vs `hbsae::fSAE.Area`	30, 50, 100, 250	10; data `42+n`
`update_hb_benchmark.R`	`fastsae` vs `hbsae`	500, 1000	10; data `42+n`
`run_hb_area_full_benchmark.R`	`fastsae` vs `fastsaehb` vs `tipsae` (Stan HMC)	skenario tunggal	`set.seed(123/42)`; tidak menulis file
`run_multipackage_hb_benchmark.R`	`fastsae` vs `hbsae`, `CARBayes`, `fastsaehb`, `tipsae`	—	5 iterasi; tidak menulis file
`compare_ebp_tipsae_truth.R`, `sim_finite_population_beta.R`, `sim_mc_finite_population.R`, `sim_mc_finite_pop_d500.R`	`fastsae` vs `tipsae` vs estimasi langsung (akurasi terhadap kebenaran model)	$D = 60/500$	$R = 100$ (seed 2024) dan $R = 25$ (seed 999); `mclapply` 5–6 core

lom `vardir` yang sama; `bench::mark(..., check = FALSE, filter_gc = FALSE, memory = TRUE, time_unit = "s")`. Perbedaan yang tetap ada dan harus dibaca sebagai *caveat*:

- `tipsae` dijalankan sebagai pemanggilan MCMC/Stan dengan `seed = 42` dan `refresh = 0`, sedangkan fastsae memakai aproksimasi Laplace INLA — dua algoritme yang berbeda secara mendasar, bukan dua implementasi algoritme yang sama.
- `hbsae::fSAE.Area` adalah pemanggilan langsung tanpa penyetelan iterasi; `fastsae::hb_area` memakai default `strategy = "laplace"`.
- Untuk tiga file FH (`eblup`, `seblup`, `steblup`) skrip pembuat baris pembanding tidak ada, sehingga kesetaraan pengaturan awal, kriteria konvergensi, dan jumlah thread tidak dapat diverifikasi dari repo; baris fastsae diukur pada data dengan desain sama tetapi draw berbeda.
- Pengukuran ulang Bayes memakai satu pemanggilan pemanasan sebelum `bench::mark()` (memuat INLA di luar pengukuran); baris pembanding diukur tanpa pemanasan. Pada median 10 iterasi efeknya kecil.
- Beberapa `n_iter` aktual berbeda dari nilai yang diminta (lihat kolom `n_iter` di file), tanda run besar memakai argumen `-iter` yang tidak tercatat di file mana pun.

3.2 Waktu komputasi

Tabel 7–9 memuat waktu median per iterasi untuk tiga keluarga Fay–Herriot frekuentis; Tabel 10 sampai Tabel 13 untuk model beta dan model hierarki Bayes. Pembandingnya: sae dan emdi untuk FH, tipsae untuk beta, hbsae untuk Bayes Gaussian.

Perlu ditegaskan apa yang dimaksud "keluarga beta" di sini: fastsae tidak mengekspor fungsi khusus beta — yang dijalankan adalah `fastsae::hb_area` dengan `family = "beta"`, sedangkan pembandingnya `tipsae::fit_sae` (`benchmarks/bench_beta_tipsae.R:9-10`). Rumus regresi beta yang mendasari pembanding itu dibahas Ferrari and Cribari-Neto [2004] dan Janicki [2020], perluasannya ke SAE area-level oleh Esteban et al. [2019] dan ke campuran beta oleh De Nicolò et al. [2024]; rujukan paketnya De Nicolò and Gardini [2024].

Tabel 7: Waktu median per iterasi (detik) menurut jumlah area n. Sumber: file `eblup_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan.

n	fastsae	emdi	sae
30	0.000549072	0.0108802	0.00155167
50	0.000580786	0.0180643	0.0019507
100	0.000640871	0.0505076	0.00344267
250	0.000676746	0.857347	0.037575
500	0.00106651	5.86366	0.196726
1000	0.00119007	51.0224	1.50363

Tiga pengamatan yang terbaca langsung dari tabel:

1. Pada masalah kecil ($n = 30$) selisih antar paket kecil bahkan untuk FH, sebab biaya tetap (parsing, pembangunan matriks desain) masih dominan.
2. Pada FH area-level, waktu pembanding tumbuh jauh lebih cepat daripada fastsae seiring n; pada n terbesar pada file, emdi membutuhkan waktu dua orde magnitudo lebih besar.

Tabel 8: Waktu median per iterasi (detik) menurut jumlah area n. Sumber: file `seblup_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan.

n	fastsae	emdi	sae
30	0.00587733	0.0107105	0.00410111
50	0.00456215	0.0180005	0.00967977
100	0.0138776	0.05299	0.0705356
250	0.0479088	0.843782	1.07476
500	0.213105	3.13662	7.11793
1000	0.999772	51.0593	67.1518

Tabel 9: Waktu median per iterasi (detik) menurut jumlah area n. Model spasio-temporal dengan T periode waktu; waktu termasuk estimasi komponen varian. Sumber: file `steblup_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan.

n	fastsae	sae
30	0.142723	0.118558
50	0.39381	0.420018
100	0.214152	17.9425
250	1.51391	58.4315
500	10.0435	253.343
1000	75.3342	1786.49

Tabel 10: Waktu median per iterasi (detik) menurut jumlah area n. Model area-level beta (`fastsae::hb_area` vs `tipsae::fit_sae`). Sumber: file `beta_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan.

n	fastsae	tipsae
30	1.20359	2.34867
50	1.22749	3.77451
100	1.26112	8.23587
250	1.31949	26.2762
500	1.40019	64.2141
1000	1.60582	187.162

Tabel 11: Waktu median per iterasi (detik) menurut jumlah area n. Model area-level beta dengan efek spasial. Sumber: file `spatial_beta_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan.

n	fastsae	tipsae
30	1.22266	25.7233
50	1.23069	65.7235
100	1.24499	100.02
250	1.3322	155.898
500	1.45965	248.821
1000	1.86648	404.342

Tabel 12: Waktu median per iterasi (detik) menurut jumlah area n. Model beta spasio-temporal ($T = 5$); data tersedia hanya untuk $n = 30$ dan $n = 50$. Sumber: file `spatio_temporal_beta_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan.

n	fastsae	tipsae
30	1.4761	540.573
50	1.71366	908.173

Tabel 13: Waktu median per iterasi (detik) menurut jumlah area n. Model hierarki Bayes area-level Gaussian (`hb_area`) vs `hbsae::fSAE.Area`. Sumber: file `hb_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan.

n	fastsae	hbsae
30	1.23615	0.0209383
50	1.24031	0.0255576
100	1.24118	0.0373128
250	1.24748	0.0744251
500	1.27122	0.143842
1000	1.29055	0.271585

3. Pada model Bayes Gaussian justru hbsae yang lebih cepat di seluruh n pada file — tidak ada pembanding lain pada skenario ini, dan temuan ini dibahas pada bagian 3.7.

3.3 Penggunaan memori

Tabel 14: Alokasi memori total per iterasi (MB) menurut jumlah area n. Sumber: file `eblup_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan.

n	fastsae	emdi	sae
30	0.339302	4.18307	0.153381
50	0.00930786	10.4676	0.373138
100	0.0177002	37.3394	0.917542
250	0.0428772	240.893	6.34988
500	0.0848389	880.314	18.2391
1000	0.503075	3773.6	71.9765

Kolom `Memory (MB)` adalah total alokasi selama run (bagian 3.1), sehingga angkanya lebih besar daripada puncak RSS yang dilaporkan `bench::mark()` sebagai `mem_usage`. Klaim `README.Rmd:124` ("< 10 MB" untuk fastsae) dan label "Peak Memory" terbantahkan oleh file data pada beberapa baris; rinciannya di bagian 3.8.

3.4 Skala terhadap jumlah area

Gambar 1 dan Gambar 2 menampilkan waktu median terhadap n pada skala log–log untuk seluruh keluarga model; Gambar 3 menampilkan memori; Gambar 4 merangkum rasio waktu pembanding/tes terhadap n.

Tabel 19 merangkum seluruh rasio per n. Rentangnya jauh lebih lebar daripada klaim di `README.Rmd`: pada ujung bawah ada skenario tempat pembanding lebih cepat dari fastsae,

Tabel 15: Alokasi memori total per iterasi (MB) menurut jumlah area n. Sumber: file `seblup_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan.

n	fastsae	emdi	sae
30	0.00878906	4.18307	1.37314
50	0.0116348	10.4676	3.31219
100	0.0211258	37.3394	17.9755
250	0.0508804	240.893	110.388
500	0.100471	949.864	776.407
1000	0.203178	3772.57	1920.01

Tabel 16: Alokasi memori total per iterasi (MB) menurut jumlah area n. Sumber: file `steblup_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan.

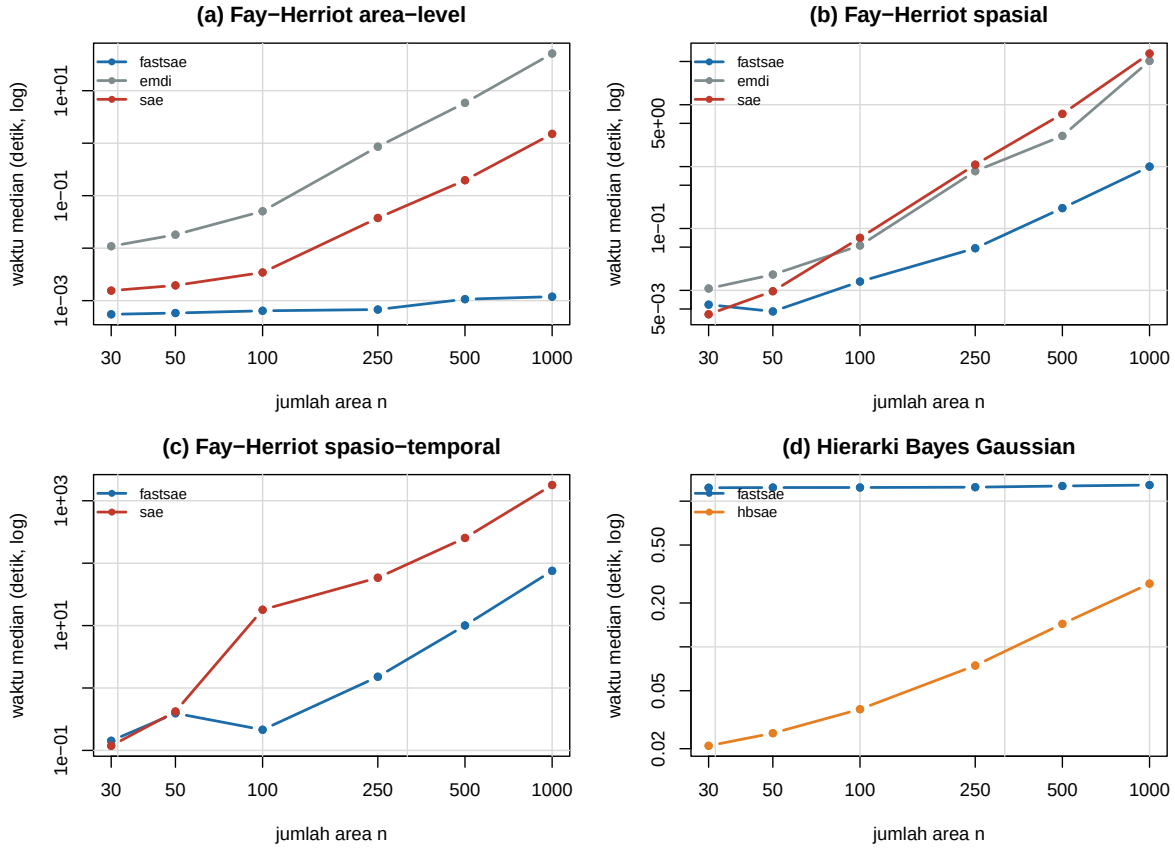
n	fastsae	sae
30	0.137718	43.8007
50	0.16114	107.829
100	0.315681	2551.1
250	0.819832	3624.71
500	1.63306	8124.37
1000	3.29282	32478.3

Tabel 17: Alokasi memori total per iterasi (MB) menurut jumlah area n. Model hierarki Bayes area-level Gaussian. Sumber: file `hb_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan.

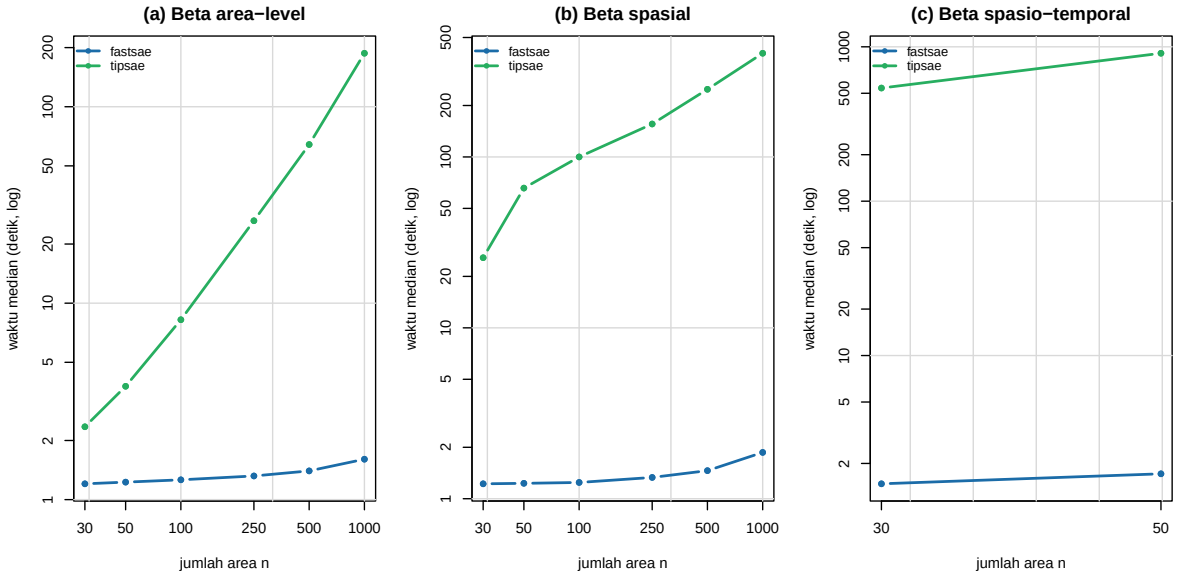
n	fastsae	hbsae
30	2.06081	5.93539
50	2.56881	5.8022
100	3.75748	12.3786
250	7.32253	45.3857
500	13.2684	112.206
1000	25.1609	220.352

Tabel 18: Ringkasan alokasi memori total per iterasi (MB): rerata, minimum dan maksimum lintas seluruh n pada tiap file sumber. Kolom "maks" adalah nilai pada n terbesar pada file tersebut.

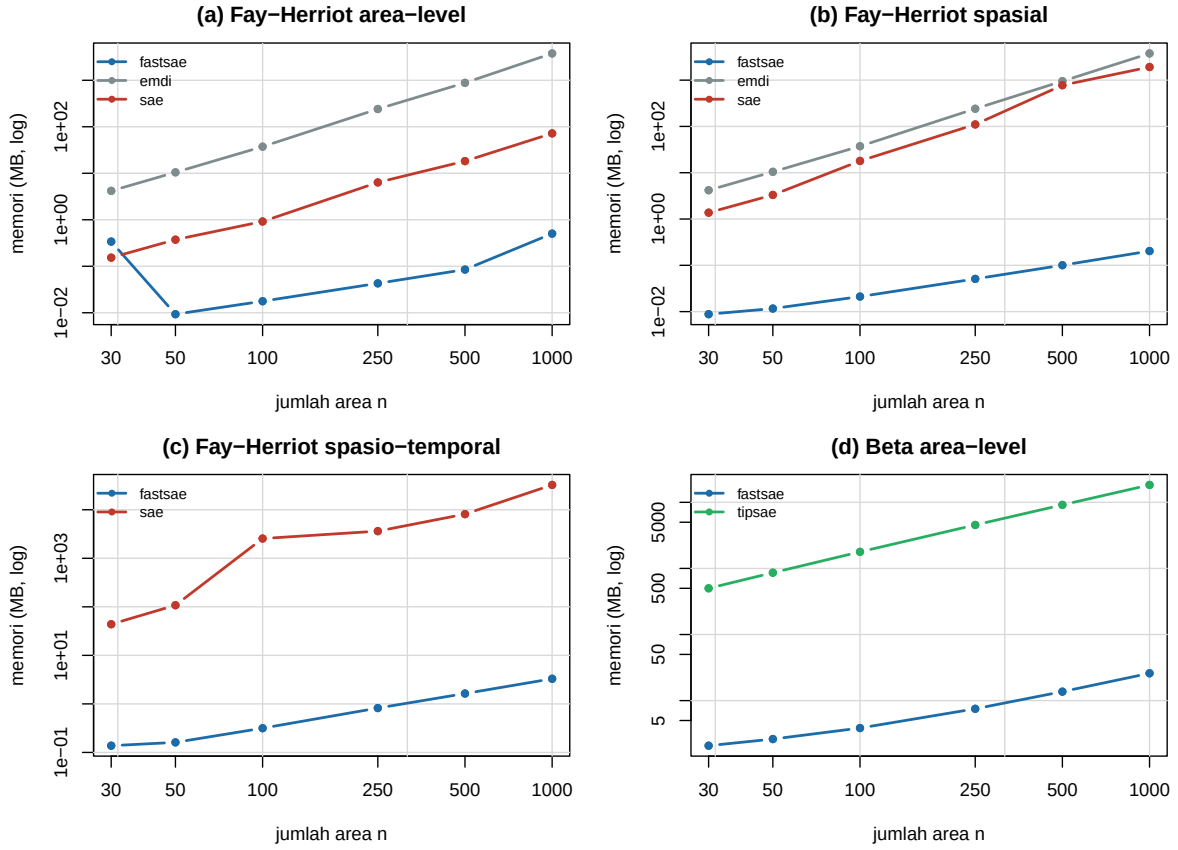
File	Metode	Rerata	Min	Maks
`eblup`	fastsae	0.166183	0.00930786	0.503075
`eblup`	emdi	824.466	4.18307	3773.6
`eblup`	sae	16.3349	0.153381	71.9765
`seblup`	fastsae	0.0660133	0.00878906	0.203178
`seblup`	emdi	835.886	4.18307	3772.57
`seblup`	sae	471.577	1.37314	1920.01
`steblup`	fastsae	1.06004	0.137718	3.29282
`steblup`	sae	7821.69	43.8007	32478.3
`beta`	fastsae	9.27345	2.07975	25.9151
`beta`	tipsae	5860.87	499.97	18335.2
`spatial_beta`	fastsae	39.9873	2.40809	161.075
`spatial_beta`	tipsae	7366.18	642.835	23038.7
`spatio_temporal_beta`	fastsae	13.9882	10.8502	17.1261
`spatio_temporal_beta`	tipsae	3913.6	2947.08	4880.13
`hb`	fastsae	9.02316	2.06081	25.1609
`hb`	hbsae	67.01	5.8022	220.352



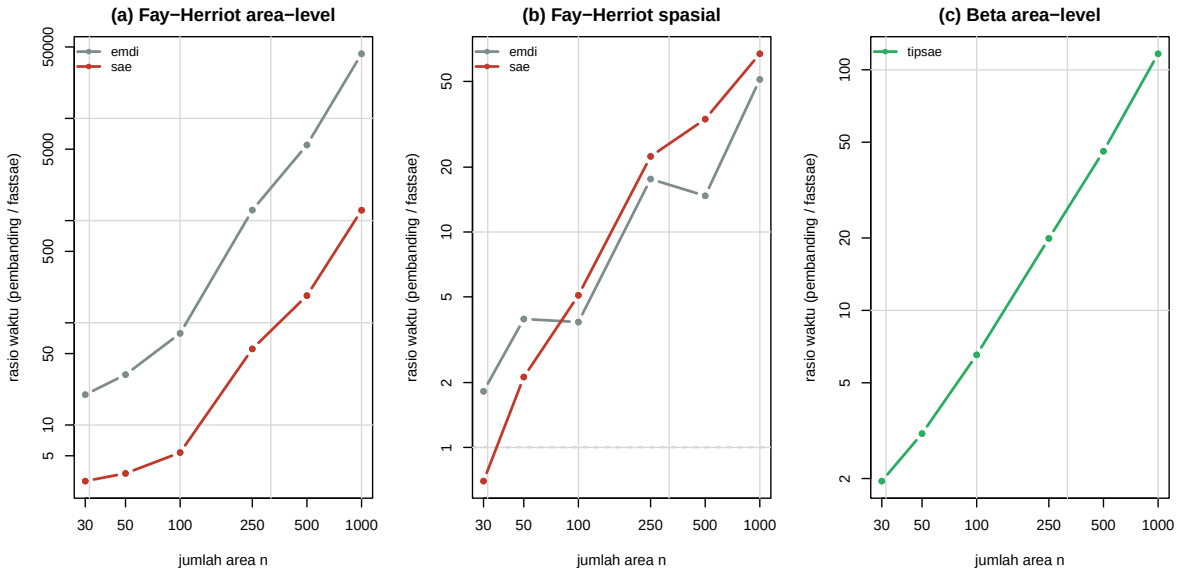
Gambar 1: Waktu median per iterasi terhadap jumlah area n (skala log pada kedua sumbu). Sumber: `inst/extdata/eblup_benchmark.rds`, `seblup_benchmark.rds`, `steblup_benchmark.rds`, `hb_benchmark.rds`; digambar oleh `scripts/02_figures.R`.



Gambar 2: Waktu median untuk tiga varian model beta (skala log–log). Sumber: `inst/extdata/beta_benchmark.rds`, `spatial_beta_benchmark.rds`, `spatio_temporal_beta_benchmark.rds`; `scripts/02_figures.R`. Panel (c) hanya memiliki $n = 30$ dan $n = 50$.



Gambar 3: Alokasi memori total per iterasi terhadap n (skala log–log). Sumber: file `inst/extdata/*_benchmark.rds` sebagaimana tercantum pada tiap panel; `scripts/02_figures.R`.



Gambar 4: Rasio waktu (pembanding/tes) terhadap n; garis putus-putus menandai rasio = 1. Sumber: file `inst/extdata/*_benchmark.rds`; `scripts/02_figures.R`.

dan pada ujung atas rasio mencapai empat orde magnitudo — karenanya laporan memilih menampilkan rentang, bukan satu angka tunggal.

Tabel 19: Rasio kecepatan = median waktu pembanding / median waktu `fastsae` (semakin besar semakin cepat `fastsae`; nilai < 1 berarti pembanding lebih cepat). Rasio dihitung per n lalu diringkas.

Model	Pembanding	n pertama	n terbesar	min	max
FH area-level (`eblup_fh`)	emdi	19.8156	42873.5	19.8156	42873.5
				rata-rata lintas n: 8294.69×	
FH area-level (`eblup_fh`)	sae	2.82598	1263.49	2.82598	1263.49
				rata-rata lintas n: 252.504×	
FH spasial (`eblup_sfh`)	emdi	1.82234	51.071	1.82234	51.071
				rata-rata lintas n: 15.498×	
FH spasial (`eblup_sfh`)	sae	0.697784	67.1671	0.697784	67.1671
				rata-rata lintas n: 21.8173×	
FH spasio-temporal (`eblup_stfh`)	sae	0.830685	23.7143	0.830685	83.784
				rata-rata lintas n: 28.8694×	
Beta area-level (`hb_area`)	tipsae	1.95139	116.552	1.95139	116.552
				rata-rata lintas n: 32.3139×	
Beta spasial	tipsae	21.0388	216.633	21.0388	216.633
				rata-rata lintas n: 109.817×	
Beta spasio-temporal	tipsae	366.216	529.961	366.216	529.961
				rata-rata lintas n: 448.088×	
Hierarki Bayes Gaussian	hbsae	0.0169383	0.210441	0.0169383	0.210441
				rata-rata lintas n: 0.0751434×	

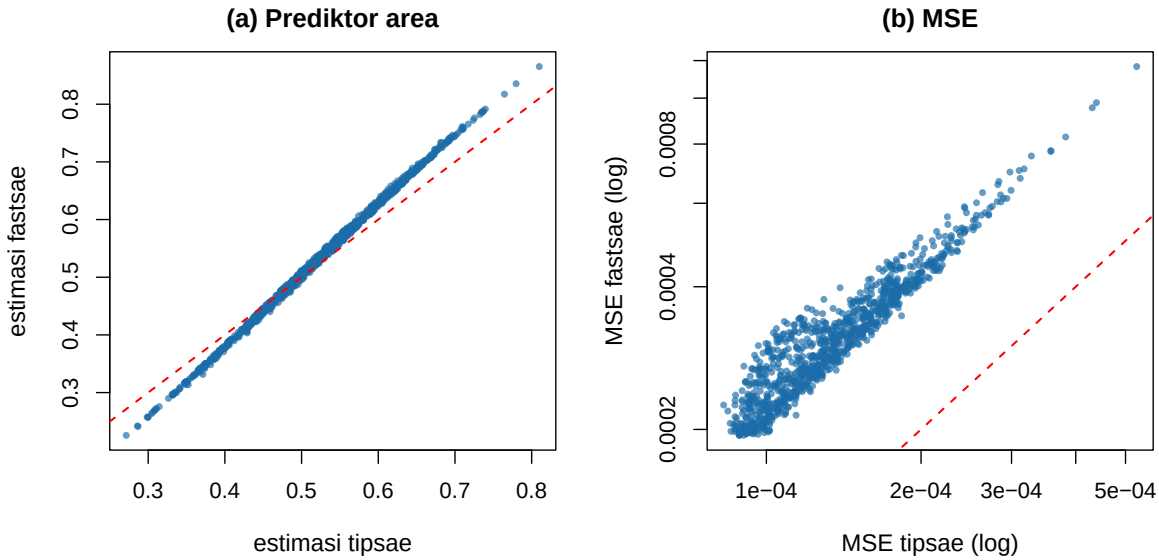
3.5 Ekuivalensi numerik terhadap pembanding

File `beta_estimates_n1000.rds` memuat pasangan estimasi dan MSE untuk 1000 area antara fastsae dan tipsae pada model beta yang sama. Tabel 20 menghitung metrik kesesuaian dari pasangan kolom itu; Gambar 5 menampilkannya secara grafis.

Kedua paket memprediksi indikator yang sama dengan selisih kecil pada prediktor (korelasi sangat tinggi; Tabel 20), sedangkan MSE berbeda lebih nyata karena kedua paket memakai

Tabel 20: Ekuivalensi numerik antara `fastsae` dan `tipsae` pada $n = 1000$ area (indikator beta). Semua metrik dihitung dari pasangan kolom pada file sumber.

Metrik	Nilai
Pearson r (estimasi)	0.999343
Spearman ρ (estimasi)	0.999208
MAE (estimasi)	0.0196916
RMSD (estimasi)	0.0236901
Galat absolut maksimum	0.0558335
Pearson r (MSE)	0.963148
Median selisih mutlak MSE	0.000171686
Median rasio MSE fastsae/tipsae	2.21687



Gambar 5: Ekuivalensi numerik `fastsae` vs `tipsae` untuk $n = 1000$ area: (a) prediktor area terhadap garis identitas; (b) MSE pada skala log. Sumber: `inst/extdata/beta_estimates_n1000.rds`; `scripts/02_figures.R`.

pendekatan ketidakpastian yang berbeda (INLA Laplace vs posterior MCMC). Angka MSE tidak boleh dibaca sebagai "satu paket lebih benar" tanpa rujukan kebenaran; pembanding terhadap kebenaran model tersedia pada skrip `benchmarks/compare_ebp_tipsae_truth.R` dan `sim_mc_finite_population.R` (hasil tersimpan di `benchmarks/output/`, di luar cakupan `inst/extdata`).

3.6 Perbandingan engine `hb_area()` pada model spasial

Skrip `benchmarks/bench_hb_area_engines.R` mengukur `hb_area()` dengan efek area ICAR (`spatial = "besag"`) pada grid beraturan berketetanggaan *queen*. Data dibangkitkan oleh `sim_area_data(D = n, W = W, n_unsampled = 0, seed = 42 + n)`. Model Gaussian (Fay–Herriot spasial) dan Beta diestimasi dengan `inla_engine()`, dengan `jax_engine()` di CPU dan di GPU, serta, untuk model Beta, dengan `tipsae::fit_sae(likelihood = "beta", spatial_error = TRUE)`, yang juga memakai efek ICAR. Setiap engine dijalankan dalam proses R tersendiri karena JAX mengunci perangkatnya per sesi. Tiap fit diulang dua kali pada data yang sama; tabel dan gambar memakai run kedua (*warm*), sedangkan run pertama, yang memuat kompilasi JAX, tersimpan di file sumber. Kesesuaian dihitung terhadap INLA pada mesin dan data yang sama.

Tabel 21: Waktu komputasi (detik, run *warm*) `hb_area()` dengan efek ICAR (Besag) pada grid beraturan: INLA, JAX/NumPyro NUTS di CPU dan di GPU, serta `tipsae` (Stan, hanya model Beta). JAX: 2 rantai × 1000 draw setelah 500 warmup. Sumber: file `hb_engine_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan.

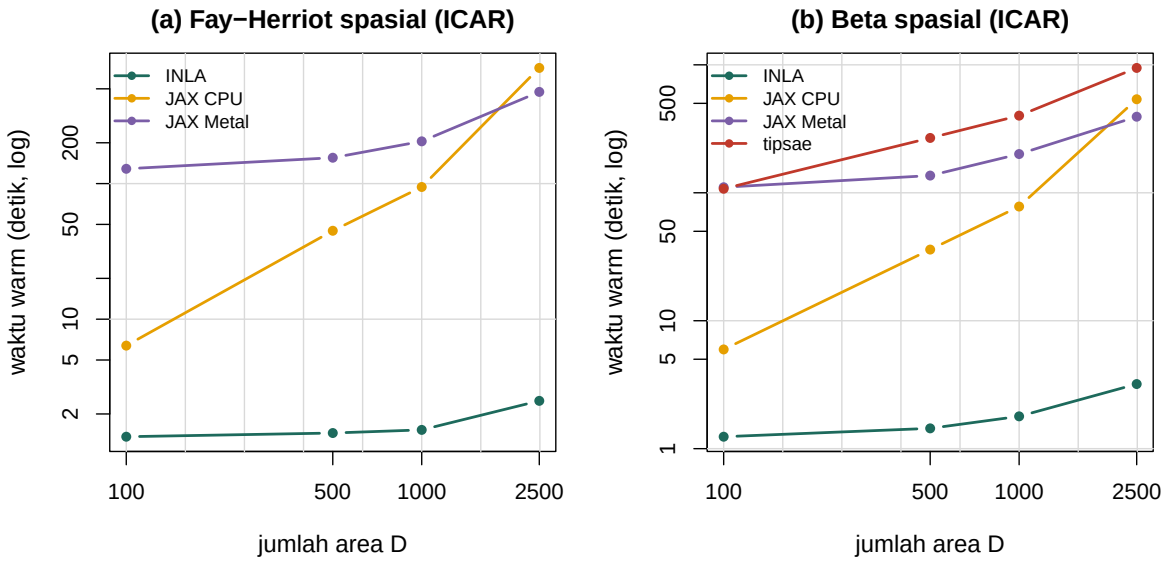
Mesin	Model	D	INLA	JAX CPU	JAX GPU	tipsae
Apple M4	Gaussian	100	1.359	6.383	128.595 (metal)	–
Apple M4	Gaussian	500	1.447	44.889	155.156 (metal)	–
Apple M4	Gaussian	1000	1.526	94.396	204.951 (metal)	–
Apple M4	Gaussian	2500	2.499	712.926	474.448 (metal)	–
Apple M4	Beta	100	1.24	5.965	110.468 (metal)	107.496
Apple M4	Beta	500	1.441	35.982	136.28 (metal)	268.243
Apple M4	Beta	1000	1.789	78.077	200.78 (metal)	400.755
Apple M4	Beta	2500	3.199	538.146	393.048 (metal)	945.842

Tabel 22: Kesesuaian tiap engine terhadap INLA pada mesin dan data yang sama, lintas seluruh D: korelasi minimum rerata posterior, rentang median rasio sd posterior (engine/INLA), dan $\hat{R}$ terpisah (*split*) terbesar. Sumber: file `hb_engine_benchmark.rds` di `inst/extdata` (repo `fastsae`); angka dihitung ulang oleh `scripts/01_tables.R`, ditampilkan hingga 6 digit signifikan.

Mesin	Model	Engine	$\min r$	Rasio sd	$\max \hat{R}$
Apple M4	Gaussian	JAX CPU	0.999974	0.99898–1.01188	1.00452
Apple M4	Gaussian	JAX Metal	0.999986	0.998922–1.01051	1.00426
Apple M4	Beta	JAX CPU	0.999961	0.963412–1.04363	1.00445
Apple M4	Beta	JAX Metal	0.999955	0.968362–1.01872	1.00706
Apple M4	Beta	tipsae	0.993906	0.766323–0.836564	1.08265

Pembacaan Tabel 21, Tabel 22 dan Gambar 6:

- INLA tercepat pada setiap D, dan waktunya hampir tidak bertambah dengan D karena INLA tidak melakukan sampling.



Gambar 6: Waktu *warm* engine `hb_area()` terhadap jumlah area D (skala log) untuk model ICAR: (a) Gaussian, (b) Beta. Sumber: `inst/extdata/hb_engine_benchmark*.rds`; `scripts/02_figures.R`.

- Waktu engine JAX tumbuh cepat dengan D, akibat basis eigen $D \times D$ (bagian 10.4; `.icar_spectral`): $O(D^3)$ sekali, $O(D^2)$ per gradien. Pada mesin uji, R memakai BLAS/LAPACK referensi, sehingga dekomposisi eigen ikut terhitung dalam waktu fit.
- GPU Apple (Metal) memiliki biaya tetap sekitar dua menit per fit: tiap putaran loop pembentukan pohon NUTS menunggu sinkronisasi GPU dan CPU. Akibatnya Metal lebih lambat dari CPU untuk D kecil dan baru lebih cepat pada D terbesar yang diuji.
- Engine JAX di CPU lebih cepat dari tipsae pada semua D, tetapi selisihnya menyempit seiring D membesar.
- Rerata posterior engine JAX praktis identik dengan INLA, di CPU maupun di GPU. tipsae juga sejalan, tetapi sd posteriornya lebih kecil, kemungkinan karena prior kedua paket berbeda. $\hat{R}$ tipsae pada satu ukuran melewati 1,05 dengan panjang rantai bawaannya (Tabel 22).

Hasil GPU NVIDIA (CUDA) dapat direproduksi dengan notebook `benchmarks/kaggle/` atau `benchmarks/colab/`. Halaman artikel pkgdown *hb_area() Engines* memuat versi interaktif perbandingan ini.

3.7 Kaveat dan keadilan

1. **Beda algoritme, bukan beda implementasi.** tipsae adalah Bayes MCMC/Stan [De Nicolò and Gardini, 2024], hbsae memakai pendekatan Bayesnya sendiri, emdi dan sae memakai lme4/aljabar Matrix di R, sedangkan fastsae memakai C++ + INLA. Selisih waktu mencerminkan perbedaan ini sekaligus.
2. **Tanpa penetapan thread.** Tidak ada skrip yang mengunci `OMP_NUM_THREADS`; bootstrap paralel (bagian 2.5) dapat memakai seluruh core mesin, sedangkan pembanding berjalan serial. Ini membuat perbandingan *memihak* ke fastsae pada skala besar, dan dilaporkan apa adanya.

3. **Kriteria konvergensi tidak disamakan.** fastsae memakai toleransi 10^{-4} dan 100 iterasi (Tabel 4); pengaturan pembanding tidak tercatat di file.

4. **Memori bukan puncak.** Kolom memori adalah akumulasi alokasi, sehingga tidak setara dengan *peak RSS* yang lazim dilaporkan.

5. **Baris pembanding tanpa skrip.** Baris sae/emdi pada tiga file FH dan `beta_estimates_n1000` tidak dapat direproduksi dari repo; laporan memakainya apa adanya dan menandainya.

6. **Engine dengan dua run.** Perbandingan engine (bagian 3.6) memakai dua run per fit, bukan `bench::mark` berulang; variasi antar-run pada fit beberapa menit dapat mencapai beberapa persen.

7. **Run inkremental.** Skrip 1–5 menulis file secara inkremental (baris lama per n diganti), sehingga file bisa berisi campuran run dari waktu yang berbeda.

3.8 Kontradiksi klaim dokumentasi terhadap data

Aturan laporan: data menang. Temuan berikut berasal dari perbandingan langsung klaim `README.Rmd`/`vignettes/benchmarks.Rmd`/`jssjournal/article.tex` dengan isi `inst/extdata` (perhitungan lengkap: `_notes/bench_methodology.md` bagian 6 dan 8), sebagaimana ditemukan pada 3 Okt 2026.

Status (14 Okt 2026). DOC-1 sampai DOC-8 dan DOC-10 sudah diperbaiki: tabel dan angka benchmark di `README.Rmd`, beranda situs (`pkgdown/index.md`), `vignettes/benchmarks.Rmd`, dan `vignettes/hb-area-inla.Rmd` kini dihasilkan dari `inst/extdata` oleh `benchmarks/update_benchmark_docs.R`, memori dilabeli sebagai alokasi total (`mem_alloc`), dan grafik ST hanya menampilkan n yang ada di data. DOC-9 berlaku untuk artikel JSS lama (`jssjournal/article.tex`), yang tidak lagi diperbarui. DOC-11 tinggal berlaku untuk baris pembanding.

Implikasinya langsung: laporan ini tidak mengutip satu pun angka kecepatan dari README. Angka yang dipakai seluruhnya berasal dari tabel dan gambar di atas, yang kesemuanya dihasilkan ulang dari file data.

4 Latar statistik

Bagian ini menyajikan perkakas yang dipakai berulang di seluruh model, dengan notasi yang sama dengan yang dipakai kode (mis. nama variabel `vardir`, `sigma2`, `rho`). Derivasi panjang dipindahkan ke lampiran B.

4.1 Tabel notasi

4.2 Model campuran linear, GLS, dan BLUP

Model campuran linear umum

$$y = \mathbf{X}\beta + \mathbf{Z}u + \varepsilon, \qquad \begin{pmatrix} u \\ \varepsilon \end{pmatrix} \sim N\left(0,\ \begin{pmatrix} \mathbf{G} & 0 \\ 0 & \mathbf{R} \end{pmatrix}\right), \tag{6}$$

Tabel 23: Klaim dokumentasi yang bertolak belakang dengan file data. Sumber: `README.Rmd`, `vignettes/benchmarks.Rmd`, `jssjournal/article.tex` vs `inst/extdata/*.rds` (butir DOC-1 – DOC-11 di `DISCREPANCIES.md`).

Butir	Klaim dokumentasi	Fakta dari data
DOC-1	`README.Rmd:118` menandai tabel "Benchmark across $n = 1000$ areas"	angka tabel adalah rata-rata lintas $n = 30 \dots 1000$
DOC-2	`README.Rmd:122` "∼190×" dan "∼6400×"	rasio yang benar tidak cocok dengan salah satunya (lihat Tabel 19)
DOC-3	`README.Rmd:124` fastsae "< 10 MB"	beberapa baris `fastsae` melebihi 10 MB (Tabel 18)
DOC-4	`README.Rmd:124` pembanding "∼400 MB"/"∼800 MB"	campuran dua file; untuk FH angkanya jauh di bawah itu
DOC-5/6	`README.Rmd:34-35` "50 to 6,000×" dan `:87` "10–100×"	rentang teramati lebih lebar di kedua ujung (Tabel 19)
DOC-7	`vignettes/benchmarks.Rmd:39` emdi spasial "8,69 s"	nilai pada file berbeda (Tabel 8)
DOC-8	label "*Peak* Memory" di README, vignette, dan artikel	nilai adalah rerata `mem_alloc`, bukan puncak
DOC-9	`jssjournal/article.tex:100,480` "73× to 544×"	ada nilai serendah 1,887× (Tabel 19)
DOC-10	`vignettes/benchmarks.Rmd:407-414, 634-639` kolom Spatial/ST $n \geq 100$	tidak ada data pendukung di `inst/extdata`
DOC-11	reproduksi hasil	empat file tidak punya skrip pembuat di repo

Tabel 24: Notasi utama dan padanannya dalam kode. Sumber: simbol pada `R/*.R` dan `src/*.cpp`.

Simbol	Nama kode	Arti
y_d atau y_{id}	`y`	pengamatan langsung / respons (baris `NA` menandai domain tak-sampel pada model area-level)
ψ_d	`vardir`, `psi`	varian pengukuran langsung $\psi_d = S_d^2/n_d$; wajib > 0 pada area tersampel
$\mathbf{X}$	`X`, `Xall`	matriks desain ($m \times p$ atau $N \times p$)
$\hat{\beta}$	`beta`	estimasi koefisien fit tetap (GLS)
u_d, θ_d	`random_effect`, `theta`	efek acak domain dan prediktor area $\theta_d = x_d'\beta + u_d$
σ^2	`sigma2`, `sigma2_u`	varians efek acak
$\mathbf{V}$	`V`, `Vi`	kovarians respons dan inversinya
$\mathbf{Q} = (\mathbf{X}'\mathbf{V}^{-1}\mathbf{X})^{-1}$	`Q`	matriks informasi GLS
$w_d = 1/(\psi_d + \sigma^2)$	`w`	bobot FH per area
B_d	`Bd`	faktor pemulusan gaya FH (lihat (14))
$h_d = x_d'\mathbf{Q}x_d$	`h`	leverage prediktor
$\mathbf{W}$	`W`, `proxmat`	matriks tetangga (bobot)
ρ	`rho`	autoregresi spasial pada SFH/STFH
$f_d = n_d/N_d$	`f`	fraksi sampel populasi (BHF, `hb_unit`)
B	`B`	jumlah replikat bootstrap
m, M, D, T	`m`, `D`, `Tt`	jumlah area ter-sampel, total baris, jumlah domain, jumlah periode waktu

memberi kovarians total $\mathbf{V} = \mathbf{Z}\mathbf{G}\mathbf{Z}' + \mathbf{R}$. Dengan β diketahui, prediktor linier tak bias terbaik (BLUP) dari u adalah

$$\tilde{u} = \mathbf{G}\mathbf{Z}'\mathbf{V}^{-1}(y - \mathbf{X}\beta), \qquad \tilde{\theta} = \mathbf{X}\beta + \mathbf{G}\mathbf{Z}'\mathbf{V}^{-1}(y - \mathbf{X}\beta). \tag{7}$$

Apabila β dinilai dengan GLS, $\hat{\beta} = (\mathbf{X}'\mathbf{V}^{-1}\mathbf{X})^{-1}\mathbf{X}'\mathbf{V}^{-1}y$, maka prediktor (7) tetap linier tak bias pada β dan dikenal sebagai *best linear unbiased predictor* [Kackar and Harville, 1984]. Pada fastsae, (7) muncul sebagai `eblup = Xbeta + GVi %*% (y - Xbeta)` pada SFH (`eblup_sfh_core.cpp:462`) dan sebagai kasus khusus pada FH.

Untuk model area-level FH, $\mathbf{Z} = \mathbf{I}$, $\mathbf{G} = \sigma^2\mathbf{I}$, $\mathbf{R} = \mathrm{diag}(\psi_d)$ sehingga $\mathbf{V}$ diagonal dan semua operasi turun ke skalar per area:

$$\hat{u}_d = \frac{\sigma^2}{\sigma^2 + \psi_d}\,(y_d - x_d'\hat{\beta}), \qquad \hat{\theta}_d = x_d'\hat{\beta} + \hat{u}_d. \tag{8}$$

4.3 EBLUP

Empirical BLUP menggantikan komponen varians σ^2 dengan estimanya $\hat{\sigma}^2$:

$$\hat{\theta}_d(\hat{\sigma}^2) = x_d'\hat{\beta}(\hat{\sigma}^2) + \hat{u}_d(\hat{\sigma}^2). \tag{9}$$

Kackar and Harville [1984] menunjukkan bahwa pendekatan orde-kedua menyediakan pendekatan varian prediktor yang lazim dipakai; Prasad and Rao [1990] dan Datta and Lahiri [2000] mengembangkan dekomposisinya menjadi suku-suku g yang dibahas pada bagian 4.6. Konsekuensi praktis pendekatan ini diabaikan oleh Kackar–Harville: penggantian $\sigma^2 \to \hat{\sigma}^2$ menggeser prediktor sedikit ke atas (*bias*); pada FH, fastsae mengoreksi orde-kedua itu *hanya* bila `method = "ML"` (`eblup_fh.cpp:196-199`).

4.4 ML, REML, dan skor

Likelihood biasa untuk FH (satu parametrik) dengan $v_d = \sigma^2 + \psi_d$:

$$\ell(\sigma^2) = -\frac{1}{2}\sum_{d=1}^{m}\left[\log(2\pi v_d) + \frac{r_d^2}{v_d}\right], \qquad r_d = y_d - x_d'\hat{\beta}, \tag{10}$$

dengan $\hat{\beta}$ fungsi GLS dari σ^2. Persamaan skor ML yang dikodekan (`eblup_fh.cpp:103`) adalah

$$s(\sigma^2) = -\frac{1}{2}\sum_d w_d + \frac{1}{2}(Py)'(Py), \qquad P = W - WX\mathbf{Q}X'W, \tag{11}$$

dan informasi Fisher $I(\sigma^2) = \frac{1}{2}\sum_d w_d^2$ (`eblup_fh.cpp:112`). REML memakai kasus spesial

$$s_R(\sigma^2) = -\frac{1}{2}\operatorname{tr} P + \frac{1}{2}(Py)'(Py), \qquad I_R(\sigma^2) = \tfrac{1}{2}\operatorname{tr}(P^2), \tag{12}$$

dengan turunan ringkas yang dikodekan: $\operatorname{tr} P = \sum_d w_d - \operatorname{tr}(\mathbf{Q}\mathbf{X}'W^2\mathbf{X})$ dan $\operatorname{tr}(P^2) = \sum w^2 - 2\sum_d w_d^2(x_d'\mathbf{Q}x_d)w_d + \operatorname{tr}(K^2)$, $K = \mathbf{Q}\mathbf{X}'W^2\mathbf{X}$ (`eblup_fh.cpp:105-117`). Tidak ada inversi matriks $m \times m$ sama sekali pada FH: kedua turunan dihitung dari akumulasi skalar.

Implementation note. Untuk model dengan dua atau lebih parameter (dua-fold, SFH, STFH), persamaan skor ditulis per elemen θ_a dengan pola $s_a = -\frac{1}{2}\operatorname{tr}(M_a) + \frac{1}{2}y'M_a y$ (REML) dan $\mathcal{I}_{ab} = \frac{1}{2}\operatorname{tr}(M_aM_b)$, dengan $M_a = P\,\partial V/\partial\theta_a$ dan $P = V^{-1} - V^{-1}X\mathbf{Q}X'V^{-1}$. Untuk SFH dengan `method="ML"`, kode memakai kuadratik $y'PDPy$ tetapi informasi $\frac{1}{2}\operatorname{tr}(V^{-1}D_aV^{-1}D_b)$ — campuran ML/REML yang didokumentasikan sebagai SFH-1 di `DISCREPANCIES.md`.

4.5 Fisher scoring

Fisher scoring menggantikan Hessian oleh informasi Fisher:

$$\theta^{(k+1)} = \theta^{(k)} + \mathcal{I}(\theta^{(k)})^{-1} s(\theta^{(k)}). \tag{13}$$

Algoritme umum yang diikuti empat inti C++ (lihat Tabel 4 untuk nilai konkretnya):

1. hitung $\mathbf{V}, \mathbf{V}^{-1}, \mathbf{Q}$ pada $\theta^{(k)}$;
2. hitung skor s dan informasi $\mathcal{I}$;
3. langkah $\Delta = \mathcal{I}^{-1} s$ (pada dua-parameter: inversi 2×2; bila gagal, fallback $\Delta_a = s_a / \mathcal{I}_{aa}$);
4. perbarui $\theta \leftarrow \theta + \Delta$; terapkan batas;
5. uji `diff` $= \max_a \left| \Delta_a / \max(\theta_a, 10^{-12}) \right| \le$ `precision`;
6. ulangi maksimum `maxiter` kali, lalu lakukan pass akhir pada $\hat{\theta}$ (menghitung ulang $\mathbf{Q}, \hat{\beta}$, galat baku, prediktor, dan MSE).

Langkah 6 penting: MSE dan galat baku dievaluasi pada $\hat{\theta}$ final, bukan pada iterate terakhir sebelum update (pada STFH, galat baku parameter komponen varians justru dihitung dari $\mathcal{I}^{-1}$ pada θ_k sebelum update terakhir — STFH-7 di `DISCREPANCIES.md`).

4.6 Dekomposisi MSE

Untuk prediktor EBLUP, Prasad and Rao [1990] dan Kackar and Harville [1984] menguraikan galat kuadrat rata-rata ke dalam tiga suku. Untuk FH, fastsae menghitung (`eblup_fh.cpp:183-202`) dengan parametrisasi

$$B_d = \frac{\psi_d}{\psi_d + \hat{\sigma}^2}, \qquad g_1 = \psi_d (1 - B_d), \qquad g_2 = B_d^2 \, h_d, \qquad g_3 = B_d^2 \, \frac{\mathrm{Var}(\hat{\sigma}^2)}{\hat{\sigma}^2 + \psi_d}, \tag{14}$$

dengan $\mathrm{Var}(\hat{\sigma}^2) = 2 / \sum_d w_d^2$ (varian asimptotik $\hat{\sigma}^2$), $h_d = x_d' \mathbf{Q} x_d$. Lalu

$$\widehat{\mathrm{MSE}}(\hat{\theta}_d) = \begin{cases} g_1 + g_2 + 2g_3, & \text{REML}, \\ g_1 + g_2 + 2g_3 - b \, B_d^2, & \text{ML}, \end{cases} \qquad b = -\frac{\mathrm{tr}(\mathbf{Q}\mathbf{X}' W^2 \mathbf{X})}{\max(\sum_d w_d^2, 10^{-30})}. \tag{15}$$

Implementation note. B_d pada (14) adalah komplemen dari $B_d = \hat{\sigma}^2 / (\hat{\sigma}^2 + \psi_d)$ yang lazim dipakai Rao–Molina: kode menulis $B_d = \psi_d / (\psi_d + \hat{\sigma}^2)$ (FH-1). Bentuk $g_1 = \psi_d (1 - B_d)$ ekuatip dengan $(B_d^{\mathrm{RM}})^2 \psi_d + (1 - B_d^{\mathrm{RM}})^2 \hat{\sigma}^2$ (FH-2). Jangan tertukar dengan $(1 - B)^2 (\psi_d + \hat{\sigma}^2)$ yang kadang muncul di literatur — bentuk itu bukan yang dikodekan.

Untuk model dua-parameter (dua-fold) varians $\mathrm{Var}(\hat{\sigma}_v^2)$, $\mathrm{Var}(\hat{\sigma}_u^2)$, dan Cov diambil dari invers matriks informasi Fisher final (ML atau REML sesuai `method`); suku silang $2\mathbb{E}_{vu} \, \mathrm{Cov}_{vu}$ termasuk di dalam g_3, dan estimatornya MSE $= g_1 + g_2 + 2g_3$ (REML), dikurangi $\mathbf{b}_{\mathrm{ML}}' \nabla g_1$ untuk ML (bagian 9.5; sebelum audit 12 Okt 2026: $g_1 + g_2 + g_3$, TFH-2). Untuk SFH dan STFH, empat suku dipakai: MSE $= g_1 + g_2 + 2g_3 - g_4$, pada SFH ditambah koreksi bias ML pada g_1 (bagian 6.5 dan 7.5).

4.7 MSE bootstrap

Dua jenis tersedia di paket:

Parametrik. Dari hasil fit $\hat{\beta}, \hat{\sigma}^2$, bangkitkan $y^* = \mathbf{X}\hat{\beta} + u^* + \varepsilon^*$ dengan u^*, ε^* dari distribusi model, refit, lalu

$$\widehat{\mathrm{MSE}}_{\mathrm{pb}} = \frac{1}{B}\sum_{b=1}^{B}(\hat{\theta}_b^* - \theta_b^*)^2. \tag{16}$$

Ini persis `eblup_twofold.cpp:593-594`, `eblup_stfh_pbmse.cpp:209-221`, `eblup_sfh_pbmse.cpp:166`, dan `eblup_bhf_core.cpp:300-303`; SFH dan STFH mengganti replikat gagal sampai tepat B replikat valid, BHF membagi dengan jumlah replikat yang berhasil. Varian bias-koreksi pada SFH memakai

$$\widehat{\mathrm{MSE}}_{\mathrm{pbbc}} = 2(g_1 + g_2) - g_1^* - g_2^* + g_3^*, \tag{17}$$

dengan g_3^* diambil dari selisih $\hat{\theta}^*$ terhadap prediksi SBLUP yang bobotnya berasal dari *fit awal* (`eblup_sfh_pbmse.cpp:148, 167-172`); kesesuaian (17) dengan rumus publik tidak terikat pada referensi mana pun di repo dan ditandai tidak pasti (SFH-10).

Nonparametrik. Hanya untuk SFH dan hanya REML (`eblup_sfh_npbmse.cpp:31-34`): residual dibagi menjadi komponen efek acak dan sampling lewat akar kuadrat matriks $\Psi_e = \mathrm{diag}(\psi) P \,\mathrm{diag}(\psi)$ dan $\Psi_u = \mathrm{Gen}^{-1}\, G\, P\, G\, \mathrm{Gen}^{-1\prime}$ yang dieigendekomposisi (kolom $p..m-1$ dipakai untuk membuang arah proyeksi $\mathbf{X}$), lalu resample indeks residual yang telah distandarisasi (`eblup_sfh_npbmse.cpp:71-140`). Generatornya $\mathrm{Gen}^{-1} = \mathbf{I} - \rho W$ bila semua domain tersampel (seperti `sae::npbmseSFH`); bila ada domain tak-sampel, Gen adalah faktor Cholesky bawah dari Σ_{ss}. Dengan $\sigma^2 = 0$ tidak ada efek area yang diresample.

4.8 Domain tanpa sampel dan prediksi sintetis

Tabel 25 merangkum perilaku tiap fungsi terhadap domain tanpa pengamatan. Definisi "tanpa sampel" berbeda antar fungsi, dan perbedaan itu penting bagi pengguna.

Tabel 25: Penanganan domain tanpa sampel. Sumber: cabang kode pada kolom terakhir.

Fungsi	Kriteria	Prediktor dan MSE
`eblup_fh`	`y = NA`	$x'_{ns}\hat{\beta}$; MSE $= \hat{\sigma}^2 + x'_{ns}\mathbf{Q}x_{ns}$; `random_effect = NA` (`eblup_fh.cpp:207-222`)
`eblup_sfh`	`y = NA`	kriging $x'_{ns}\hat{\beta} + \sigma^2\Sigma_{rs}\mathbf{V}^{-1}r$ dari kovarians SAR penuh yang sama dengan fit; MSE orde kedua penuh (`eblup_sfh_core.cpp:641-756`)
`eblup_stfh`	respons `NA`	*tidak diizinkan*: panel wajib seimbang, `cli_abort` (`R/eblup_stfh.R:168-176`)
`eblup_bhf`	domain tanpa unit sampel	prediktor sintetis $\bar{x}'_{pop}\hat{\beta}$, `samp_size = 0`, peringatan (`eblup_unit.cpp:94-100`)
`eblup_twofold`	`y = NA`	area tanpa subarea: $x'\hat{\beta}$; subarea tak-sampel pada area tersampel: $x'\hat{\beta} + \hat{v}_i$ (`eblup_twofold.cpp:356-395`)
`hb_unit`	seluruh domain	menambahkan 1 baris `y = NA` per domain, lalu memakai fitted INLA (`hb_unit.R:244-278`)
`hb_area`, `hb_twofold`	baris `y = NA`	tetap dievaluasi oleh INLA; domain yang *tidak ada dalam data* tidak diprediksi (HB-17)

4.9 Benchmarking: pengertian internal dan eksternal

Dalam literatur SAE, *benchmarking* berarti mengkalibrasi prediktor model ke total atau agregat yang diketahui sehingga konsistensi antar tingkat kerapian terpenuhi [Rao and Molina, 2015].

Dua varian muncul di paket ini:

- **Benchmarking eksternal** memakai total populasi yang diketahui dari sumber luar survei (mis. total penduduk per provinsi).
- **Benchmarking internal** menggunakannya sebagai *validasi*: `diagnose()` membandingkan total prediksi terhadap total yang diketahui dan menandai status `CONSISTENT` bila selisihnya di bawah toleransi (`benchmark.R:602`).

Empat metode kalibrasi (`ratio`, `difference`, `optimal`, `logit`) beserta pendukung hierarkinya dijelaskan pada bagian 14; di sini cukup dicatat bahwa seluruhnya bekerja pada vektor prediktor + MSE dan bobot populasi, sehingga dapat dipakai pada hasil model mana pun yang mengembalikan `df_eblup`/`df_hb`.

5 Model Fay–Herriot area-level: `eblup_fh`

5.1 Setting data dan kapan dipakai

Data berupa satu baris per domain: estimasi langsung y_d (mis. rerata tahun sekolah per kabupaten), varians pengukurannya ψ_d (`vardir`), dan kovariat x_d berdimensi p (jumlah sekolah, dll.). Model ini tepat ketika: (i) survei menghasilkan estimasi langsung beserta variansnya, (ii) domain sangat kecil sampai y_d tidak stabil, dan (iii) ada kovariat yang tersedia untuk semua domain termasuk yang tidak tersampel. Data contoh `mys` (42 kabupaten, 10 di antaranya tanpa sampel) persis berbentuk ini. Model ini adalah titik masuk klasik SAE [Fay and Herriot, 1979] dan dibahas lengkap di Rao and Molina [2015].

5.2 Persamaan model

$$y_d = x_d'\beta + u_d + e_d, \qquad u_d \overset{\text{iid}}{\sim} N(0, \sigma^2), \qquad e_d \sim N(0, \psi_d), \tag{18}$$

dengan β tanpa prior (dihilangkan lewat GLS) dan e_d saling bebas. Varians total per area $v_d = \sigma^2 + \psi_d$ dan bobot $w_d = 1/v_d$ muncul di kode sebagai dasar seluruh turunan (`eblup_fh.cpp:89`). Dalam notasi blok, $\mathbf{V} = \mathrm{diag}(v_1, \ldots, v_m)$.

Domain tanpa sampel ditandai *bukan* lewat indeks melainkan lewat `y = NA`: baris dengan `y` non-finite dipisahkan lebih dulu dan $\mathbf{X}$-nya disimpan sebagai `Xns` (`eblup_fh.cpp:33-47`). Argumen `domain` hanya pelabelan kolom (`R/eblup_fh.R:230`).

5.3 Prediktor titik

Dengan $\hat\beta = (\mathbf{X}'W\mathbf{X})^{-1}\mathbf{X}'Wy$ (`eblup_fh.cpp:135-143`), prediktor area adalah EBLUP

$$\hat\theta_d = x_d'\hat\beta + \hat u_d, \qquad \hat u_d = \frac{\hat\sigma^2}{\hat\sigma^2 + \psi_d}(y_d - x_d'\hat\beta), \tag{19}$$

yang ekuivalen dengan pemulusan y_d terhadap prediktor sintetis:

$$\hat\theta_d = B_d\,(x_d'\hat\beta) + (1 - B_d)\,y_d, \qquad B_d = \frac{\psi_d}{\psi_d + \hat\sigma^2}. \tag{20}$$

Pada domain tanpa sampel predikturnya jatuh ke prediktor sintetis murni (`eblup_fh.cpp:207-222`):

$$\hat\theta_{ns} = x_{ns}'\hat\beta, \qquad \widehat{\mathrm{MSE}}_{ns} = \hat\sigma^2 + x_{ns}'\mathbf{Q}x_{ns}, \qquad \texttt{random_effect} = \texttt{NA}. \tag{21}$$

5.4 Estimasi komponen varians

Algoritme Fisher scoring persis seperti bagian 4.5 dengan skor ML (11) dan skor REML (12). Rekam jejak lengkapnya:

1. **Nilai awal:** $\hat{\sigma}^{2(0)} = \max(\text{median}(\psi_d), 0)$ (`eblup_fh.cpp:64-65`).
2. **Loop:** `while (diff > precision && k < maxiter)` (`baris 88`), `precision` $= 10^{-4}$, `maxiter` $= 100$.
3. **Per iterasi:** W, $\mathbf{Q} = (\mathbf{X}'W\mathbf{X})^{-1}$ lewat `inv_sympd` dengan fallback `solve` (`baris 95-96`), $Py = Wy - \mathbf{XQX}'Wy$ (`baris 98-99`).
4. **Update:** bila informasi $I \leq 0$ nilainya diganti `std::numeric_limits<double>::min()` *sebelum* dipakai membagi (`baris 120`), sehingga langkah menjadi raksasa lalu hasil non-finite atau negatif *dipaksa* 0 (`baris 121-123`).
5. **Konvergensi:** `diff` $= \left|(\hat{\sigma}^{2\text{new}} - \hat{\sigma}^2)/\max(\hat{\sigma}^2, 10^{-12})\right|$ (`baris 125`); status `convergence` dan `n_iter` dikembalikan, dan wrapper R memperingati bila gagal (`R/eblup_fh.R:255-258`).
6. **Pass akhir:** hitung ulang $\mathbf{Q}$, $\hat{\beta}$, galat baku `se = sqrt(diag(Q))`, z, dan p dua sisi pada $\hat{\sigma}^2$ final (`baris 135-154`).

Log-likelihood, AIC, dan BIC dihitung dari (10) dengan m = jumlah area tersampel, AIC $= -2\ell + 2(p+1)$, BIC $= -2\ell + (p+1)\log m$ (`baris 170-174`).

Implementation note. Loglik, AIC, dan BIC sengaja berbentuk ML meski `method = "REML"` (FH-3), sama dengan sae::eblupFH dan kriteria Marhuenda, Morales & Pardo (2014). Likelihood terbatas bergantung pada $\mathbf{X}$, sehingga tidak dapat membandingkan model dengan kovariat berbeda; karena itu likelihood terbatas tidak dilaporkan, sama seperti di sae. Koreksi bias orde-kedua $-b\,B_d^2$ hanya dipakai untuk ML (FH-4) — perilaku yang sama dengan sae::mseFH. Kolom `estcoef` memuat dua kolom galat baku yang identik (`std.error` dan `stderr_beta`; FH-5), dan `vardir` wajib positif ketat pada area tersampel (FH-6), yang tidak diperiksa sae.

5.5 Estimasi MSE

MSE analitik (15) adalah satu-satunya metode di `eblup_fh()` — tidak ada pilihan bootstrap. RSE dilaporkan sebagai RSE $= 100\sqrt{\text{MSE}}/\left|\hat{\theta}\right|$ dan menjadi `NaN` bila $\hat{\theta} = 0$ (`baris 227-230`). Pada domain tanpa sampel dipakai (21) tanpa suku g_3, sesuai Prasad and Rao [1990] untuk prediktor yang tidak memakai data domain.

Varian log (Log-FH). Dengan `transform = "log"` (`R/eblup_fh.R:139-147, 195-227`) model dipasang pada skala log: $y_d^* = \log y_d$ dengan $\psi_d^* = \psi_d / y_d^2$ (metode delta; $y_d > 0$ wajib). Estimasi dikembalikan ke skala asal dengan koreksi rerata bersyarat model log-normal [tanpa suku $O(m^{-1})$ dari Slud and Maiti, 2006]:

$$\hat{\theta}_d = \exp(\hat{\mu}_d^* + \tfrac{1}{2}\hat{\sigma}^2(1 - \hat{\gamma}_d)), \qquad \hat{\gamma}_d = \frac{\hat{\sigma}^2}{\hat{\sigma}^2 + \psi_d^*}, \qquad \widehat{\text{MSE}}_d \approx \hat{\theta}_d^2\, \widehat{\text{MSE}}(\hat{\mu}_d^*), \tag{22}$$

dengan $\hat{\gamma}_d = 0$ pada domain tanpa sampel (prediktor sintetis). Kolom `eblup_log` dan `mse_log` menyimpan nilai skala log, dan `model = "Log-FH"`. Bila dipadukan dengan `self_benchmark`, benchmarking internal tidak dilakukan lewat model teraugmentasi (yang hanya eksak di skala log), melainkan dengan *ratio benchmarking* pada skala asal ($\text{MSE}_{bm} = c_g^2\,\text{MSE}$).

5.6 Pseudo-code

5.7 Signature, argumen, objek keluaran, dan contoh

Listing 3: Signature `eblup_fh` (`R/eblup_fh.R:98-112`).

```
eblup_fh(formula, vardir, domain = NULL, data,
         method = c("REML", "ML"), transform = c("none", "log"),
         mse_type = c("analytical", "none"),
         maxiter = 100, precision = 1e-4, print_result = TRUE,
         self_benchmark = FALSE, benchmark_weight = NULL,
         benchmark_target = NULL, benchmark_group = NULL)
```

- `formula` — respons $\sim$ kovariat; respons boleh berisi `NA` untuk domain tak-sampel (`na.action = na.pass`).
- `vardir` — nama kolom, formula satu sisi, vektor numerik (resolver `.get_variable`), atau objek hasil `gvf_smooth()` (dipakai `smooth_vardir`-nya).
- `transform` — `"none"` (default) atau `"log"` (Log-FH, (22)).
- `domain` — kolom/faktor label domain (bukan penanda sampel).
- `method` — `"REML"` (default) atau `"ML"`.
- `maxiter`, `precision` — parameter Fisher scoring.
- `print_result` — mencetak `print.fastsae`.
- `self_benchmark`, `benchmark_*` — benchmarking internal (bagian 14).

Objek keluaran: `list` berkelas `"fastsae"` berisi `estcoef` (beta, dua kolom galat baku, z, p), `random_effect_var` ($\hat{\sigma}^2$), `goodness` (loglik ML, AIC, BIC), `n_iter`, `convergence`, `df_eblup` dengan kolom `domain, y, eblup, vardir, random_effect, mse, rse` (Log-FH: + `eblup_log, mse_log`), plus `formula`, `model` (`"FH"` atau `"Log-FH"`), `transform`, `call`, `data`.

Listing 4: Contoh yang dapat dijalankan (dari `man/eblup_fh.Rd`; data `mys` berisi 42 domain, 10 di antaranya tanpa sampel).

```
library(fastsae)
m1 ← eblup_fh(y ~ x1 + x2 + x3, data = mys, vardir = "vardir")
head(m1$df_eblup, 3)
m1$random_effect_var       # sigma^2
m1$goodness                # loglik, AIC, BIC

# metode ML dan kontrol iterasi
m2 ← eblup_fh(y ~ x1 + x2 + x3, data = mys, vardir = "vardir",
              method = "ML", maxiter = 50, precision = 1e-6)
```

5.8 Referensi kunci

Fay and Herriot [1979] memperkenalkan model area-level ini; Rao and Molina [2015] menyajikan EBLUP dan dekomposisi MSE-nya; Prasad and Rao [1990] dan Kackar and Harville [1984] memberi kerangka suku g dan pendekatan orde-kedua; Datta and Lahiri [2000] menyajikan ukuran ketidakpastian terpadu; Molina and Marhuenda [2015] adalah referensi implementasi paket `sae` yang dipakai sebagai pembanding numerik. Transformasi log mengikuti Slud and Maiti [2006].

Algorithm 2:

Masukan *formula, vardir, domain, data, method* $\in \{REML, ML\}$, *maxiter, precision* **Keluaran** *df_eblup (prediktor, MSE, RSE), estcoef, goodness* ;
// *Lapis R* -- R/eblup_fh.R

method, transform $\leftarrow$ match.arg; // baris 113-114

domain $\leftarrow$ 1:nrow(data) bila NULL (116–120);

mf $\leftarrow$ model.frame(*formula, na.action = na.pass*) (123);

$\psi \leftarrow$ vardir (atau smooth_vardir dari gvf_smooth()) (125–133);

if *transform = "log"* **then**

  $y \leftarrow \log y$, $\psi \leftarrow \psi / y^2$ (139–147)

if *self_benchmark* **then**

  augmentasi model (skala identitas) (154–171)

$\mathbf{X} \leftarrow$ model.matrix; **gagal** bila anyNA($\mathbf{X}$) (173–178);

$\hat{\theta}, \widehat{\text{MSE}}, \ldots \leftarrow$ eblup_core($\mathbf{X}, y, \psi$, *method, maxiter, precision*) (180–187);

if *transform = "log"* **then**

  transformasi balik (22) (195–227)

sisipkan domain ke df_eblup; class $\leftarrow$ "fastsae" (229–253);

if *!convergence* **then**

  warning (.warn_no_convergence) dan kembalikan

print(*objek*) bila print_result;
;
// *Lapis C++* -- src/eblup_fh.cpp

$I_s \leftarrow$find_nonfinite(y); $I_{ns} \leftarrow$find_finite(y) ; // 33-47

gagal bila $m = 0$ atau ada $\psi_d \le 0$ pada area tersampel (53–59);

$\hat{\sigma}^2 \leftarrow \max(\text{median}(\psi), 0)$;

$k \leftarrow 0$; diff $\leftarrow$ precision $+ 1$;

while *diff* $>$ *precision* ***dan*** $k <$ *maxiter* **do**

  $W \leftarrow \text{diag}(w)$; $\mathbf{Q} \leftarrow (\mathbf{X}'W\mathbf{X})^{-1}$; $Py \leftarrow Wy - \mathbf{XQX}'Wy$ (89–99);

  if *method* $= ML$ **then**

    $s \leftarrow -\frac{1}{2}\sum_d w_d + \frac{1}{2}(Py)'(Py)$; $I \leftarrow \frac{1}{2}\sum_d w_d^2$ (103, 112)

  else

    $s \leftarrow -\frac{1}{2}\,\text{tr}\,P + \frac{1}{2}(Py)'(Py)$; $I \leftarrow \frac{1}{2}\,\text{tr}(P^2)$ (105–117)

  ;

  bila $I \le 0$ maka $I \leftarrow$ numeric_limits::min() (120);

  $\hat{\sigma}^{2\text{new}} \leftarrow \hat{\sigma}^2 + s/I$ (122);

  bila $\hat{\sigma}^{2\text{new}} < 0$ atau non-finite $\rightarrow \hat{\sigma}^{2\text{new}} \leftarrow 0$ (123);

  diff $\leftarrow \left|(\hat{\sigma}^{2\text{new}} - \hat{\sigma}^2)/\max(\hat{\sigma}^2, 10^{-12})\right|$ (125); $\hat{\sigma}^2 \leftarrow \hat{\sigma}^{2\text{new}}$ (126); $k \mathrel{+}= 1$ (127);

hitung ulang $\mathbf{Q}, \hat{\beta}$, se, z, p pada $\hat{\sigma}^2$ final (135–154);

$\hat{\theta}_d \leftarrow x_d'\hat{\beta} + \hat{u}_d$; loglik ML, AIC, BIC (159–180);

$g_1, g_2, g_3 \leftarrow$ (14); $\widehat{\text{MSE}} \leftarrow g_1 + g_2 + 2g_3$ (REML); pada ML dikurangi suku bias $b\,B_d^2$ (183–202);

for $d \in I_{ns}$ **do**

  $\hat{\theta}_d \leftarrow x_{ns}'\hat{\beta}$; $\widehat{\text{MSE}} \leftarrow \hat{\sigma}^2 + x_{ns}'\mathbf{Q}x_{ns}$; $\hat{u}_d \leftarrow$ NA (207–222)

RSE $\leftarrow 100\sqrt{\text{MSE}}/\left|\hat{\theta}\right|$; susun df_eblup dan estcoef (227–261);

6 Model Fay–Herriot spasial: `eblup_sfh`

6.1 Setting data dan kapan dipakai

Bentuk datanya sama dengan FH (satu baris per domain: y_d, ψ_d, kovariat x_d) tetapi ditambah satu objek wajib: matriks tetangga W berdimensi $D \times D$ *termasuk* baris/kolom domain tanpa sampel (`R/eblup_sfh.R:199-214`); W yang sudah dipotong ke domain tersampel ditolak. Bila `data` berkelas `sf` dan `W` tidak diberikan, W queen row-standar dibangun otomatis lewat `create_weights()` (baris 199-203). Model ini dipakai bila residual mengandung korelasi spasial, misalnya karena faktor geografis yang tidak terukur; ekuivalennya adalah model *spatial error*/SAR pada data area-level [Pratesi and Salvati, 2008, Rao and Molina, 2015]. `spatial_test()` (bagian 14) menguji autokorelasi itu sebelum model dipilih.

6.2 Persamaan model

$$y_d = x_d'\beta + v_d + e_d, \qquad v = (\mathbf{I} - \rho\mathbf{W})^{-1}u,\ u \sim N(0, \sigma^2\mathbf{I}_D), \qquad e_d \sim N(0, \psi_d), \tag{23}$$

dengan efek SAR didefinisikan pada *semua* D domain dan matriks presisi

$$A = (\mathbf{I} - \rho\mathbf{W}')(\mathbf{I} - \rho\mathbf{W}) = \mathbf{I} - \rho(\mathbf{W} + \mathbf{W}') + \rho^2\mathbf{W}'\mathbf{W}, \qquad |\rho| < 1, \tag{24}$$

sehingga $\mathrm{Var}(v) = \sigma^2\Sigma$, $\Sigma = A^{-1}$; dalam arti ini efek spasialnya *SAR/error-components*, bukan CAR proper (perbandingan kedua notasi ada pada bagian 13.3). Domain tersampel s (m buah) memiliki kovarians marginal $\sigma^2\Sigma_{ss}$ — *bukan* kovarians SAR yang dibangun dari sub-matriks W_{ss}. Dengan E matriks seleksi $m \times D$ dan $\Psi = \mathrm{diag}(\psi_s)$,

$$\mathbf{V} = \sigma^2 E\Sigma E' + \Psi, \qquad \frac{\partial \mathbf{V}}{\partial \sigma^2} = \Sigma_{ss}, \qquad \frac{\partial \mathbf{V}}{\partial \rho} = -\sigma^2\big(\Sigma C \Sigma\big)_{ss}, \quad C = \frac{\partial A}{\partial \rho} = 2\rho\,\mathbf{W}'\mathbf{W} - (\mathbf{W} + \mathbf{W}'). \tag{25}$$

Faktorisasi tanpa invers $D \times D$. Semua besaran fit dan MSE diperoleh dari *solve* dengan A dan $M = A + \sigma^2 E'\Psi^{-1}E$ (`src/sar_engine.h`, `src/eblup_sfh_core.cpp:74-85`):

$$\begin{aligned} \mathbf{V}^{-1} &= \Psi^{-1} - \sigma^2\Psi^{-1}EM^{-1}E'\Psi^{-1} \quad \text{(Woodbury)}, & \mathbf{V}^{-1}E\Sigma &= \Psi^{-1}EM^{-1}, \\ \log|\mathbf{V}| &= \log|\Psi| + \log|M| - \log|A|\,, & G - G\mathbf{V}^{-1}G &= \sigma^2(M^{-1})_{ss}. \end{aligned} \tag{26}$$

Bila pola $\mathbf{I} + W + W' + W'W$ jarang (non-nol $\le 25\%$), A dan M difaktorkan dengan LDL′ sparse Eigen (urutan AMD, analisis simbolik sekali); selain itu dengan LLT/LDLT padat. Tidak ada invers $D \times D$ maupun perkalian berordo D^3.

6.3 Prediktor titik

Dengan $Q = (\mathbf{X}'\mathbf{V}^{-1}\mathbf{X})^{-1}$ dan $\hat\beta = Q\mathbf{X}'\mathbf{V}^{-1}y$, prediktor adalah EBLUP matriks persis (7):

$$\hat\theta_s = \mathbf{X}\hat\beta + G\mathbf{V}^{-1}(y - \mathbf{X}\hat\beta), \qquad G\mathbf{V}^{-1} = \sigma^2(M^{-1})_{ss}\Psi^{-1} \quad (\texttt{eblup_sfh_core.cpp:488-497}). \tag{27}$$

Pada domain tanpa sampel prediktor adalah *kriging simpel* dari kovarians SAR penuh yang sama dengan fit (baris 641-660):

$$\hat v_r = \sigma^2\Sigma_{rs}\,\mathbf{V}^{-1}(y_s - \mathbf{X}_s\hat\beta), \qquad \hat\theta_r = x_r'\hat\beta + \hat v_r. \tag{28}$$

6.4 Estimasi komponen varians

Parameter yang diiterasikan: $\theta = (\sigma^2, \rho)$. Tahap pertama adalah Fisher scoring klasik seperti `sae::eblupSFH` (`sfh_fit`, baris 259-327): nilai awal $\sigma^{2(0)} = \text{median}(\psi)$, $\rho^{(0)} = 0.5$, skor dan informasi

$$s_a = -\tfrac{1}{2}\operatorname{tr}(KD_a) + \tfrac{1}{2}y'PD_aPy, \qquad \mathcal{I}_{ab} = \tfrac{1}{2}\operatorname{tr}(KD_aKD_b), \qquad K = \begin{cases} P & \text{REML} \\ \mathbf{V}^{-1} & \text{ML} \end{cases} \tag{29}$$

dengan $D_1 = \Sigma_{ss}$, $D_2 = \partial\mathbf{V}/\partial\rho$; langkah $\Delta = \mathcal{I}^{-1}s$, ρ dipotong ke $[-0.999, 0.999]$, berhenti bila $\max_a |\Delta\theta_a/(\theta_a + 10^{-12})| \le$ `precision` (10^{-4}) atau setelah `maxiter` (100) iterasi.

Bila iterasi klasik gagal atau berakhir dengan $\sigma^2 < 0$ (yang oleh `sae` hanya dipotong ke 0, sehingga ρ dan β tidak optimal untuk $\sigma^2 = 0$), dijalankan *fallback* terkendala `sfh_fit_safeguarded` (baris 187-257): Fisher scoring terproyeksi dengan himpunan aktif ($\sigma^2 \ge 0$, $|\rho| \le 0.999$) dan *step halving* pada log-likelihood (terbatas). Solusi interior iterasi klasik tidak berubah.

Batas akhir: $\rho = -0.999$ dipetakan ke -1 seperti `sae`, *hanya* bila $(\mathbf{I} + W')(\mathbf{I} + W)$ tidak singular (-1 bukan nilai eigen W, yang gagal misalnya pada grid rook bipartit); batas atas tidak pernah dipetakan ke $+1$ (`sfh_lower_rho`, baris 58-72). Galat baku σ^2, ρ berasal dari informasi sesuai `method` (REML: P, ML: $\mathbf{V}^{-1}$); log-likelihood pada `goodness` adalah log-likelihood ML di estimasi akhir (baris 527-532).

6.5 Estimasi MSE

Empat suku Prasad–Rao dihitung lewat (26), tanpa invers $D \times D$ (baris 540-616):

$$g_1 = \operatorname{diag}(G - G\mathbf{V}^{-1}G) = \sigma^2 \operatorname{diag}(M^{-1})_{ss}, \tag{30}$$

$$g_2 = \operatorname{diag}(R\,Q\,R'), \qquad R = \mathbf{X} - G\mathbf{V}^{-1}\mathbf{X}, \tag{31}$$

$$g_3(i) = \psi_i^2 \sum_{a,b} \mathcal{I}^{ab} \left(\mathbf{V}^{-1}D_a\mathbf{V}^{-1}D_b\mathbf{V}^{-1}\right)_{ii}, \tag{32}$$

$$g_4(i) = \tfrac{1}{2}\sum_{j,k} \mathcal{I}^{jk}\psi_i^2 \left(\mathbf{V}^{-1}\mathbf{V}_{jk}\mathbf{V}^{-1}\right)_{ii}, \tag{33}$$

dengan $\mathbf{V}_{\sigma^2\rho} = -(\Sigma C \Sigma)_{ss}$ dan $\mathbf{V}_{\rho\rho} = 2\sigma^2(\Sigma C \Sigma C \Sigma - \Sigma W' W \Sigma)_{ss}$. Lalu

$$\widehat{\text{MSE}}(\hat{\theta}_d) = g_1 + g_2 + 2g_3 - g_4, \qquad \text{dan bila } \texttt{method = "ML"}\text{: } \widehat{\text{MSE}} \mathrel{-}= \mathbf{b}'_{\text{ML}}\nabla g_1, \tag{34}$$

dengan $\mathbf{b}_{\text{ML}} = \tfrac{1}{2}\mathcal{I}_{\text{ML}}^{-1}h$, $h_a = -\operatorname{tr}(Q\mathbf{X}'\mathbf{V}^{-1}D_a\mathbf{V}^{-1}\mathbf{X})$. Untuk ML dipakai informasi ML (`sae` memakai informasi REML; keduanya setara asimtotik, tetapi saat $\sigma^2 \approx 0$ dan $\rho \approx 1$ hanya informasi ML yang meniadakan suku h dan ∇g_1 yang membesar). Bila ρ berada di batas ($|\rho| = 0.999$ atau -1), ekspansi dalam ρ tidak sah: ρ diperlakukan tetap dan hanya blok σ^2 informasi yang diinvers (galat baku ρ menjadi `NA`).

Area tak-tersampel. Kriging (28) mendapat MSE orde kedua penuh (baris 662-750): g_1 dan g_2 kriging, g_3 dari turunan eksak prediktor kriging terhadap θ, dan koreksi bias g_1 plug-in (turunan pertama dan kedua $G_{rr} - G_{rs}\mathbf{V}^{-1}G_{sr}$), ditambah koreksi ML bila `method = "ML"`. Untuk domain tersampel rumus ini mereproduksi $g_1 + g_2 + 2g_3 - g_4$ sampai $o(m^{-1})$.

Varian bootstrap. Dua pilihan tambahan lewat `mse_type` (sebelum 1.0.0: `mse_method`, kini *deprecated*); keduanya memakai data penuh dan W penuh, sehingga fit, replikat, dan kriging memakai model yang sama:

- `"pbmse"`: bootstrap parametrik (16) — replikat $y_s^* = \mathbf{X}_s\hat{\beta} + v_s^* + e^*$, $v^* = (\mathbf{I} - \hat{\rho}W)^{-1}u^*$ pada semua domain; refit Fisher scoring penuh per replikat dalam region OpenMP (algoritme 1); skema batch menjamin tepat B replikat valid (`eblup_sfh_pbmse.cpp:68-160`). Versi bias-koreksi (17) dihitung bersamaan [Molina et al., 2009].
- `"npbmse"`: bootstrap nonparametrik atas residual terstandarisasi; galat sampling dan efek acak dipisahkan lewat eigen-dekomposisi, lalu indeks residual diresample; hanya untuk `method = "REML"` (`eblup_sfh_npbmse.cpp:31-34`; SFH-9). Generator efek acak domain tersampel adalah $(\mathbf{I} - \rho W)^{-1}$ bila semua domain tersampel (seperti `sae::npbmseSFH`), selain itu faktor Cholesky dari Σ_{ss}.

Kolom bootstrap (`mse_pb`/`mse_pbbc` atau `mse_npb`/`mse_npbbc`) bernilai `NA` untuk area tak-tersampel; `mse`/`rse` mereka berasal dari kriging analitik (`R/eblup_sfh.R:220-237`).

6.6 Pseudo-code

6.7 Signature, argumen, objek keluaran, dan contoh

Listing 5: Signature `eblup_sfh` (`R/eblup_sfh.R:131-149`).

```
eblup_sfh(formula, vardir, domain = NULL, data,
          method = c("REML", "ML"),
          mse_type = c("analytical", "pbmse", "npbmse", "none"),
          W = NULL, B = 100, n_threads = 1, seed = -1,
          maxiter = 100, precision = 1e-4,
          self_benchmark = FALSE, benchmark_weight = NULL,
          benchmark_target = NULL, benchmark_group = NULL,
          print_result = TRUE, mse_method)  # mse_method: deprecated
```

- `W` — matriks tetangga $D \times D$ pada semua domain (baris berjumlah 1, seperti di `sae`; baris nol atau baris yang tidak berjumlah 1 diberi peringatan); wajib kecuali `data` berkelas `sf`.
- `mse_type` — `"analytical"` (default), `"pbmse"`, `"npbmse"`, atau `"none"` (`mse = NA`).
- `B` — jumlah replikat bootstrap (default 100); `n_threads` — > 0 memakai `omp_set_num_threads`, ≤ 0 memakai semua core; `seed = -1` tidak menyetel ulang benih (`eblup_sfh_pbmse.cpp:29`), sehingga draw bootstrap memakai RNG R yang sedang berjalan.
- `method` — REML (default) atau ML.
- `self_benchmark`, `benchmark_*` — benchmarking internal (bagian 14).

Objek keluaran: `"fastsae"` dengan `estcoef` (`beta, std.error, stderr_beta, zvalue, pvalue`), `random_effect_var`, `rho`, `estvarcomp` (2 baris: `sigma2_u`, `rho`, dengan galat baku dari informasi sesuai `method`), `goodness` (`loglikelihood, AIC, BIC`), `df_eblup` (`domain, y, vardir, eblup, random_effect, mse, rse` + kolom bootstrap), `W`, `n_iter`, `convergence`, `method`, `model = "SFH"`, `level = "area"`.

Listing 6: Contoh yang dapat dijalankan (dari `man/eblup_sfh.Rd`).

```
library(fastsae)
m1 ← eblup_sfh(y ~ x1 + x2 + x3, data = mys, vardir = ~vardir,
               W = mys_proxmat)
m1$rho                          # autoregressive spatial parameter
m1$estvarcomp                   # sigma2_u and rho with std.error
```

Algorithm 3:

Masukan *formula, `vardir`, `data`, W, `method`, `mse_type`, `B`, `n_threads`, `seed`*
Keluaran *`df_eblup` (+ kolom bootstrap bila ada), `rho`, `estvarcomp`, `goodness`* ;
// Lapis R
`method`, `mse_type` ← `match.arg` (150-151);
`y` ← respons (`na.pass`); `NA` pada kovariat atau $\psi_s \le 0 \to$ gagal (159-197);
if *`W` kosong* **then**
  `data sf`: `create_weights(queen, "W")`; selain itu gagal (199-206)
gagal bila W bukan $D \times D$ (208-214);
if *`mse_type`* $\in$ *{pbmse, npbmse}* **then**
  `.seblup_pbmse`/`.seblup_npbmse` pada data dan W penuh (220-237)
else
  `.seblup_core`$(\mathbf{X}, y, \psi, W$`, method, maxiter, precision)` (239-247)
;
;
// Lapis C++
$s \leftarrow$ `find_finite`(y), $r \leftarrow$ `find_nonfinite`(y) (414-435);
`SarEngine`: pola sparse $\mathbf{I} + W + W' + W'W$, analisis simbolik sekali (459);
$\sigma^2 \leftarrow \text{median}(\psi)$; $\rho \leftarrow 0.5$;
while *`diff`* $>$ *`precision`* ***dan*** $k <$ *`maxiter`* **do**
  faktorkan $A(\rho)$ dan M; $\mathbf{V}^{-1}, Q, P$ lewat (26);
  $s, \mathcal{I} \leftarrow$ (29); $\Delta \leftarrow \mathcal{I}^{-1}s$; bila gagal: hentikan;
  $\rho \leftarrow \text{clamp}(\rho + \Delta_\rho, -0.999, 0.999)$; $\sigma^2 \leftarrow \sigma^2 + \Delta_\sigma$;
  `diff` $\leftarrow \max_a \left|\Delta\theta_a/(\theta_a + 10^{-12})\right|$;
if *tidak konvergen* ***atau*** $\sigma^2 < 0$ **then**
  fallback terkendala `sfh_fit_safeguarded` (187-257)
$\rho \leftarrow$ `sfh_lower_rho`(ρ, W); $\sigma^2 \leftarrow \max(0, \sigma^2)$ (461-468);
faktorkan ulang; $\hat{\beta}$, galat baku, z, p (normal); $\hat{\theta}_s \leftarrow$ (27) (473-525);
loglik ML, AIC, BIC (527-538);
$g_1, g_2, g_3, g_4 \leftarrow$ (30)–(33); MSE $\leftarrow g_1 + g_2 + 2g_3 - g_4$ (540-616);
if *`method`* $=$ *ML* **then**
  koreksi MSE $-= \mathbf{b}'_{\text{ML}} \nabla g_1$ (619-639)
;
if *ada domain r* **then**
  kriging (28) dan MSE orde kedua (641-756)
;
RSE $\leftarrow 100\sqrt{\text{MSE}}/\left|\hat{\theta}\right|$; susun keluaran (768-818);

```
m2 ← eblup_sfh(y ~ x1 + x2 + x3, data = mys, vardir = ~vardir,
               W = mys_proxmat, mse_type = "pbmse", B = 50)
```

6.8 Referensi kunci

Pratesi and Salvati [2008] mengembangkan FH spasial; Rao and Molina [2015] menyajikan kerangka EBLUP yang dipakai untuk semua suku g; blok `@references` fungsi ini hanya memuat Rao and Molina [2015] (`R/eblup_sfh.R`). Referensi pembanding implementasi: `sae` [Molina and Marhuenda, 2015].

7 Model Fay–Herriot spasio-temporal: `eblup_stfh`

7.1 Setting data dan kapan dipakai

Data berisi satu baris per (domain, waktu) untuk *seluruh* grid $D \times T$, dengan periode yang sama di setiap domain (pemeriksaan baris, `R/eblup_stfh.R:284-296`). Respons boleh `NA`: sel seperti itu adalah sel tanpa estimasi langsung, baik periode yang hilang pada domain tersampel maupun domain yang tidak pernah tersampel. Sel tersebut tidak dibuang tetapi diprediksi (bagian 7.3). Variabel bantu wajib tersedia di setiap sel (baris 262-267), dan `vardir` wajib positif di sel tersampel (baris 274-276). W harus mencakup semua domain, termasuk yang tidak pernah tersampel. Inti C++ membutuhkan urutan *domain-major*; data diurutkan secara internal (domain menurut kemunculan pertama, yang juga harus menjadi urutan baris W; waktu menaik) dan hasilnya dikembalikan ke urutan baris pemanggil (baris 204-250, 379-391). Model ini dipakai bila estimasi tersedia untuk beberapa periode per domain dan kita ingin memakai kedua jenis korelasi: spasial antar domain dan temporal antar waktu dalam domain yang sama. Rujukan utamanya Marhuenda et al. [2013]; implementasinya mengikuti `sae::eblupSTFH`, yang hanya menerima panel lengkap.

7.2 Persamaan model

$$y_{dt} = \theta_{dt} + e_{dt}, \qquad \theta_{dt} = x'_{dt}\beta + u_d^{(1)} + u_{dt}^{(2)}, \qquad e_{dt} \sim N(0, \psi_{dt}), \tag{35}$$

dengan dua komponen yang didefinisikan pada *seluruh* grid (penulisan *domain-major*, $Z = \mathbf{I}_D \otimes \mathbf{1}_T$):

$$u^{(1)} \sim N\big(0,\ \sigma_1^2 \Omega_1\big), \qquad \Omega_1 = \big[(\mathbf{I} - \rho_1 \mathbf{W})'(\mathbf{I} - \rho_1 \mathbf{W})\big]^{-1}, \tag{36}$$

$$u^{(2)} \sim N\big(0,\ \mathbf{I}_D \otimes \sigma_2^2 \Omega_2\big), \qquad (\Omega_2)_{tt'} = \frac{\rho_2^{|t-t'|}}{1-\rho_2^2}. \tag{37}$$

Hanya sel tersampel S (yaitu y_{dt} tidak `NA`) yang mempunyai y_{dt}. Misalkan S_d himpunan periode tersampel domain d dan $n = |S|$. Maka

$$\mathbf{V}_S = A + Z_S\, \sigma_1^2 \Omega_1 Z_S', \qquad A = \operatorname{blockdiag}_d\big(\sigma_2^2\, \Omega_2[S_d, S_d] + \operatorname{diag}(\psi_d)\big), \tag{38}$$

dengan blok kosong untuk domain tanpa sampel. Efek spasial *konstan sepanjang waktu* dalam satu domain, dan $u^{(2)}$ adalah proses AR(1) per domain. Pada panel lengkap S mencakup seluruh grid dan (38) menjadi bentuk biasa dengan $M = DT$.

Implementation note. Struktur ini *bukan* salah satu interaksi Knorr–Held tipe I–IV: tidak ada interaksi spasial×temporal di mana efek spasial berubah tiap waktu. Model `model = "S"`

mematikan struktur temporal ($\Omega_2 = \mathbf{I}$, A diagonal). Perbedaan ini tercatat sebagai STFH-1 dan harus dipahami saat membaca hasil.

Evaluasi terstruktur, tanpa matriks $n \times n$. Dengan Woodbury (`src/eblup_stfh_core.cpp:104-121`)

$$\begin{aligned} \mathbf{V}^{-1} &= A^{-1} - (A^{-1}Z)\,\mathbf{C}^{-1}(A^{-1}Z)', \qquad \mathbf{C} = (\sigma_1^2\Omega_1)^{-1} + Z'A^{-1}Z, \\ P &= A^{-1} - U\,\mathrm{blockdiag}(\mathbf{C}^{-1}, Q)\,U', \end{aligned} \tag{39}$$

dengan $U = [A^{-1}Z,\ \mathbf{V}^{-1}\mathbf{X}]$ $(n \times (D+p))$ dan $Z = Z_S$. Turunan $\mathbf{V}_a$ berbentuk ZB_aZ' (σ_1^2, ρ_1) atau $\mathrm{blockdiag}_d(E_a[S_d, S_d])$ (σ_2^2, ρ_2), sehingga skor, informasi Fisher, galat baku, dan MSE hanya membutuhkan blok $D \times D$, $(D+p) \times (D+p)$, dan $|S_d| \times |S_d|$: biaya $O(D^3 + nT(D+p))$ per iterasi alih-alih $O(n^3)$, dan memori $O(n(D+p))$ alih-alih $O(n^2)$. Hasilnya identik secara aljabar dengan perhitungan padat. Log-determinan memakai lemma determinan $\log|\mathbf{V}| = \log|A| + \log|\sigma_1^2\Omega_1| + \log|\mathbf{C}|$. Indeks per domain (baris awal di data ringkas dan periode tersampel) dibangun oleh `st_make_index` (baris 83-101). Matriks $T \times T$ yang diterapkan per blok diiris menjadi $[S_d, S_d]$; pada panel lengkap tidak ada pengirisan dan aritmetikanya sama persis dengan versi panel lengkap sebelumnya.

7.3 Prediktor titik

Dengan $r = y_S - \mathbf{X}_S\hat{\beta}$, kedua komponen acak diprediksi dari proyeksi spasial dan temporalnya (baris 1055-1078):

$$\hat{u}^{(1)} = \sigma_1^2\Omega_1\,Z'\mathbf{V}^{-1}r, \qquad \hat{u}_d^{(2)} = \sigma_2^2\,\Omega_2[\,\cdot\,, S_d](\mathbf{V}^{-1}r)_d, \qquad \hat{\theta}_{dt} = x_{dt}'\hat{\beta} + \hat{u}_d^{(1)} + \hat{u}_{dt}^{(2)}. \tag{40}$$

$\hat{u}^{(1)}$ identik pada seluruh periode satu domain. Untuk `model = "S"` suku temporal menjadi $\sigma_2^2(\mathbf{V}^{-1}r)$ di sel tersampel.

Sel tanpa estimasi langsung. (40) berlaku untuk setiap sel $i = (d, t)$ pada grid, tersampel atau tidak: ia adalah BLUP dari θ_i berdasarkan data tersampel [Saei and Chambers, 2005], yaitu

$$\tilde{\theta}_i = x_i'\hat{\beta} + a_i'\mathbf{V}^{-1}(y_S - \mathbf{X}_S\hat{\beta}), \qquad a_i = \mathrm{Cov}(\theta_i, y_S) = \sigma_1^2\,\Omega_1[d, d(k)] + \mathbf{1}\{d(k) = d\}\,\sigma_2^2\,\Omega_2[t, t(k)], \tag{41}$$

untuk baris tersampel k dengan domain $d(k)$ dan periode $t(k)$ (`st_full_u`, baris 672-697; baris 1373-1398). Periode yang hilang pada domain tersampel meminjam kekuatan dari periode lain domain itu (AR(1)) dan dari tetangganya (SAR). Domain yang tidak pernah tersampel hanya meminjam dari tetangga, karena $u^{(2)}$ independen antar domain. Ini adalah *kriging* spasial, seperti pada model SAR Fay–Herriot dengan area di luar sampel [Benedetti et al., 2026]. Untuk `model = "S"`, $u^{(2)}$ sel yang hilang independen dari data sehingga hanya bagian spasial yang dipinjam. Asumsinya *missing at random*: pola sel tersampel tidak bergantung pada θ.

7.4 Estimasi komponen varians

Parameter: $\theta = (\sigma_1^2, \rho_1, \sigma_2^2, \rho_2)$ untuk `"ST"` dan $(\sigma_1^2, \rho_1, \sigma_2^2)$ untuk `"S"`. Nilai awal default $0.5\,\mathrm{median}(\psi_S)$ untuk varians dan 0.5 untuk korelasi (baris 874-878), semua dapat dioverride lewat argumen `*_start`. Skoring selalu berbentuk REML (`method` di-hardcode `"REML"`,

`R/eblup_stfh.R:371`) pada sel tersampel, yaitu likelihood marginal dengan sel yang hilang diintegrasikan keluar:

$$S_a = -\tfrac{1}{2}\operatorname{tr}(PV_a) + \tfrac{1}{2} y_S' P V_a P y_S, \qquad F_{ab} = \tfrac{1}{2}\operatorname{tr}(PV_aPV_b), \tag{42}$$

dihitung lewat (39). Langkah $F^{-1}S$, ρ_1, ρ_2 dipetakan ke ± 0.999 bila $|\rho| \geq 1$, konvergensi relatif $\leq 10^{-4}$ dengan maksimum 100 iterasi (baris 921-953) — persis iterasi `sae::eblupSTFH`.

Bila iterasi klasik gagal (biasanya karena iterasi varians menjadi negatif dan $\mathbf{V}$ tak-definit saat $\rho_1 \to 1$) atau konvergen ke varians negatif, dijalankan *fallback* REML terkendala `stfh_fit_safeguarded` (baris 557-668, 955-984): Fisher scoring terproyeksi pada $\sigma^2_{1,2} \geq 0$, $|\rho| \leq 0.999$, parameter di batas aktif ditahan (termasuk korelasi dari varians yang bernilai 0, yang tidak teridentifikasi), dan *step halving* sampai log-likelihood REML tidak turun. Fit yang konvergen pada iterasi klasik tidak berubah.

Galat baku β dari $\sqrt{\operatorname{diag}(Q)}$; nilai p dari distribusi normal (kolom `tvalue` disalin ke `zvalue` di R, STFH-6). Galat baku komponen varians mengikuti `sae::eblupSTFH` (invers informasi iterasi klasik terakhir, STFH-7) bila iterasi itu konvergen, dan dari informasi di estimasi akhir bila fallback dipakai; parameter yang tidak teridentifikasi di batas mendapat `NA` (baris 1120-1158).

7.5 Estimasi MSE

Jenis MSE dipilih lewat `mse_type`: `"analytical"` (default sejak 1.0.0), `"bootstrap"`, atau `"none"` (kolom `mse` dan `rse` diisi `NA`; STFH-2). Argumen `compute_mse` dari 0.1.0 masih diterima dengan peringatan *deprecation*: `FALSE` berarti `"none"` dan `TRUE` berarti `"bootstrap"` bila `B` diberikan, selain itu `"analytical"`.

Analitik (default), sel tersampel. MSE Prasad–Rao orde kedua [Marhuenda et al., 2013] dalam bentuk terstruktur (baris 1161-1370):

$$g_1 = \psi - \psi^2 \operatorname{diag}(\mathbf{V}^{-1}), \qquad g_2 = \psi^2 \operatorname{diag}(\mathbf{V}^{-1}\mathbf{X}Q\mathbf{X}'\mathbf{V}^{-1}), \tag{43}$$

$$g_3(i) = \psi_i^2 \sum_{a,b} F^{ab}\,(PV_aPV_bP)_{ii}, \qquad g_4(i) = \tfrac{1}{2}\sum_{j,k} F^{jk}\psi_i^2\,(PV_{jk}P)_{ii}, \tag{44}$$

$\widehat{\mathrm{MSE}} = g_1 + g_2 + 2g_3 - g_4$. Bentuk g_3 diturunkan dari $\tilde{\theta}(\theta) = y - \Psi P(\theta) y$; ia setara dengan bentuk buku teks $\psi_i^2 \sum F^{ab}(\mathbf{V}^{-1}V_a\mathbf{V}^{-1}\mathbf{V}\mathbf{V}^{-1}V_b\mathbf{V}^{-1})_{ii}$ sampai $o(m^{-1})$, tetapi tidak terpengaruh komponen V_a yang berada di ruang kolom $\mathbf{X}$ (yang meledak pada bentuk buku teks saat $\rho_1 \to 1$ dengan W row-standar). g_4 menangkap kelengkungan $\mathbf{V}$ dalam ρ_1, ρ_2 (turunan kedua V_{jk}). Korelasi di batas ($|\rho| \geq 0.999$) diperlakukan tetap, seperti pada SFH. Komponen g_1–g_4 juga dikembalikan sebagai elemen objek.

Analitik, sel tanpa estimasi langsung. Untuk prediktor (41) dengan $R_i = x_i' - a_i'\mathbf{V}^{-1}\mathbf{X}$ (baris 1396-1500):

$$\begin{aligned} g_1(i) &= \operatorname{Var}(\theta_i - x_i'\beta) - a_i'\mathbf{V}^{-1}a_i, \qquad g_2(i) = R_i Q R_i', \\ g_3(i) &= \sum_{a,b} F^{ab}\,\ell_a' P \ell_b, \qquad \ell_a' = \partial_a a_i' - (a_i'\mathbf{V}^{-1} + R_i Q \mathbf{X}'\mathbf{V}^{-1})V_a, \\ \widehat{\mathrm{MSE}}(i) &= g_1 + g_2 + g_3, \end{aligned} \tag{45}$$

dengan $\operatorname{Var}(\theta_i - x_i'\beta) = \sigma_1^2\Omega_1[d,d] + \sigma_2^2\Omega_2[t,t]$ (atau $+\sigma_2^2$ untuk `"S"`). g_3 berasal dari turunan eksak prediktor terhadap komponen varians, $\partial_a \tilde{\theta}_i = \ell_a' P y$. Kolom `g4` bernilai 0 untuk sel ini.

Implementation note. Koreksi orde kedua untuk bias $g_1(\hat{\theta})$, yaitu $\frac{1}{2}\operatorname{tr}(F^{-1}\nabla^2 g_1)$ (dipakai untuk domain tak-tersampel pada `eblup_sfh()`), sempat diimplementasikan dan cocok dengan turunan beda hingga dari implementasi padat. Namun koreksi itu tidak stabil secara numerik bila ρ_1 mendekati ± 1 (Ω_1 hampir singular) atau bila suatu komponen varians lemah teridentifikasi: dalam simulasi (Tabel 26) beberapa replikasi menghasilkan suku yang meledak sehingga bias relatif rata-ratanya ratusan hingga ribuan persen. Tanpa koreksi itu, $g_1 + g_2 + g_3$ memberi bias relatif beberapa persen dan cakupan mendekati 95% untuk kedua jenis sel tanpa sampel. Karena itu koreksi tidak dipakai.

Bootstrap parametrik (`mse_type = "bootstrap"`, `src/eblup_stfh_pbmse.cpp`):

1. fit awal penuh pada data asli;
2. bangkitkan $u^{(1)*} \sim N(0, \sigma_1^2\Omega_1)$ pada semua D domain lewat faktor Cholesky *bawah* $\Omega_1 = LL'$, $u^{(2)*}$ AR(1) per domain pada semua T periode, dan $\varepsilon^* \sim N(0, \psi)$ hanya pada sel tersampel — *sekuensial, sebelum* region paralel (baris 158-188);
3. dalam `#pragma omp parallel for schedule(dynamic)`: $y_S^* = \mu_S^* + \varepsilon^*$ dengan $\mu^* = \mathbf{X}\hat{\beta} + Zu^{(1)*} + u^{(2)*}$, lalu komponen varians *diestimasi ulang* dengan REML pada sel tersampel (`stfh_fit_arma`) seperti `sae::pbmseSTFH`, dan setiap sel grid diprediksi dengan (41);
4. replikat yang fit-nya gagal diganti draw baru sampai tepat B replikat valid (maksimum $5B$ percobaan); $\widehat{\mathrm{MSE}}_i = B^{-1}\sum_b(\hat{\theta}_{b,i}^* - \mu_{b,i}^*)^2$ untuk *setiap* sel, tersampel atau tidak.

Bila `mse_type` tidak diberikan tetapi `B` diberikan, bootstrap yang dipilih (kompatibilitas dengan versi sebelumnya, baris 190-198).

Validasi. Prediksi semua sel, $\hat{\beta}$, g_1, dan g_2 dibandingkan dengan implementasi padat (pembentukan $\mathbf{V}_S$ penuh) dan cocok sampai presisi mesin. g_3 cocok sampai 10^{-10} relatif, dan g_4 sel tersampel cocok dengan turunan beda hingga (`tests/testthat/test-eblup_stfh_unsampled.R`). Pada panel lengkap, hasilnya identik bit demi bit dengan versi sebelum dukungan sel tak tersampel. Tabel 26 merangkum simulasi Monte Carlo berbasis model.

Tabel 26: Simulasi Monte Carlo berbasis model untuk `eblup_stfh` dengan sel tanpa estimasi langsung: $D = 50$, $T = 5$, 200 replikasi (bootstrap $B = 100$ pada 50 replikasi pertama), $(\sigma_1^2, \rho_1, \sigma_2^2, \rho_2) = (0.5, 0.5, 0.2, 0.5)$. MSE empiris EBLUP dan estimator sintetis $x'\hat{\beta}$; bias relatif (%) estimator MSE analitik dan bootstrap; cakupan (%) interval $\hat{\theta} \pm 1{,}96\sqrt{\widehat{\mathrm{MSE}}}$ analitik, dirata-ratakan per jenis sel. Sumber: `benchmarks/output/sim_stfh_unsampled.rds` (`benchmarks/sim_stfh_unsampled.R`); angka dihitung ulang oleh `scripts/01_tables.R`.

Jenis sel	Sel	MSE empiris		RB (%)		Cakupan
		EBLUP	sintetis	analitik	bootstrap	(%)
tersampel	207	0.0682052	0.861458	0.0447287	3.19653	94.8237
periode hilang	23	0.223885	0.850855	-0.956272	2.03308	93.8696
domain tanpa sampel	20	0.806336	0.930835	3.20254	2.29949	94.925

7.6 Pseudo-code

7.7 Signature, argumen, objek keluaran, dan contoh

Algorithm 4:

Masukan *formula,* `vardir`*,* `data`*,* `domain`*,* `time`*,* W *(*$D \times D$*),* `model` $\in$*{ST, S}, nilai awal,* `mse_type`*,* `B`*,* `n_threads`*,* `seed` **Keluaran** `df_eblup` *(setiap domain* $\times$ *waktu, termasuk kolom* `sampled`*),* `estvarcomp`*,* `goodness`*,* g_1–g_4 ; *// Lapis R*

urutkan domain-major (domain menurut kemunculan, waktu menaik) (204-250);

gagal bila variabel bantu `NA`, semua respons `NA`, `vardir` sel tersampel tidak positif, baris grid tidak lengkap, atau W bukan $D \times D$ (260-316);

validasi `sigma*_start` ≥ 0 dan `rho*_start` $\in (-1, 1)$ (317-330);

$\hat{\theta}, \ldots \leftarrow$ `.eblup_stfh_core` (343-356);

if `mse_type` $\neq$ *none* **then**

  analitik: salin `mse_analytical`; bootstrap: `.pbmse_stfh` (402-436)

else

  `mse = rse = NA` (437-444)

;

kembalikan ke urutan baris pemanggil (472);

; *// Lapis C++*

$S \leftarrow$ sel dengan y tidak `NA`; indeks per domain (`st_make_index`, 853-872);

$\sigma^2_{1,2} \leftarrow 0.5\,\text{median}(\psi_S)$ (bila tidak diberikan); $\rho_{1,2} \leftarrow$ argumen (874-878);

while `diff` $>$ `precision` ***dan*** $k <$ `maxiter` **do**

  bangun A^{-1} per blok $|S_d| \times |S_d|$, $\mathbf{C}$ $(D \times D)$, U (39) (`st_build`);

  $S, F \leftarrow$ (42) terstruktur (`st_score_info`);

  $\theta_{k+1} \leftarrow \theta_k + F^{-1}S$; bila $|\rho_{1,2}| \geq 1$ maka $\rho_{1,2} \leftarrow \pm 0.999$;

  `diff` $\leftarrow \max \left|(\theta_{k+1} - \theta_k)/\theta_k^{\text{safe}}\right|$ (921-953);

if *gagal* ***atau*** *varians negatif* **then**

  fallback REML terkendala `stfh_fit_safeguarded` (955-984)

$\sigma^2 \leftarrow \max(0, \cdot)$; $\hat{\beta} \leftarrow Q\mathbf{X}'_S V^{-1} y_S$ (986-1053);

$\hat{u}^{(1)}, \hat{u}^{(2)} \leftarrow$ (40) di sel tersampel (1055-1078);

loglik, AIC, BIC lewat lemma determinan (1080-1104);

galat baku β dan komponen varians (1106-1158);

$g_1, \ldots, g_4$ (43)–(44) terstruktur di sel tersampel (1161-1370);

BLUP (41) dan MSE (45) untuk sel lain (1373-1500);

Listing 7: Signature `eblup_stfh` (`R/eblup_stfh.R:164-188`).

```
eblup_stfh(formula, vardir, data, domain, time, W,
           model = c("ST", "S"), maxiter = 100, precision = 1e-4,
           sigma21_start = NULL, rho1_start = 0.5,
           sigma22_start = NULL, rho2_start = 0.5,
           mse_type = c("analytical", "bootstrap", "none"),
           B = 100, n_threads = 1, seed = -1,
           self_benchmark = FALSE, benchmark_weight = NULL,
           benchmark_target = NULL, benchmark_group = NULL,
           print_result = TRUE, compute_mse)  # compute_mse: deprecated
```

- `domain`, `time` — nama kolom/kolom formula penanda domain dan waktu (wajib; satu baris per sel grid, urutan baris bebas).
- `W` — matriks tetangga $D \times D$ (hanya antar domain, tanpa dimensi waktu) untuk *semua* domain, dengan urutan domain menurut kemunculan pertama.
- `model` — `"ST"` (default, AR(1) temporal) atau `"S"` (tanpa struktur temporal).
- `mse_type` — `"analytical"` (default), `"bootstrap"`, atau `"none"` (`mse = NA`). `compute_mse` sudah *deprecated*.
- `self_benchmark` — kalibrasi agregat berbobot sel tersampel per kelompok (`.setup_self_benchmark` menormalkan bobot pada sel tersampel).

Objek keluaran: `"fastsae"` berisi `estcoef` (`beta, std.error, tvalue, pvalue, stderr_beta, zvalue`), `estvarcomp` (4 atau 3 baris `estimate, std.error`), `goodness` (`loglikelihood, AIC, BIC` dari log-likelihood penuh seperti sae::eblupSTFH; sebelum 1.0.0 elemen pertama bernama `loglike`, STFH-8), `df_eblup` dengan kolom `domain, time, y, eblup, vardir, mse, rse, mse_pb, random_effect_u1, random_effect_u2, sampled` (`y` dan `vardir` `NA` di sel tanpa estimasi langsung), `mse_analytical`, `rse_analytical`, `g1`–`g4`, `mse_type`, `B`, `convergence`, `n_iter`, `method = "REML"`.

Listing 8: Contoh yang dapat dijalankan (dari `man/eblup_stfh.Rd`); domain 21 dan 25 tidak pernah tersampel.

```
library(fastsae)

m1 ← eblup_stfh(y ~ x1 + x2 + x3, data = mys_panel,
                vardir = ~vardir, domain = ~area, time = ~year,
                W = mys_proxmat, model = "ST")

# MSE analitik untuk setiap sel, tersampel atau tidak
m2 ← eblup_stfh(y ~ x1 + x2 + x3, data = mys_panel,
                vardir = ~vardir, domain = ~area, time = ~year,
                W = mys_proxmat, model = "ST",
                mse_type = "analytical")
head(m2$df_eblup[!m2$df_eblup$sampled, ])
```

7.8 Referensi kunci

Marhuenda et al. [2013] adalah rujukan model dan sekaligus rujukan yang disebut roxygen (`R/eblup_stfh.R`) bersama Rao and Molina [2015]; gabungan data deret waktu dengan potongan lintas bagian yang menjadi dasar model semacam ini dirintis Rao and Yu [1994]. Prediksi sel di luar sampel mengikuti Saei and Chambers [2005] dan, untuk efek SAR pada area di luar sampel, Benedetti et al. [2026]. Implementasi pembandingnya sae [Molina and Marhuenda, 2015]. Seluruh rujukan terdaftar di `references.bib`.

8 Model unit-level Battese–Harter–Fuller: `eblup_bhf`

8.1 Setting data dan kapan dipakai

Model ini membutuhkan dua sumber data yang berbeda:

- `unit_data` — satu baris per unit sampel: respons y_j, kovariat x_j, dan label domain;
- `Xpop` — *rata-rata kovariat populasi* per domain ($\bar{x}_{\text{pop},d}$) beserta ukuran populasi `popsize_var` (N_d).

Cocok ketika tersedia mikrodata survei (unit) dan ringkasan populasi per domain. Dataset contoh `cornsoybean` (37 unit di 12 kabupaten Iowa) bersama `cornsoybeanmeans` adalah data klasik BHF [Battese et al., 1988]. Catatan penting: `Xpop` harus berisi *semua* domain pada sampel — bila ada domain sampel yang tidak ada di `Xpop`, C++ melempar error, berbeda dengan sae::eblupBHF yang mengembalikan `NA` disertai peringatan (BHF-6).

8.2 Persamaan model

$$y_{ij} = x'_{ij}\beta + u_i + e_{ij}, \qquad u_i \overset{\text{iid}}{\sim} N(0, \sigma_u^2), \qquad e_{ij} \sim N(0, \sigma_e^2), \tag{46}$$

dengan $i = 1, \ldots, D$ domain, $j = 1, \ldots, n_i$ unit sampel, dan N_i unit populasi. Model ini model komponen-kesalahan (*error components*) — tanpa struktur spasial — yang diperkenalkan Battese et al. [1988]. Penting: karena hanya *ringkasan* populasi yang masuk ($\bar{x}_{\text{pop},d}$ dan N_d), model tidak membutuhkan mikrodata populasi.

8.3 Prediktor titik

Komponen varians diestimasi oleh rutin C++ sendiri, `.bhf_fit` → `.bhf_fit_cpp` (`R/eblup_bhf.R:96, 227-236`, `src/eblup_bhf_core.cpp`); lme4 tidak lagi dipanggil dan hanya ada di `Suggests` sebagai pembanding pada tes. Hasilnya

$$\hat{\beta}, \quad \hat{\sigma}_u^2, \quad \hat{\sigma}_e^2, \quad \hat{u}_d = \hat{\gamma}\, w_d\, (\bar{y}_{s,d} - \bar{x}'_{s,d}\hat{\beta}), \qquad w_d = \frac{n_d}{1 + n_d\hat{\gamma}},\ \hat{\gamma} = \hat{\sigma}_u^2/\hat{\sigma}_e^2, \tag{47}$$

sama dengan `lme4::lmer(y ~ ... + (1|dom), REML = ...)` sampai toleransi numerik. Penyusunan prediktor ada di C++ (`src/eblup_unit.cpp`) dengan fraksi sampel $f_d = n_d/N_d$:

$$\hat{\theta}_d = f_d\, \bar{y}_{s,d} + (\bar{x}_{\text{pop},d} - f_d\, \bar{x}_{s,d})'\hat{\beta} + (1 - f_d)\hat{u}_d, \tag{48}$$

untuk domain dengan $n_d > 0$, dan prediktor sintetis murni

$$\hat{\theta}_d = \bar{x}'_{\text{pop},d}\hat{\beta}, \qquad \texttt{samp_size} = 0,\ \texttt{peringatan} \tag{49}$$

untuk domain tanpa unit sampel (`cli_warn` di R, baris 138-142).

Implementation note. (48) identik dengan rumus yang dipakai `sae::eblupBHF` dan ekuivalen dengan $f_d\bar{y}_s + (1 - f_d)(\bar{x}'_{\text{unsamp}}\hat{\beta} + \hat{u}_d)$ — *bukan* bentuk alternatif $f_d\bar{y} + (1 - f_d)(\bar{x}'_{\text{pop}}\hat{\beta} + \hat{u}_d)$ yang kadang dipakai ketika seluruh populasi dianggap tersensus (BHF-9). Pilihan ini masuk akal untuk BHF asli di mana hanya n_d unit yang disurvei.

8.4 Estimasi komponen varians

Dengan $\gamma = \sigma_u^2/\sigma_e^2$, kovarians marginal domain d adalah $\sigma_e^2(I_{n_d} + \gamma J_{n_d})$. β dan σ_e^2 di-*profile* secara analitik, sehingga estimasi menjadi maksimisasi satu dimensi atas $\gamma \geq 0$. Setiap evaluasi hanya membutuhkan *cross-product* dalam-domain (tetap) dan rata-rata domain:

$$X'V_0^{-1}X = W_{xx} + \sum_d w_d\, \bar{x}_{s,d}\bar{x}'_{s,d}, \qquad Q(\gamma) = W_{rr} + \sum_d w_d\, \bar{r}_d^2, \tag{50}$$

dengan $\bar{r}_d = \bar{y}_{s,d} - \bar{x}'_{s,d}\hat{\beta}$ dan biaya $O(Dp^2 + p^3)$ per evaluasi, tidak bergantung pada jumlah unit. Log-likelihood terprofil adalah $-\frac{1}{2}\{n \log Q + \sum_d \log(1 + n_d\gamma)\}$ (ML) atau $-\frac{1}{2}\{(n-p)\log Q + \sum_d \log(1+n_d\gamma) + \log\left|X'V_0^{-1}X\right|\}$ (REML), lalu $\hat{\sigma}_e^2 = Q/n$ atau $Q/(n-p)$ dan $\hat{\sigma}_u^2 = \hat{\gamma}\hat{\sigma}_e^2$.

Maksimum global dicari dengan memindai grid $\log \gamma \in [10^{-8}, 10^{8}]$ untuk pergantian tanda skor, menghaluskan tiap akar dengan metode Brent, dan memilih log-likelihood terbesar; batas $\gamma = 0$ ikut menjadi kandidat bila skornya di sana tidak positif. Konsekuensi terhadap objek keluaran:

- `estcoef` memuat kolom `beta, std.error, stderr_beta, tvalue, zvalue, pvalue`, dengan galat baku dari $\hat{\sigma}_e^2(X'V_0^{-1}X)^{-1}$ dan `pvalue` $= 2\,\Phi(-|t|)$ (BHF-8).
- `convergence` berasal dari fit (`FALSE` bila skor masih positif di ujung atas grid); desain yang *rank-deficient* atau sampel yang terlalu kecil ditolak lebih dulu (`R/eblup_bhf.R:229-234`). Tidak ada `goodness` (loglik/AIC/BIC) dan tidak ada `n_iter` (BHF-10).

8.5 Estimasi MSE

MSE *hanya* bootstrap parametrik [González-Manteiga et al., 2008] dan dihitung secara default (`mse_type = "bootstrap"`); `mse_type = "none"` memberi `mse = rse = NA`. Argumen `compute_mse` dari 0.1.0 masih diterima dengan peringatan *deprecation*. Persiapan ada di R (`.pbmse_unit`, `R/eblup_bhf.R:248-325`); replikat dijalankan di C++ (`.pbmse_bhf_cpp`) dan diparalelkan dengan OpenMP sebanyak `n_threads`. Tiap replikat:

1. bangkitkan $u_i^* \sim N(0, \hat{\sigma}_u^2)$ untuk semua domain dan $e_{ij}^* \sim N(0, \hat{\sigma}_e^2)$ untuk semua unit sampel, lalu $y_{ij}^* = x'_{ij}\hat{\beta} + u_i^* + e_{ij}^*$;
2. refit REML/ML pada y^* dengan rutin C++ yang sama, lalu EBLUP ulang (48);
3. nilai "kebenaran" populasi replikat: $\mu_d = \bar{x}'_{\text{pop},d}\hat{\beta}$ ditambah u_d^* pada *ketiga* cabang; bila $r_d = N_d - n_d > 0$, $\theta_d^{*,\text{true}} = \mu_d + u_d^* + \frac{n_d}{N_d}\bar{e}_{s,d}^* + \frac{r_d}{N_d}\mathcal{N}(0, \hat{\sigma}_e^2/r_d)$; bila $r_d = 0$ (enumerasi penuh) hanya $\bar{e}_{s,d}^*$ ditambahkan; bila $n_d = 0$, $\mu_d + u_d^* + \mathcal{N}(0, \hat{\sigma}_e^2/N_d)$;
4. akumulasi $(\hat{\theta}_d^* - \theta_d^{*,\text{true}})^2$ hanya untuk replikat yang refit-nya berhasil, lalu bagi dengan jumlah replikat yang berhasil (BHF-4).

Implementation note. Semua bilangan acak diambil dari RNG R secara berurutan di thread utama (per blok replikat: u^*, e^*, lalu galat rata-rata non-sampel), sehingga hasil hanya bergantung pada `seed`, bukan pada `n_threads`. Default berbeda dari `sae`: $B = 100$ di sini, $B = 200$ pada `sae::pbmseBHF` (BHF-2); `seed = -1` (default) artinya tanpa seed (BHF-3). Bootstrap memakai matriks rata-rata populasi yang sama dengan estimasi titik, termasuk `popnmean_xpop` bila diberikan (BHF-5).

RSE dihitung $100\sqrt{\text{MSE}}/\left|\hat{\theta}\right|$ dengan guard $\left|\hat{\theta}\right| <$ `.Machine$double.eps` $\rightarrow$ `NA` (baris 172-175).

8.6 Pseudo-code

Algorithm 5:

Masukan *formula, `unit_data`, `Xpop`, `domain_var`, `popsize_var`, `method`, `mse_type`, `B`, `n_threads`, `seed`* **Keluaran** *`df_eblup` (domain, eblup, samp_size, mse, rse), `estcoef`, `fit`* `mf` ← `model.frame(formula, na.action = na.omit)`; `dom` dipotong mengikuti baris ter-omit (79-83);
$y_s, \mathbf{X}_s$ ← respons dan `model.matrix` (87-88);
`selectdom` ← `unique(c(dom, domain Xpop))`; indeks domain 0-based (91-93);
`fit` ← `.bhf_fit_cpp`: profil REML/ML atas γ, grid log + Brent (96, 227-236);
$\hat{\beta}, \hat{u}_d, \hat{\sigma}^2_u, \hat{\sigma}^2_e$ ← (47) (97-98, 117-118);
`meanxpop` ← `model.matrix` atas `Xpop`, atau argumen `popnmean_xpop` (103-115);
$\hat{\theta}$ ← `.eblup_bhf_cpp(selectdom, dom, Xs, meanxpop, ys, popnsize, betah, upred)` (121-130);
if *ukuran `meanxpop/popnsize` tidak cocok dengan `selectdom`* **then**
    gagal (C++, `src/eblup_unit.cpp:23-31`)
;
if *`mse_type` = bootstrap* **then**
    MSE ← `.pbmse_unit(...)` (145-169)
else
    `mse = rse = NA`
;
perakitan `df_eblup`, `estcoef`, kelas `c("fastsae","fastsae_unit")` (171-216);
Data ; *// Bootstrap parametrik* -- `.pbmse_unit` (248-325) dan `.pbmse_bhf_cpp`
if *`seed`* ≥ 0 **then**
    `set.seed(seed)` (310)
;
for *setiap blok replikat* **do**
    ambil u^* (semua domain), e^* (semua unit) dan galat rata-rata non-sampel dari RNG R, berurutan;
    for *replikat b dalam blok* **do** `// paralel OpenMP, n_threads`
        $y^* \leftarrow \mathbf{X}\hat{\beta} + u^*_{d(j)} + e^*_j$; refit REML/ML;
        if *refit gagal* **then**
            tandai tidak valid; `next`
        $\hat{\theta}^*$ ← (48) dengan y^*; $\theta^{*,\text{true}}_d$ ← langkah 3 bagian 8.5;
    akumulasi $(\hat{\theta}^*_d - \theta^{*,\text{true}}_d)^2$ untuk replikat valid;
MSE ← akumulasi$/B_{\text{valid}}$; RSE ← $100\sqrt{\text{MSE}}/\left|\hat{\theta}\right|$ (172-175);

8.7 Signature, argumen, objek keluaran, dan contoh

Listing 9: Signature `eblup_bhf` (`R/eblup_bhf.R:62-75`).

```
eblup_bhf(formula, unit_data, Xpop, domain_var, popsize_var,
          method = c("REML", "ML"), popnmean_xpop = NULL,
          B = 100, mse_type = c("bootstrap", "none"),
          n_threads = 1, seed = -1, print_result = TRUE,
          compute_mse)  # compute_mse: deprecated
```

- `Xpop` — `data frame` berisi kolom domain (sebagai `domain_var`, bukan nama kolom) dan kolom kovariat yang sama dengan formula; nilainya rata-rata populasi.
- `popsize_var` — nama kolom ukuran populasi N_d pada `Xpop`.
- `popnmean_xpop` — matriks alternatif rata-rata populasi (opsional).
- `n_threads` — jumlah thread OpenMP untuk bootstrap MSE (BHF-1).
- `mse_type` — `"bootstrap"` (default) atau `"none"` (kolom `mse`, `rse` diisi `NA`). `compute_mse` sudah *deprecated*.

Objek keluaran: `list` berkelas `c("fastsae", "fastsae_unit")` berisi `df_eblup` (`domain, eblup, samp_size, mse, rse`), `estcoef`, `random_effect_var` ($\hat{\sigma}_u^2$), `fit` berisi `method, random_effect_var, sigma2_e, beta, random_effect, residuals`, `formula`, `model = "BHF"`, `level = "unit"`, `convergence`, `data`, `call`. *Tidak ada* `goodness` dan `n_iter`.

Listing 10: Contoh yang dapat dijalankan (dari `man/eblup_bhf.Rd`).

```
library(dplyr)
df_meanpop ← cornsoybeanmeans |>
  rename(CornPix = MeanCornPixPerSeg, SoyBeansPix = MeanSoyBeansPixPerSeg)
df_cornsoybean ← cornsoybean |>
  rename(CountyIndex = County)

res ← eblup_bhf(
  formula = CornHec ~ CornPix + SoyBeansPix,
  Xpop = df_meanpop,
  unit_data = df_cornsoybean,
  domain_var = "CountyIndex",
  popsize_var = "PopnSegments"
)
head(res$df_eblup)
```

8.8 Referensi kunci

Blok roxygen fungsi ini memuat Battese et al. [1988] ; kerangka EBLUP unit-level dan MSE dibahas di Rao and Molina [2015]; pembanding numeriknya `sae` [Molina and Marhuenda, 2015] dan `lme4::lmer` [Bates et al., 2015], yang dipakai di tes untuk memverifikasi estimasi REML/ML; bootstrap parametrik mengikuti González-Manteiga et al. [2008].

9 Model sub-area dua-lipat: `eblup_twofold`

9.1 Setting data dan kapan dipakai

Data memiliki tiga tingkat: area (mis.
kabupaten), *sub-area* di dalam area (mis.
kecamatan), dan unit sampel di dalam sub-area. Satu baris data = satu unit sampel, dengan kolom penanda `domain` (area) dan `subarea`. Model ini dipakai ketika tersedia kovariat pada tingkat unit tetapi target estimasi pada dua tingkat sekaligus (area dan sub-area), termasuk sub-area yang tidak tersampel. Rujukan aslinya Torabi and Rao [2014]; roxygen fungsi ini menyebut model dua-lipat tanpa mengutip DOI (`R/eblup_twofold.R:7-14`).

9.2 Persamaan model

Dua tingkat persamaan (komentar `eblup_twofold.cpp:4-8`):

$$y_{ij} = \theta_{ij} + e_{ij}, \qquad e_{ij} \sim N(0, \psi_{ij}), \tag{51}$$

$$\theta_{ij} = x'_{ij}\beta + v_i + u_{ij}, \qquad v_i \sim N(0, \sigma_v^2), \quad u_{ij} \sim N(0, \sigma_u^2), \tag{52}$$

dengan v_i = efek area (notasi kode: `s2v`) dan u_{ij} = efek sub-area (`s2u`). Untuk tiap area d kovariasinya adalah

$$V_d = \sigma_v^2 \mathbf{1}\mathbf{1}' + \operatorname{diag}(\sigma_u^2 + \psi_d), \qquad S_d = \sigma_v^2 \mathbf{1}\mathbf{1}' + \sigma_u^2 \mathbf{I}, \tag{53}$$

dan inversinya memakai identitas Sherman–Morrison per blok (2) (biaya $O(n_d)$, bukan $O(n_d^3)$).

9.3 Prediktor titik

$$\hat{v}_i = \sigma_v^2 \, \mathbf{1}'V^{-1}r, \qquad \hat{u}_{ij} = \sigma_u^2 \, (V^{-1}r)_{ij}, \qquad \hat{\theta}_{ij} = x'_{ij}\hat{\beta} + \hat{v}_i + \hat{u}_{ij}, \tag{54}$$

dengan $r = y - \mathbf{X}\hat{\beta}$ dan V^{-1} per blok lewat (2) (baris 356-395). Tiga kasus domain:

- sub-area tersampel pada area tersampel: (54);
- sub-area *tak*-sampel pada area tersampel: $\hat{\theta}_{ij} = x'_{ij}\hat{\beta} + \hat{v}_i$ dan $\hat{u} = 0$ (data hilang hanya menyisakan informasi area);
- area tanpa sub-area tersampel sama sekali: $\hat{\theta} = x'\hat{\beta}$ (prediktor sintetis).

Domain tanpa sampel didefinisikan lewat respons `y = NA` (baris 319-321), sama seperti FH. Validasi mensyaratkan minimal dua area tersampel, $N_s > p$, dan $\psi > 0$ pada sub-area tersampel (baris 330-341).

9.4 Estimasi komponen varians

Fisher scoring dua parameter (σ_v^2, σ_u^2), nilai awal $0.5\,\text{median}(\psi)$ keduanya (baris 197-198), toleransi 10^{-4}, maksimum 100 iterasi (baris 202). Per iterasi (fungsi `twofold_pass`, baris 84-185) dihitung $Q = (\mathbf{X}'V^{-1}\mathbf{X})^{-1}$, $r = V^{-1}(y - \mathbf{X}\hat{\beta})$, lalu skor dan informasi:

$$s_v = -\tfrac{1}{2}q_d + \tfrac{1}{2}(\mathbf{1}'r)^2, \quad s_u = -\tfrac{1}{2}\operatorname{tr} P + \tfrac{1}{2}r'r, \quad q_d = \mathbf{1}'V^{-1}\mathbf{1} - (\mathbf{X}'V^{-1}\mathbf{1})'Q(\mathbf{X}'V^{-1}\mathbf{1}), \tag{55}$$

(dijumlahkan atas area). Informasi Fisher REML dihitung *eksak*: P tidak blok-diagonal, sehingga dibutuhkan suku lintas-area. Dengan $s_{1d} = \mathbf{1}'V_d^{-1}\mathbf{1}$, $a_d = \mathbf{X}_d'V_d^{-1}\mathbf{1}$, $S_a = \sum_d a_d a_d'$, $M = \mathbf{X}'V^{-2}\mathbf{X}$, $A_d = V_d^{-1}\mathbf{X}_d$, dan $z_d = V_d^{-1}\mathbf{1}$,

$$\begin{aligned}
\mathcal{I}_{vv} &= \tfrac{1}{2}\Big[\sum_d s_{1d}^2 - 2\sum_d s_{1d}\, a_d'Qa_d + \operatorname{tr}(QS_aQS_a)\Big], \\
\mathcal{I}_{uu} &= \tfrac{1}{2}\Big[\operatorname{tr}(V^{-2}) - 2\operatorname{tr}(Q\mathbf{X}'V^{-3}\mathbf{X}) + \operatorname{tr}(QMQM)\Big], \\
\mathcal{I}_{vu} &= \tfrac{1}{2}\Big[\sum_d (z_d'z_d - 2z_d'A_dQa_d) + \operatorname{tr}(QMQS_a)\Big],
\end{aligned} \tag{56}$$

untuk REML (baris 135-180); untuk ML suku yang memuat Q hilang dan $\operatorname{tr} P$, q_d diganti oleh $\operatorname{tr} V^{-1}$ dan $\mathbf{1}'V^{-1}\mathbf{1}$. Langkah $\Delta = \mathcal{I}^{-1}s$; bila informasi berkondisi buruk (secara relatif: dengan satu sub-area per area hanya $\sigma_v^2 + \sigma_u^2$ yang teridentifikasi) dipakai pseudo-invers, lalu fallback

diagonal $s_k/\mathcal{I}_{kk}$ (baris 205-228); nilai negatif dipaksa 0 (baris 230-235), dan konvergensi relatif (baris 237-239).

Implementation note. Dua hal yang tidak ada pada FH: (i) varian estimasi komponen diambil dari *invers matriks informasi Fisher* final, bukan bentuk tertutup $\mathrm{Var}(\hat{\sigma}^2) = 2/\sum w^2$ (TFH-3); (ii) sebelum rilis 1.0.0, goodness fit REML dihitung dari log-likelihood terbatas. Kini, seperti FH dan SFH, `loglikelihood`, AIC, dan BIC memakai log-likelihood penuh $-\frac{1}{2}[n \log 2\pi + \text{core}]$ pada estimasi untuk ML maupun REML, dan BIC memakai n = jumlah sub-area tersampel (TFH-9).

9.5 Estimasi MSE

Analitik (default). Untuk sub-area tersampel (baris 396-480):

$$g_1 = \mathrm{diag}(S - SV^{-1}S), \tag{57}$$

$$g_2 = \mathrm{diag}(DQD'), \qquad D = (\mathbf{I} - SV^{-1})\mathbf{X}_d, \tag{58}$$

$$g_3 = \mathbb{E}_{vv}\,\mathrm{var}_v + \mathbb{E}_{uu}\,\mathrm{var}_u + 2\,\mathbb{E}_{vu}\,\mathrm{cov}_{vu}, \tag{59}$$

di mana $\mathrm{var}_v, \mathrm{var}_u, \mathrm{cov}_{vu}$ adalah elemen $(\mathcal{I}^{-1})$ final sesuai `method` (ML atau REML; pseudo-invers bila hanya $\sigma_v^2 + \sigma_u^2$ teridentifikasi) dan $\mathbb{E}_{uu} = \mathrm{diag}(M_u V M_u')$ dengan $M_u = V^{-1} - SV^{-2}$, sedangkan $M_v = \mathbf{1}(\mathbf{1}'V^{-1})' - SV^{-1}\mathbf{1}(\mathbf{1}'V^{-1})'$. Estimator MSE-nya

$$\widehat{\mathrm{MSE}} = g_1 + g_2 + 2g_3 \quad \text{(REML)}, \qquad \widehat{\mathrm{MSE}} = g_1 + g_2 + 2g_3 - \mathbf{b}_{\mathrm{ML}}'\nabla g_1 \quad \text{(ML)}, \tag{60}$$

dengan $\mathbf{b}_{\mathrm{ML}} = \frac{1}{2}\mathcal{I}^{-1}h$, $h_j = -\,\mathrm{tr}(Q\mathbf{X}'V^{-1}V_jV^{-1}\mathbf{X})$. Faktor dua pada g_3 diperlukan karena $g_1(\hat{\theta})$ plug-in sendiri bias ke bawah sebesar $\approx g_3$: $g_1 + g_2 + g_3$ mendekati MSE sebenarnya, tetapi *estimator*-nya $g_1 + g_2 + 2g_3$ [Prasad and Rao, 1990, Datta and Lahiri, 2000]. Versi sebelum audit 12 Okt memakai $g_1 + g_2 + g_3$.

Implementation note. Di sinilah perbedaan paling menonjol dengan publikasi. Komentar kode (`eblup_twofold.cpp:19-24`) menyatakan bahwa persamaan (3.4) Torabi and Rao [2014] mengandung suku $[A][B]$ yang semu dan koefisien varian u yang salah bentuk (plus tanda salah pada lampiran), sehingga pendekatan matriks g_3 dipakai dan divalidasi dengan turunan beda pemisah (korelasi 0,97/0,96/0,74 terhadap uji numerik). Laporan ini melaporkannya sebagai TFH-1: apa yang dikodekan adalah pendekatan matriks, bukan rumus tertutup Torabi–Rao.

Untuk sub-area tak-sampel (baris 482-546): area tanpa sub-area tersampel memperoleh $x'Qx + \sigma_v^2 + \sigma_u^2$ (baris 489-493); sub-area tak-sampel pada area tersampel memperoleh $g_1^* + g_2^* + 2g_3^*$ (dikurangi suku bias ML bila `method = "ML"`) dengan $g_1^* = \sigma_v^2 - \sigma_v^4(\mathbf{1}'V^{-1}\mathbf{1}) + \sigma_u^2$ dan $g_2^* = (x - \sigma_v^2\mathbf{X}'V^{-1}\mathbf{1})'Q(x - \sigma_v^2\mathbf{X}'V^{-1}\mathbf{1})$ (baris 500-545). Nilai negatif dalam rentang $(-10^{-8}, 0)$ dipangkas ke 0; lebih negatif dibiarkan sehingga RSE menjadi `NaN` (TFH-8).

Bootstrap parametrik. Dengan `mse_type = "bootstrap"` (default $B = 100$, sama dengan fungsi EBLUP lain): semua gambaran dibuat lebih dulu secara sekuensial ($v^* \in \mathbb{R}^{m\times B}$, $u^* \in \mathbb{R}^{N\times B}$, $e^* \in \mathbb{R}^{N_s\times B}$; baris 552-555), lalu region `#pragma omp parallel for schedule(static)` (baris 558) berjalan per replikat: $\theta^* = \mathbf{X}\hat{\beta} + v_i^* + u_j^*$, $y^* = \theta^*_{\text{sampel}} + e^*$, refit Fisher scoring penuh, prediktor bootstrap dengan prediktor yang *sama* seperti estimasi titik (termasuk cabang tak-sampel), dan $\widehat{\mathrm{MSE}} = \mathrm{rata}((\hat{\theta}^* - \theta^*)^2)$ (baris 559-594). Karena semua RNG dipanggil sebelum pragma, hasil tidak bergantung jumlah thread (TFH-6: pemakaian prediktor yang sama persis juga dinyatakan kode sebagai perbaikan bug).

Algorithm 6:

Masukan *formula, vardir, domain (wajib), subarea, method, mse_type, B, seed, maxiter, precision* **Keluaran** *df_eblup (area × sub-area), random_effect_var* = (σ_v^2, σ_u^2), *goodness* `domain` ← wajib (96-99); `subarea` ← `seq_len(nrow(data))` bila `NULL` (100-104);

if *self_benchmark* **then**

  augmentasi model (116-131)

`area_idx` ← `as.integer(factor(domain)) - 1L` ; `// 0-based (140)`

if *seed* ≥ 0 **then**

  `set.seed(seed)` lewat `.with_seed()`; aliran RNG sesi dipulihkan sesudahnya

;

$\ldots \leftarrow$ `.eblup_twofold_core(`$\mathbf{X}, y, \psi$`, area, method, mse_type, B, maxiter, precision)` (144-154);

Data ; `// Lapis C++`

gagal bila `method`/`mse_type` tak sah, $N_s \leq p$, $\psi \leq 0$, atau area tersampel < 2 (311-341);

$\sigma_v^2 \leftarrow \sigma_u^2 \leftarrow 0.5\,\text{median}(\psi)$ (197-198);

while *diff* $>$ *precision dan* $k <$ *maxiter* **do**

  per blok d: $V_d^{-1}a \leftarrow$ (2) (`twofold_pass`, 84-185);

  if *ML* **then**

    $s, \mathcal{I} \leftarrow$ bentuk ML

  else

    $s, \mathcal{I} \leftarrow$ (55)–(56), termasuk suku lintas-area

  ;

  $\Delta \leftarrow \mathcal{I}^{-1}s$; kondisi buruk → `pinv`, lalu $\Delta_k = s_k/\mathcal{I}_{kk}$ (205-228);

  $\sigma_k^2 \leftarrow \max(\sigma_k^2 + \Delta_k, 0)$; non-finite → nilai lama (230-235);

  `diff` $\leftarrow \max_k \left|(\sigma_k^{2(new)} - \sigma_k^2)/\max(\sigma_k^2, 10^{-12})\right|$ (237-239);

pass final pada varian final: $\hat{\beta}, Q, \mathcal{I}^{-1}$, loglik ML/REML (245-298);

$\hat{\theta} \leftarrow$ (54), termasuk cabang tak-sampel (356-395);

if *mse_type* = *analytical* **then**

  $g_1, g_2, g_3 \leftarrow$ (57)–(59); MSE ← (60) per baris tersampel (396-480); cabang tak-sampel (482-546)

else

  $v^*, u^*, e^* \leftarrow$ bangkit sekuensial (552-555);

  `#pragma omp parallel for schedule(static)` (558);

  for $b = 0, \ldots, B-1$ *paralel* **do**

    $\theta^*, y^* \leftarrow$; refit `twofold_fit`; SqDiff$[:, b] \leftarrow (\hat{\theta}^* - \theta^*)^2$ (559-594)

;

AIC, BIC dengan dasar m_{fit}; susun `df_eblup` (`domain, subarea, y, eblup, vardir, random_effect_area, random_effect_subarea, mse, rse`) (597-631);

9.6 Pseudo-code

9.7 Signature, argumen, objek keluaran, dan contoh

Listing 11: Signature `eblup_twofold` (`R/eblup_twofold.R:76-93`).

```
eblup_twofold(formula, vardir, domain = NULL, subarea = NULL, data,
              method = c("REML", "ML"),
              mse_type = c("analytical", "bootstrap", "none"),
              B = 100, seed = -1, maxiter = 100,
              precision = 1e-4, self_benchmark = FALSE,
              benchmark_weight = NULL, benchmark_target = NULL,
              benchmark_group = NULL, print_result = TRUE)
```

- `domain` — wajib (bila `NULL` fungsi berhenti); `subarea` — opsional, default penomoran unit.
- `mse_type` — `"analytical"` (default), `"bootstrap"` (B default 100), atau `"none"` (`mse = NA`).
- `seed` — default `-1` (tanpa seed; `NULL` juga diterima). Seed tetap dipasang lewat `.with_seed()` pada semua jalur (TFH-7), lalu aliran RNG sesi dipulihkan.

Objek keluaran: `"fastsae"` berisi `random_effect_var` bernama `sigma2_v` (area) dan `sigma2_u` (sub-area), `estcoef`, `df_eblup`, `goodness` (`loglikelihood, AIC, BIC` dengan AIC $= -2\ell + 2(p+2)$ dan BIC $= -2\ell + (p+2)\log m_{\text{fit}}$), `n_iter`, `convergence`, `method` (`"REML"` atau `"ML"`), `level = "subarea"`, `model = "TWOFOLD"`.

Listing 12: Contoh yang dapat dijalankan (dari `man/eblup_twofold.Rd`); membutuhkan kolom penanda area dan sub-area.

```
library(fastsae)
m1 ← eblup_twofold(y ~ x1 + x2, data = your_data, vardir = "vardir",
                   domain = "area", subarea = "kec")
m1$random_effect_var        # sigma2_v (area), sigma2_u (subarea)

m2 ← eblup_twofold(y ~ x1 + x2, data = your_data, vardir = "vardir",
                   domain = "area", subarea = "kec",
                   mse_type = "bootstrap", B = 100, seed = 42)
```

9.8 Referensi kunci

Rujukan model dua-lipat: Torabi and Rao [2014] (disebut `DESCRIPTION` beserta DOI-nya dan menjadi dasar komentar kode `eblup_twofold.cpp:19-24`); kerangka EBLUP: Rao and Molina [2015]; pembanding: `sae` [Molina and Marhuenda, 2015].

10 Model hierarki Bayes area-level: `hb_area`

10.1 Setting data dan kapan dipakai

Data berbentuk FH (satu baris per domain: respons y_d, kovariat x_d, dan opsional ψ_d), tetapi estimasi dilakukan sepenuhnya dalam kerangka Bayes lewat INLA [Rue et al., 2009, Lindgren and Rue, 2015]. Fungsi ini paling luas cakupannya di paket ini: enam family likelihood, enam struktur spasial, lima struktur temporal, dan tujuh pilihan interaksi spatio-temporal. Cocok ketika (i) indikator non-Gaussian (rasio, proporsi, jumlah) perlu dimodelkan langsung, atau (ii) struktur spasial/temporal eksplisit diinginkan dengan ketidakpastian penuh posterior.

10.2 Persamaan model

Kerangka umum yang dirangkai fungsi ini adalah model latent Gaussian:

$$g(\mu_d) = x_d'\beta + \varphi_d, \qquad y_d \mid \mu_d \sim \mathcal{F}(\mu_d; \tau), \qquad \varphi \sim N(0,\ \tau^{-1}Q^{-1}), \tag{61}$$

dengan g link family (default identitas, logit, log; binomial dan beta juga menerima probit, cloglog, loglog lewat `inla_engine(link = )`) dan Q struktur presisi yang dipilih lewat argumen `spatial`, `temporal`, dan `st_interaction`. Struktur spasial yang tersedia persis seperti yang didefinisikan INLA (baris 570-649 `R/hb_area.R`):

- `"none"`: φ_d i.i.d. — hanya dibuat bila tidak ada interaksi lain (baris 571-574);
- `"bym2"`: BYM2 dengan presisi $Q = \tau(s\,\varphi_b Q_{\text{bym2}} + (1-s)\mathbf{I})$ pada skala parsimonius (bagian 13.3);
- `"bym"`: BYM klasik, dua parameter presisi (`prec.unstruct`, `prec.spatial`);
- `"besag"`: ICAR murni Q_{ICAR};
- `"generic1"` (alias `"leroux"`): $Q = \tau(\mathbf{I} - (\beta/\lambda_{\max})C)$ dengan $C = \mathbf{I} - R$ dan R matriks struktur ICAR (bentuk Leroux);
- `"slm"`: model lag spasial $x = (\mathbf{I} - \rho\mathbf{W})^{-1}(\mathbf{X}\beta + \varepsilon)$ dengan ρ dipetakan ke $[0, 1]$ lewat rentang akar eigen $\mathbf{W}$.

Struktur temporal: `"none"`, `"rw1"`, `"rw2"`, `"ar1"`, `"iid"` pada indeks waktu `..time_id..` (baris 651-728), dengan interaksi opsional yang dipetakan ke empat tipe Knorr–Held (Tabel 27).

Tabel 27: Pemetaan `st_interaction` ke bentuk interaksi Knorr–Held. Sumber: penyusunan string `f(...)` di `R/hb_area.R:651-728`.

Pilihan kode	Bentuk presisi	Padanan KH
`none`	Q_{time} saja (efek utama waktu)	—
`domain-specific`	grup per domain, model temporal	(dipakai tipsae)
`separable`	$Q_{\text{space}} \otimes Q_{\text{time}}$	—
`type1`	$\mathbf{I}_{DT}$ i.i.d. + efek waktu	KH I
`type2`	$\mathbf{I}_D \otimes Q_{\text{time}}$ + efek utama	KH II
`type3`	$Q_{\text{ICAR}} \otimes \mathbf{I}_T$ + efek utama	KH III
`type4`	$Q_{\text{ICAR}} \otimes Q_{\text{time}}$ + efek utama	KH IV

Interaksi KH II–IV adalah term *tambahan* pada salinan indeks (`..time_id2..`, `..domain_id2..`), karena INLA hanya mengizinkan satu `f()` per kovariat. type2 memakai `f(..time_id2.., model = temporal, group = ..domain_id.., control.group = iid)`; type3/type4 memakai `f(..domain_id2.., model = "besag", group = ..time_id..)` dengan `control.group iid` (type3) atau `temporal` (type4). type3/type4 mensyaratkan efek spasial terstruktur (`bym2`/`bym`/`besag`/`leroux`). `separable` dengan `leroux` memakai `besagproper2`, model Leroux yang sama, karena `generic1` ber-group menolak diagonal `Cmatrix` negatif.

Implementation note. Sebelum rilis 1.0.0, type2 selalu gagal (indeks waktu ganda, HB-1), type3/type4 dengan `spatial` $\neq$ `"none"` selalu gagal (indeks domain ganda, HB-2), dan type3/type4 dengan `spatial = "none"` diam-diam tidak membuat interaksi (HB-3). Kini kombinasi terakhir dihentikan dengan pesan, demikian juga `slm` + `separable` (HB-5).

10.3 Prediktor titik

Prediksi diambil dari ringkasan nilai terpasang posterior (`inla_utils.R:296-305`):

$$\begin{aligned} \hat{\theta}_d = \mathbb{E}[\mu_d \mid y] \text{ (skala respons, lewat link invers)}, \quad \mathrm{sd}_d = \sqrt{\mathrm{Var}(\mu_d \mid y)}, \\ \mathrm{CI}_{95\%} = (q_{0.025}, q_{0.975}) \text{ (kuantil posterior)}. \end{aligned} \tag{62}$$

Untuk area-level, nilai terpasang INLA memang sudah mewakili rata-rata area, sehingga tidak ada transformasi skala tambahan. Kolom turunan dihitung sesuai family: `trials` → `estimated_total` $= \hat{\theta} \times$ trials; `exposure` → `rate` dan `estimated_count` $= \hat{\theta} \times$ exposure.

Implementation note. Domain tanpa sampel tidak pernah membuat baris baru: baris yang hilang dari `data` tidak dievaluasi INLA, sedangkan baris dengan `y = NA` tetap dievaluasi karena `na.pass` (HB-17). Jadi "unsampled" di sini berarti *baris ada tetapi responsnya NA*, bukan domain yang tidak ada dalam data — kontras dengan `hb_unit()` yang menambahkan satu baris `NA` per domain.

10.4 Estimasi komponen varians

Tidak ada Fisher scoring: seluruh inferensi diserahkan ke `INLA::inla()` dengan pengaturan berikut (`R/hb_area.R:738-783`):

- `control.compute` = `dic, waic, cpo, mlik, config` semuanya `TRUE`;
- `control.inla` = `strategy` pilihan pengguna (`"laplace"` default);
- `control.predictor` = `compute = TRUE, link = 1`, dan `control.family$control.link$model` = link pilihan pengguna (divalidasi per family; sebelum 1.0.0 argumen `link` tidak pernah dipakai, HB-6). Draws posterior untuk *self-benchmarking* dan `benchmark_sae()` memakai invers-link yang sama;
- prior komponen acak spasial/temporal: `prior_prec` default `pc.prec(1, 0.01)`; prior korelasi spasial `prior_phi` default `pc(0.5, 0.5)`; `prior_prec_time` default juga `pc.prec(1, 0.01)` (baris 559-566);
- `control.fixed` hanya diset untuk *self-benchmarking* WFQ (prior difus agar kovariat augmentasi tidak menyusut, baris 765-775); selain itu berlaku default INLA: rata-rata tetap 0, presisi tetap 0.001 (intersep: presisi 0) (HB-21);
- argumen `INLA::inla()` tambahan dari `inla_engine(...)` digabung dengan `modifyList`, sehingga daftar `control.*` dapat ditimpa per elemen (baris 762-763; HB-13 tidak berlaku lagi).

Varians efek acak yang dilaporkan (`random_effect_var`) adalah rerata posterior varians $\mathbb{E}[1/\tau \mid y]$ dari marginal presisinya (`.post_mean_variance`, `inla_utils.R:20-36`), bukan $1/\mathbb{E}[\tau \mid y]$ yang lebih kecil karena ketaksamaan Jensen. Untuk `spatial = "bym"` yang dilaporkan adalah komponen spasial (sebelum 1.0.0: komponen iid, HB-9), dan term interaksi ruang–waktu tidak pernah dibaca sebagai efek utama. Likelihood Gaussian dengan `vardir` memakai `scale` $= 1/\psi_d$ dan presisi tetap 1 sehingga $\mathrm{Var}(y_d) = \psi_d$ persis; family lain memakai `scale`/(`Ntrials`, `E`) sesuai parameterisasi INLA (baris 785-841). Pengaturan engine diberikan lewat `engine = inla_engine(strategy, prior_*, ...)` atau `engine = jax_engine(...)` (MCMC NumPyro). Argumen lama `method = "laplace"` kini usang dan dipetakan ke `inla_engine(strategy = "laplace")`; jalur `lme4` (HB-14, HB-15) telah dihapus.

Perangkat engine JAX. `jax_engine(device = ...)` memilih CPU, GPU NVIDIA (`"cuda"`) atau GPU Apple (`"metal"`). JAX mengunci perangkatnya pada fit pertama satu sesi R; `init_jax_environment` (`inst/python/hb_area_numpyro.py:76`) menyetel `JAX_PLATFORMS` dan `jax_default_device`. Pada `"auto"`, CUDA dipakai bila ada GPU NVIDIA, selain itu CPU (juga di macOS). Untuk model spasial, efek ICAR/BYM2 dinyatakan lewat basis eigen $D \times D$ Laplacian graf (`.icar_spectral`, `R/hb_area_jax.R:27`): biaya $O(D^3)$ sekali dan $O(D^2)$ per evaluasi gradien, sehingga waktu engine JAX tumbuh cepat dengan D (bagian 3.6).

Implementation note. jax-metal 0.1.1 tidak dapat mengompilasi NUTS NumPyro apa adanya. Dua operasinya tidak didukung: (i) `popcnt` (`jnp.bitwise_count`) pada indeks checkpoint U-turn, dan (ii) `lax.cond` yang semua cabangnya mengeluarkan konstanta ("Unable to serialize MPS module"). Selain itu, loop rantai yang digabung menjadi satu kernel tidak pernah selesai di Metal. Bila perangkatnya Metal, `_patch_numpyro_for_metal` (baris 42) mengganti (i) dengan hitung bit *shift/mask* yang identik (`_popcount32`, baris 25; diuji terhadap NumPyro untuk 0–4095) dan melewatkan keluaran setiap cabang (ii) lewat `lax.optimization_barrier`, yang tidak mengubah nilai. Rantai dijalankan sekuensial dan per langkah dari Python (baris 452–508). Draw Metal sama dengan draw CPU hingga galat pembulatan. Bila kompilasi tetap gagal, fit kembali ke CPU dengan peringatan (`.warn_device_fallback`, `R/python_env.R:145`).

10.5 Estimasi MSE

MSE dihitung dari momen posterior (`inla_utils.R:299-304`):

$$\widehat{\mathrm{MSE}}(\hat{\theta}_d) = \mathrm{sd}_d^2, \qquad \mathrm{RSE}_d = 100\,\mathrm{sd}_d / \left|\hat{\theta}_d\right|, \qquad \mathrm{CI} = \hat{\theta}_d \pm z_{0.975}\,\mathrm{sd}_d \text{ (jalur fallback).} \tag{63}$$

Tanpa benchmarking, ketidakpastian murni dari ringkasan marginal INLA. Dengan `self_benchmark = TRUE` dan `benchmark_method = "posterior"` (default), `inla.posterior.sample` menarik `draws` sampel prediktor linier (`inla_engine(draws, seed)`), setiap sampel di-*benchmark* (rasio untuk family positif, selisih untuk Gaussian), dan $\hat{\theta}$, sd, MSE, serta CI dihitung dari sampel terbenchmark (`.posterior_benchmark`, `inla_utils.R:399-427`); estimasi sebelum benchmarking disimpan di `hb_unbenchmarked`. Fungsi ini membutuhkan paket `sn`; tanpa itu dipakai metode WFQ (baris 358-368).

10.6 Mode INLA dan validasi terhadap Stan

INLA menyediakan dua formulasi. Mode `"classic"` (Rue, Martino & Chopin 2009) memasukkan prediktor linier η ke dalam latent field sehingga marginalnya dihitung langsung (Laplace atau *simplified Laplace*). Mode `"compact"`, default INLA sekarang (van Niekerk, Krainski, Rustand & Rue 2023), mengeluarkan η dari latent field, menghitung marginalnya sesudah inferensi sebagai Gaussian bersyarat pada hiperparameter, dan mengoreksi rataannya dengan koreksi Variational Bayes berperingkat rendah; mode ini dirancang untuk model *data-rich* ($n \gg$ ukuran latent field). Pada SAE level area, n kira-kira sama dengan jumlah efek area, sehingga keuntungan kecepatannya hampir tidak ada.

`hb_area` kini memakai `inla.mode = "classic"` untuk `family = "gaussian"` dan mempertahankan `compact` untuk family lain. Dasarnya perbandingan dengan Stan (cmdstanr 2.39, 4 rantai $\times$ 4000 draw, tanpa divergensi, ESS $\geq$ 4700) dengan model dan prior identik: prior PC pada presisi ($\sigma \sim \mathrm{Exp}(-\log 0{,}01)$), ICAR ter-*scale* dengan kendala jumlah nol, BYM2 parametrisasi Riebler dengan $\phi \sim U(0,1)$ (di INLA `prior_phi = logitbeta(1, 1)`), β dengan prior bawaan INLA. Data: grid 7×7 (84 sisi), 12 area tak tersampel; RW1 pada `mys_panel`. Kolom memuat selisih rata-rata maksimum dibagi SD posterior Stan dan rentang rasio SD INLA/Stan atas semua area.

Konfigurasi	compact		classic	
	$\lvert\Delta\mu\rvert$/sd	rasio SD	$\lvert\Delta\mu\rvert$/sd	rasio SD
Gaussian iid (3 data set)	$\leq 0{,}24$	1,03–1,18	$\leq 0{,}03$	0,96–1,02
Gaussian Besag (3)	$\leq 0{,}23$	1,02–1,16	$\leq 0{,}06$	0,92–1,02
Gaussian BYM2 (3)	$\leq 0{,}25$	1,02–1,15	$\leq 0{,}02$	0,95–1,02
Binomial iid / BYM2	$\leq 0{,}05$	0,97–1,02	$\leq 0{,}03$	0,98–1,02
Poisson iid / BYM2	$\leq 0{,}08$	0,96–1,06	$\leq 0{,}02$	0,96–1,02
Beta iid / BYM2	$\leq 0{,}10$	0,97–1,02	$\leq 0{,}02$	0,97–1,02
Gamma iid / Besag	$\leq 0{,}09$	0,96–1,03	$\leq 0{,}02$	0,97–1,02
Gaussian iid + RW1 (`mys_panel`)	0,04	0,97–1,03	0,03	0,97–1,03

Pada `mys` ($\sigma_u^2 \approx 1{,}9$, besar dibanding `vardir`) kedua mode akurat terhadap posterior eksak (integrasi numerik 1D). Galat Monte Carlo Stan sekitar 0,013 SD untuk rata-rata dan 1% untuk SD. Untuk Gaussian waktu kedua mode sama (D sampai 2025, BYM2 $\approx$ 10 detik). Untuk family lain `classic` jauh lebih lambat dan tumbuh kira-kira kuadratik dalam n (beta iid: $n = 1000$ 9,9 vs 1,0 detik, $n = 2000$ 32 vs 1,3 detik; binomial $n = 2000$ 12 vs 1,6 detik), padahal `compact` sudah akurat (rerata $\leq$ 0,16 SD, rasio SD 0,95–1,08 atas semua run); karena itu `classic` hanya untuk Gaussian. Kelemahan `classic`: ringkasan σ^2 Gaussian (`random_effect_var`) bias ke bawah saat σ^2 kecil (misalnya 0,055 vs Stan 0,118), sedangkan `compact` mendekati Stan; estimasi area tetap akurat. Belum divalidasi: Leroux/`generic1`, SLM, interaksi ruang–waktu tipe I–IV, dan `nbinomial`.

10.7 Pseudo-code

10.8 Signature, argumen, objek keluaran, dan contoh

Listing 13: Signature `hb_area` (`R/hb_area.R:231-253`) dan `inla_engine` (`R/engine.R:83`).

```
hb_area(formula, data, domain = NULL, time = NULL,
        family = c("gaussian","binomial","poisson","nbinomial","beta","gamma"),
        spatial  = c("none","bym2","bym","besag","generic1","leroux","slm"),
        temporal = c("none","rw1","rw2","ar1","iid"),
        st_interaction = c("none","domain-specific","separable",
                           "type1","type2","type3","type4"),
        W = NULL, vardir = NULL, trials = NULL, exposure = NULL,
        engine = "inla",   # atau inla_engine(...) / jax_engine(...)
        self_benchmark = FALSE, benchmark_weight = NULL,
        benchmark_target = NULL, benchmark_group = NULL,
        benchmark_method = c("posterior","wfq"),
        print_result = TRUE, method = NULL, ...)

inla_engine(strategy = c("laplace","simplified.laplace","gaussian"),
            link = NULL, scale_model = TRUE,
            prior_prec = NULL, prior_phi = NULL, prior_rho = NULL,
            prior_prec_time = NULL, prior_rho_time = NULL,
            prior_prec_subarea = NULL, draws = NULL, seed = 0L, ...)
# NULL = default fungsi: hb_area memakai pc.prec(1, 0.01),
# pc(0.5, 0.5) untuk phi, dan draws = 1000

jax_engine(warmup = 500, samples = 1000, chains = 2,
           device = c("auto","cpu","cuda","metal"), seed = 42,
           prior_beta = c("normal","horseshoe"), robust = FALSE,
           keep_draws = TRUE, progress = interactive())
```

- `family` menentukan kolom pendukung yang wajib: Gaussian → `vardir` (opsional; bila

Algorithm 7:

Masukan *formula, data, domain, time, family, spatial, temporal, st_interaction, W, vardir, trials, exposure, engine* **Keluaran** *objek fastsae_hb_area berisi df_hb, fit, hyperpar* `match.arg` untuk argumen pilihan; `"leroux"` → `"generic1"` (255-260);

`engine` ← `.resolve_inla_engine` (argumen lama → `inla_engine`, dengan peringatan) (261-278);

`domain_vec` ← `.get_variable`; `time` wajib bila `temporal` ≠ `"none"` (286-310);

`mf` ← `model.frame(na.pass)`; y ← respons (313-314);

if *family* = *binomial* **then**

  `trials` wajib; y ← `round`$(y \cdot \text{trials})$ bila pecahan (323-339)

if *family* = *gamma* **then**

  $y > 0$ wajib (341-346)

;

W_{obj} ← `.convert_spatial_weights` (atau dari `sf`) bila `spatial` ≠ `"none"` (348-356);

if *self_benchmark* **then**

  siapkan target; WFQ: augmentasi model (358-395)

if *engine* = *jax_engine()* **then**

  kembalikan `.fit_jax_area` (MCMC NumPyro) (400-421)

;

; *// .fit_inla_area* (473-908)

indeks `..domain_id..`, `..obs_id..`, `..time_id..` (520-538);

`hyper_time` ← {`prec` : `pc.prec`$(1, 0.01)$}; tambah `rho` bila `ar1` (559-566);

baris spasial ← Tabel 27/teks bagian 10.2 (570-649);

baris temporal + interaksi ← Tabel 27 (651-728);

susun formula `y ~ ...` (`"slm"`: tanpa intercept) (730-736);

`control.compute` (dic, waic, cpo, mlik, config), `control.predictor` (`link = 1`), `control.inla`; gabung argumen `inla_engine` (738-783);

for *family* **do**

  sesuaikan `scale`, `Ntrials`, `E`, `control.family` — (61) (785-841)

;

`fit` ← `tryCatch(do.call(INLA::inla, ...))` (843-852);

$\hat{\theta}$, sd, CI, `mse`, `rse` ← (62)–(63) (`.extract_inla_results`, 854-875);

if *self_benchmark posterior* **then**

  `.posterior_benchmark` (877-899)

perakitan `df_hb`, `hyperpar`, `goodness` (DIC, WAIC, marginal likelihood), kelas `c("fastsae_hb_area","fastsae_hb","fastsae")`;

ada, dipakai sebagai `scale`); Binomial → `trials`; Poisson/Negatif Binomial → `exposure`; Beta → `vardir` atau `trials` untuk presisi.

- `W` — matriks ketetanggaan (dapat berupa `listw`, `nb`, `matrix`, atau `inla.graph`); bobot dibakar ke graf biner simetris (bagian 13.3).
- `engine` — `"inla"`/`inla_engine(...)` (default) atau `jax_engine(...)`; argumen INLA yang dulu diberikan langsung ke `hb_area()` (`method`, `strategy`, `prior_*`, `...`) masih diterima dengan peringatan dan diteruskan ke `inla_engine()`.
- `W` kosong dengan `data` berkelas `sf`: matriks ketetanggaan queen biner dibangun otomatis lewat `create_weights()`.

Objek keluaran: `c("fastsae_hb_area", "fastsae_hb", "fastsae")` berisi `df_hb` (kolom `domain, y, hb, sd, mse, rse, ci_lower, ci_upper, linear_pred` + kolom family), `fit` (objek `inla`), `hyperpar`, `goodness`, `random_effect_var`, `engine`, `engine_spec`, `W`, `n_iter`, `convergence`, `formula`, `method`, `call`, `data`.

Listing 14: Contoh yang dapat dijalankan (dari `man/hb_area.Rd`).

```
library(fastsae)
if (requireNamespace("INLA", quietly = TRUE)) {
  # Gaussian Fay-Herriot tanpa spasial
  m_norm ← hb_area(y ~ x1 + x2 + x3, data = mys, vardir = "vardir",
                   family = "gaussian")

  # Gaussian dengan efek spasial BYM2
  m_bym2 ← hb_area(y ~ x1 + x2 + x3, data = mys, vardir = "vardir",
                   family = "gaussian", spatial = "bym2",
                   W = mys_proxmat)
  head(m_norm$df_hb)
}
```

10.9 Referensi kunci

Rue et al. [2009] untuk metode INLA, Lindgren and Rue [2015] untuk implementasi R-INLA, Riebler et al. [2016] untuk BYM2 (blok roxygen fungsi ini memuat lima rujukan: Rao and Molina [2015], Riebler et al. [2016], Marhuenda et al. [2013], De Nicolò and Gardini [2024], Rue et al. [2009]; lihat blok `@references` di `R/hb_area.R`).

11 Model hierarki Bayes unit-level: `hb_unit`

11.1 Setting data dan kapan dipakai

Mirip `eblup_bhf()`: dua sumber data — `unit_data` (mikrodata survei) dan `Xpop` (ringkasan populasi per domain) — tetapi inferensinya Bayes penuh lewat INLA (hanya `inla_engine()`). `popsize_var` bersifat opsional; bila diberikan, prediksi dikombinasikan dengan rata-rata sampel (koreksi populasi hingga). Dibandingkan `hb_area()`, fungsi ini secara eksplisit membangkitkan *baris prediksi* untuk setiap domain, sehingga domain tanpa sampel selalu ikut dievaluasi. Family yang didukung hanya `gaussian`, `binomial`, `poisson`, dan struktur spasial hanya `none`, `bym2`, `besag`.

11.2 Persamaan model

$$y_{ij} = x'_{ij}\beta + u_i + e_{ij}, \qquad u \sim N(0,\ \tau_u^{-1}Q_u^{-1}), \qquad e_{ij} \sim \mathcal{F}, \tag{64}$$

dengan $Q_u = \mathbf{I}$ (`spatial = "none"`), $Q_u = Q_{\text{bym2}}$, atau $Q_u = Q_{\text{ICAR}}$ (baris 280-294). Tidak ada `Ntrials` maupun `E` yang dikirim ke INLA (baris 303-311), sehingga binomial terbaca sebagai Bernoulli per unit dan Poisson tanpa offset/ekspresi. Formula acak yang dirangkai (baris 280-294):

```
y ~ <fixed_str> + f(..dom_id.., model = 'iid',
                    hyper = list(prec = prior_prec))
```

11.3 Prediktor titik

Data INLA disusun dari baris sampel (y nyata) *ditambah satu baris prediksi per domain dengan* `y = NA` (baris 244-278); prediksi dibaca dari indeks `(n_sample+1):nrow(combined_df)` (baris 334-342):

$$\hat{\theta}_i = \mathbb{E}[\mu_i \mid y], \qquad \text{sd}_i = \sqrt{\text{Var}(\mu_i \mid y)}, \tag{65}$$

skala respons (superpopulasi). Bila `popsize_var` diberikan, rerata domain populasi hingga adalah $\bar{Y}_d = f_d\,\bar{y}_{s,d} + (1-f_d)\bar{Y}_{d,r}$, $f_d = n_d/N_d$ (dipangkas ke $[0,1]$), dan baris prediksi dievaluasi pada rata-rata kovariat *non-sampel* $\bar{x}_{d,r} = (N_d\bar{X}_d - n_d\bar{x}_{s,d})/(N_d - n_d)$, bukan pada $\bar{X}_d$ — sama seperti EBLUP BHF (baris 257-274). Lalu (baris 350-390)

$$\begin{aligned} \hat{\theta}_d^{\text{adj}} &= f_d\,\bar{y}_{s,d} + (1-f_d)\,\hat{\theta}_{d,r}, \qquad \text{CI} = \hat{\theta}_d^{\text{adj}} \pm 1.96\,\text{sd}^{\text{adj}}, \\ \text{sd}^{\text{adj}} &= (1-f_d)\sqrt{\text{Var}(\mu_{d,r} \mid y) + \hat{\sigma}_e^2/(N_d - n_d)}, \end{aligned} \tag{66}$$

dengan suku $\hat{\sigma}_e^2/(N_d - n_d)$ (rerata posterior σ_e^2) hanya untuk Gaussian, dan penjagaan batas CI sesuai family (binomial $\to [0,1]$, Poisson ≥ 0). Untuk domain tanpa sampel $f_d = 0$, sehingga $\hat{\theta}_d^{\text{adj}} = \hat{\theta}_d$ — prediktor murni. Tanpa `popsize_var`, kuantil posterior INLA dipakai apa adanya (baris 391-397).

Implementation note. Pada jalur `popsize_var`, kolom `linear_pred` tetap memuat η baris prediksi tanpa koreksi, sementara `hb` sudah tercampur $f_d\bar{y}_s + (1-f_d)\hat{\theta}_{d,r}$, sehingga keduanya tidak lagi saling invers; sd-nya aproksimasi dari momen, bukan hasil posterior (HB-19). Jenis interval juga berbeda: kuantil posterior bila tanpa `popsize_var`, dan normal ± 1.96 bila ada (HB-18).

11.4 Estimasi komponen varians

- Prior presisi komponen acak: `prior_prec` default `loggamma(0.01, 0.01)`, berbeda dari `hb_area()` dan `hb_twofold()` yang memakai `pc.prec(1, 0.01)` (HB-8). Ini disengaja: PC prior itu memberi peluang 0,01 pada simpangan baku > 1 dan terlalu menyusutkan efek area pada respons berskala mentah (pada data `cornsoybean`, korelasi HB–EBLUP turun dari $> 0{,}85$ ke 0,84).
- `strategy` default `"laplace"`; `num.threads` dipaksa `"1:1"` dengan pemulihan `on.exit` (baris 296-301), sehingga fungsi ini selalu serial.
- **Start REML untuk pencarian mode.** Prior log-gamma yang samar menyisakan punggungan hampir datar menuju $\sigma_u^2 = 0$ tempat pencarian mode INLA dapat berhenti (pada `cornsoybean` $\sigma_u^2 \approx 1$ alih-alih ≈ 60, bergantung jumlah thread). Untuk

Gaussian non-spasial, pencarian mode dimulai dari estimasi REML BHF (`.bhf_fit`, bagian 8): `control.mode = list(theta = -log(c(`$\hat{\sigma}_e^2, \hat{\sigma}_u^2$`)), restart = TRUE)` (baris 312-318, `.hb_unit_reml_start` 539-548), dengan urutan hiperparameter INLA presisi likelihood lalu presisi efek acak. `control.mode` dari `inla_engine(...)` tetap didahulukan.

- **Mode `"classic"`.** Dengan mode default INLA (`"compact"`) panggilan identik masih memberi 2–3 hasil berbeda walau nilai awal REML-nya sama. Terhadap posterior eksak (integrasi numerik pada grid $(\log \tau_e, \log \tau_u)$; dengan hiperparameter tetap posterior (β, u) Gaussian), mode `compact` meleset sampai 2,0 pada rerata county dan SD-nya 8–15% terlalu besar, sedangkan `classic` selisih $\leq 0{,}31$ dengan rasio SD 0,995–1,014 dan jauh lebih stabil (antar-konteks eksekusi rerata berbeda $< 0{,}1\%$ dan SD $< 2\%$, tidak bit-identik). Karena itu `inla.mode = "classic"` dipasang untuk Gaussian (bila `INLA::inla` mengenal argumen itu); binomial/Poisson tetap `compact` karena `classic` jauh lebih lambat (binomial, 4000 unit: 72 vs 2,6 detik); `inla_engine(inla.mode = "compact")` mengembalikan default INLA. Untuk binomial/Poisson rerata domain diprediksi pada rata-rata kovariat, $g^{-1}(\bar{x}_{d,r}^{\top}\beta + u_d)$, yang bias untuk link nonlinier (Jensen): pada simulasi -8% (binomial) dan -16% (Poisson), coverage 90% dan 78%. Fungsi memberi warning; estimasi tak bias butuh kovariat unit populasi (EBP, Molina & Rao 2010). Ringkasan σ_u^2 (`random_effect_var`) tetap tidak andal: posterior eksak sangat miring (rerata 40,9, median 7,0), INLA melaporkan ≈ 70.
- `control.compute = dic, waic, cpo, mlik` tanpa `config` (baris 309); argumen `inla_engine(...)` digabung lewat `modifyList`, sehingga daftar `control.*` dapat ditimpa per elemen (baris 319-320).

Estimasi $\hat{\sigma}_u^2$ dan, untuk Gaussian, $\hat{\sigma}_e^2$ adalah rerata posterior varians $\mathbb{E}[1/\tau \mid y]$ dari marginal presisi (`.post_mean_variance`; baris 415-438); `phi` diambil bila ada (baris 440-444).

11.5 Estimasi MSE

$$\widehat{\mathrm{MSE}} = \mathrm{sd}^2, \qquad \mathrm{RSE} = 100\,\mathrm{sd}/\left|\hat{\theta}\right|, \tag{67}$$

dengan sd sudah melalui (66) bila `popsize_var` diberikan (baris 459-460). Varians langsung untuk keperluan pelaporan diestimasi dari data: Gaussian $\to \hat{\sigma}_e^2/n_d$, binomial $\to \bar{p}(1-\bar{p})/n$, Poisson $\to \bar{y}/n$ (baris 447-456). Tidak ada posterior sampling di fungsi ini.

11.6 Pseudo-code

11.7 Signature, argumen, objek keluaran, dan contoh

Listing 15: Signature `hb_unit` (`R/hb_unit.R:123-136`).

```
hb_unit(formula, unit_data, Xpop, domain_var, popsize_var = NULL,
        family = c("gaussian","binomial","poisson"),
        spatial = c("none","bym2","besag"),
        W = NULL, popnmean_xpop = NULL,
        engine = "inla",    # atau inla_engine(strategy, scale_model,
                            #   prior_prec = loggamma(0.01, 0.01) bila NULL,
                            #   prior_phi = pc(0.5, 0.5) bila NULL, ...)
        print_result = TRUE, ...)  # ...: argumen lama, usang
```

Objek keluaran: `c("fastsae_hb_unit", "fastsae_hb", "fastsae")` berisi `df_hb` (`domain, y, hb, linear_pred, vardir, sd, mse, rse, ci_lower, ci_upper, samp_size,`

Algorithm 8: `hb_unit` — alur sebenarnya (`R/hb_unit.R:123-548`)

Masukan *formula,* ***`unit_data`****,* ***`Xpop`****,* ***`domain_var`****,* ***`popsize_var`****,* ***`family`****,* ***`spatial`****,* W*,* ***`popnmean_xpop`****,* ***`engine`*** **Keluaran** *objek* ***`fastsae_hb_unit`*** *berisi* ***`df_hb`****,* ***`fit`****,* ***`goodness`*** `engine` ← `.resolve_inla_engine`; hanya INLA (137-150);

`.check_inla_installed()`; `match.arg` (153-156);

sampel ← `model.frame(na.omit)`; y_s ← respons (182-190);

`all_domains` ← `unique(Xpop[[domain_var]])`;

gagal bila ada domain sampel tanpa di `Xpop` (200-206);

`unsampled_domains` ← `setdiff(all_domains, unique(dom_s))` (209);

bila `nrow(Xpop)` > jumlah domain: agregasi `mean` per domain (211-221);

`meanxpop` ← `model.matrix` atas ringkasan domain (227-235);

W ← `.convert_spatial_weights` (238-242);

data INLA ← baris sampel + **1 baris `y = NA`** per domain, pada $\bar{x}_{d,r}$ bila `popsize_var` (244-278);

formula ← `y ~ <fixed> + f(..dom_id.., ...)` (280-294);

`num.threads` ← `"1:1"` (dipulihkan via `on.exit`) (296-301);

if *Gaussian non-spasial* **then**

  `control.mode` ← start REML (312-318)

`fit` ← `do.call(INLA::inla, ...)` (322-331);

$\hat{\theta}$, sd, CI ← prediksi indeks `(n_sample+1):n` (333-342);

if ***`popsize_var`*** *diberikan* **then**

  f_d ← (66); $\hat{\theta}^{\text{adj}}$, sd^{adj}, CI ← (350-390)

else

  CI ← kuantil INLA (391-397)

;

$\hat{\sigma}_u^2, \hat{\sigma}_e^2 \leftarrow \mathbb{E}[1/\tau \mid y]$; `phi` (415-445);

MSE, RSE ← (67); `goodness` = list(DIC, WAIC, *mlik*) (458-491);

kelas `c("fastsae_hb_unit","fastsae_hb","fastsae")` (520);

`sample_mean, random_effect`; ditambah `pop_size` dan `estimated_total` bila `popsize_var` ada), `estcoef`, `hyperpar`, `random_effect_var`, `residual_var`, `phi`, `goodness` berupa `list(dic, p_eff_dic, waic, p_eff_waic, mlik)`, `unsampled_domains`, `method`, `fit`, `engine`, `engine_spec`, `call`.

Listing 16: Contoh yang dapat dijalankan (dari `man/hb_unit.Rd`).

```
library(fastsae)
data(cornsoybean); data(cornsoybeanmeans)

df_pop ← cornsoybeanmeans
names(df_pop)[names(df_pop) == "MeanCornPixPerSeg"]    ← "CornPix"
names(df_pop)[names(df_pop) == "MeanSoyBeansPixPerSeg"] ← "SoyBeansPix"
df_sample ← cornsoybean
names(df_sample)[names(df_sample) == "County"] ← "CountyIndex"

fit_hb_u ← hb_unit(formula = CornHec ~ CornPix + SoyBeansPix,
                   unit_data = df_sample, Xpop = df_pop,
                   domain_var = "CountyIndex",
                   popsize_var = "PopnSegments")
print(fit_hb_u)
```

11.8 Referensi kunci

Blok roxygen fungsi ini memuat Battese et al. [1988] (struktur unit-level), Rao and Molina [2015], Riebler et al. [2016], dan Rue et al. [2009] (blok `@references` di `R/hb_unit.R`); kerangka INLA dirujuk ke Lindgren and Rue [2015].

12 Model hierarki Bayes sub-area dua-lipat: `hb_twofold`

12.1 Setting data dan kapan dipakai

Data dan penanda sama dengan `eblup_twofold()`: satu baris per unit sampel dengan kolom `domain` (area, wajib) dan `subarea` (opsional, default penomoran unit). Perbedaannya inferensial: model ditempatkan dalam kerangka hierarki Bayes lewat INLA, dengan dua komponen acak terpisah (efek area dan efek sub-area) serta pilihan prediktor sintetis pada tingkat area. Fungsi ini diekspor dengan satu nama, `hb_twofold` (`NAMESPACE:14`); hanya `inla_engine()` yang didukung.

12.2 Persamaan model

$$y_{ij} = x'_{ij}\beta + v_i + u_{ij}, \qquad v \sim N(0,\ \tau_v^{-1}Q_v^{-1}), \qquad u_{ij} \overset{\text{iid}}{\sim} N(0,\ \tau_u^{-1}), \tag{68}$$

dengan Q_v dipilih `spatial` $\in$ {`none` (i.i.d.), `bym2`, `besag`} dan u selalu i.i.d. pada indeks sub-area (baris 250-266):

```
y ~ <fixed_str> + <rand_area> +
    f(..subarea_id.., model = 'iid',
      hyper = list(prec = prior_prec_subarea))
```

`rand_area` adalah `f(..area_id.., model='iid'|'bym2'|'besag', graph=..., hyper=list(prec = prior_prec_area[, phi = prior_phi]))`. Likelihood: Gaussian (wajib `vardir`, dipakai sebagai `scale` $= 1/\psi$, baris 216-224), Binomial (wajib `trials`), Poisson (`exposure` opsional; baris 286-296).

12.3 Prediktor titik

Prediksi sub-area diambil dari `summary.fitted.values` untuk $1..n_{\text{subarea}}$ (baris 312-318), lalu tingkat area diperoleh sebagai rata-rata berbobot (baris 352-368):

$$\hat{\theta}_i^{\text{area}} = \sum_{j \in i} w_{ij}\, \hat{\theta}_{ij}, \qquad w_{ij} = \frac{w_{ij}^{\text{user}}}{\sum_{k \in i} w_{ik}^{\text{user}}}, \qquad w_{ij} = \frac{1}{n_i} \text{ bila } \texttt{weight = NULL}. \tag{69}$$

Bobot yang tidak valid (NA/negatif) diubah menjadi 0; bila jumlah bobot area bernilai 0, kembali ke $1/n_i$ (baris 353-365).

12.4 Estimasi komponen varians

Sama seperti dua fungsi Bayes lain: INLA dengan `strategy` default `"laplace"`, `control.compute` berisi `dic, waic, cpo, mlik` plus `config` yang diaktifkan *hanya* bila `compute_area = TRUE` (karena butuh konfigurasi untuk posterior sampling), dan `num.threads = 1` (baris 269-285); argumen `inla_engine(...)` digabung lewat `modifyList` (baris 298-299). Fit Gaussian memakai `inla.mode = "classic"` (binomial/Poisson tetap `compact`, seperti `hb_area`): terhadap posterior eksak (integrasi numerik pada grid $(\log \tau_v, \log \tau_u)$, data simulasi vignette two-fold) mode default `"compact"` meleset sampai 0,096 pada rerata sub-area dan SD-nya 1–10% terlalu besar, sedangkan `classic` selisih $\leq$ 0,010 dengan rasio SD 0,984–0,998. Ringkasan σ_u^2 `classic` (0,047) lebih jauh dari nilai eksak (0,072) daripada `compact` (0,070); estimasi sub-area dan area tetap output yang andal. Prior: `prior_prec` (area) dan `prior_prec_subarea` keduanya `pc.prec(1, 0.01)`, `prior_phi pc(0.5, 0.5)` (baris 155-171). Hyperparameter akhir: $\hat{\sigma}_v^2$ dan $\hat{\sigma}_u^2$ adalah rerata posterior varians $\mathbb{E}[1/\tau \mid y]$ pada indeks area dan sub-area (`.post_mean_variance`), dan `phi` bila ada (baris 450-478).

12.5 Estimasi MSE

Tingkat sub-area (baris 330-331):

$$\widehat{\text{MSE}}_{ij} = \text{sd}_{ij}^2, \qquad \text{RSE}_{ij} = 100\, \text{sd}_{ij} / \left|\hat{\theta}_{ij}\right|. \tag{70}$$

Tingkat area memakai sampling posterior (baris 371-406), bila `compute_area = TRUE`:

1. `ps` ← `.inla_draws(fit, draws, seed)` dengan `inla_engine(draws = )` default 200 dan `seed` default 1. Helper ini memasang seed juga di sisi R (INLA memilih konfigurasi hiperparameter dengan RNG R, sehingga `seed` di `inla.posterior.sample` saja tidak reprodusibel; dulu `sd_area` berubah sampai 9%) lalu memulihkan aliran RNG sesi. Bobot NA/negatif diberi 0 dengan warning;
2. tarik baris `Predictor:1`..`Predictor:`$n_{\texttt{subarea}}$ dari `rownames(ps[[1]]$latent)`;
3. transformasi balik link manual (binomial $1/(1+e^{-x})$, Poisson e^x, Gaussian tanpa transformasi);
4. agregasi per gambar posterior $\theta_i^{\text{area},(k)} = \sum_j w_{ij} \theta_{ij}^{(k)}$;
5. $\text{sd}^{\text{area}} = \text{sd}(\theta^{\text{area}})$, CI dari kuantil 0.025/0.975 gambar posterior.

$$\widehat{\text{MSE}}_{\text{area}} = \left(\text{sd}^{\text{area}}\right)^2, \qquad \text{fallback bila sampling gagal: } \text{sd}^{\text{area}} = \sqrt{\textstyle\sum_j \left(w_{ij}\, \text{sd}_{ij}\right)^2}, \tag{71}$$

dengan fallback itu berasumsi sub-area saling bebas (baris 407-418), dan CI jatuh ke $\pm 1.96\,\text{sd}$ (HB-18). Bila `compute_area = FALSE`, `df_area = NULL` (baris 351).

12.6 Pseudo-code

Algorithm 9: `hb_twofold` — alur sebenarnya (`R/hb_twofold.R:136-520`)

Masukan *formula,* ***`vardir`*** *(wajib bila Gaussian),* ***`domain`*** *(wajib),* ***`subarea`****,* ***`data`****,* ***`family`****,* ***`spatial`****,* W*,* ***`weight`****,* ***`trials`****,* ***`exposure`****,* ***`engine`****,* ***`compute_area`*** **Keluaran** ***`df_hb`*** *(sub-area),* ***`df_area`*** *(area, bila diaktifkan),* ***`goodness`*** `engine` ← `.resolve_inla_engine`; hanya INLA (155-171);
`.check_inla_installed()` (172); `match.arg` (174-175);
`domain` wajib; `subarea` ← `1..n` bila `NULL` (186-203);
y, `terms` ← `model.frame(na.pass)` (206-207);
Gaussian: `vardir` wajib dan harus berhingga > 0 di sub-area tersampel (`.check_vardir`; sebelumnya diam-diam diganti 1), `scale_vec` $= 1/\psi$ (baris tak-tersampel NA/$\leq 0/\infty \to 1$) (216-224); Binomial: `trials` wajib (225-229);
W ← `.convert_spatial_weights` (234-238);
formula ← (68) (250-266);
`num.threads` ← `"1:1"`; `control.compute` = {dic, waic, cpo, mlik, `config = compute_area`} (269-285);
`fit` ← `do.call(INLA::inla, ...)` (302-310);
$\hat{\theta}_{ij}, \mathrm{sd}_{ij}, \mathrm{CI}$ ← (312-318); $\widehat{\mathrm{MSE}}$ ← (70) (330-331);
w_{ij} ← (69) (353-365); $\hat{\theta}_i^{\mathrm{area}}$ ← (368);
if ***`compute_area`*** **then**
  `.inla_draws`($n =$ `draws`, `seed`) ;
  transformasi balik link per gambar (385-390);
  agregasi berbobot per gambar; $\mathrm{sd}^{\mathrm{area}}, \mathrm{CI}$ ← kuantil (391-400);
  fallback (71) bila sampling gagal (407-418)
if ***`compute_area`*** $=$ *FALSE* **then**
  `df_area` ← `NULL` (351)
;
$\hat{\sigma}_v^2, \hat{\sigma}_u^2, \phi$, `goodness` ← (450-487);
kelas `c("fastsae_hb_twofold","fastsae_hb","fastsae")` (515);

12.7 Signature, argumen, objek keluaran, dan contoh

Listing 17: Signature `hb_twofold` (`R/hb_twofold.R:136-153`).

```
hb_twofold(formula, vardir = NULL, domain, subarea = NULL, data,
           family = c("gaussian","binomial","poisson"),
           spatial = c("none","bym2","besag"), W = NULL, weight = NULL,
           trials = NULL, exposure = NULL,
           engine = "inla",  # atau inla_engine(strategy, scale_model,
                             #   prior_prec (area), prior_prec_subarea,
                             #   prior_phi, draws = 200 bila NULL, seed, ...)
           compute_area = TRUE, print_result = TRUE, ...)  # ...: argumen lama, usang
```

Objek keluaran: `c("fastsae_hb_twofold", "fastsae_hb", "fastsae")` berisi `df_hb` (juga `df_subarea` sebagai alias, baris 495), `df_area` (atau `NULL`), `random_effect_var` bernama `sigma2_v` dan `sigma2_u`, `hyperpar`, `goodness` berupa `list`, `fit`, `engine`, `engine_spec`, `method`, `call`, `data`.

Listing 18: Contoh yang dapat dijalankan (dari `man/hb_twofold.Rd`).

```r
library(fastsae)
set.seed(42)
m ← 10
dat ← do.call(rbind, lapply(1:m, function(d) {
  nd ← sample(2:4, 1)
  data.frame(area = d, subarea = paste0(d, "-", seq_len(nd)),
             x1 = rnorm(nd), vardir = runif(nd, 0.2, 0.8))
}))
dat$y ← 1 + dat$x1 + rnorm(m)[dat$area] +
  rnorm(nrow(dat), sd = 0.5) + rnorm(nrow(dat), sd = sqrt(dat$vardir))

fit ← hb_twofold(y ~ x1, vardir = "vardir", domain = "area",
                 subarea = "subarea", data = dat)
summary(fit)
```

12.8 Referensi kunci

Blok roxygen memuat Torabi and Rao [2014] (model dua-lipat), Rao and Molina [2015], dan Riebler et al. [2016] (blok `@references` di `R/hb_twofold.R`); kerangka INLA mengacu ke Rue et al. [2009] dan Lindgren and Rue [2015].

13 Kerangka Bayes: INLA, GMRF, struktur spasial, dan prior

Bagian ini menjelaskan perangkat yang dipakai ketiga fungsi `hb_--` (`hb_area()`, `hb_unit()`, `hb_twofold()`). Semua rumus di bawah adalah rumus yang *dirujukkan* oleh kode dan oleh dokumentasi INLA yang terpasang; penyesuaian khusus paket ini ditandai *implementation note*.

13.1 Model latent Gaussian dan aproksimasi INLA

Keluarga yang dapat ditangani INLA [Rue et al., 2009] berbentuk

$$y_i \mid x_i, \theta \sim p(y_i \mid g(\mu_i) = \eta_i), \qquad \eta_i = x_i, \qquad x \mid \theta \sim N(0,\, Q(\theta)^{-1}), \qquad \pi(\theta), \tag{72}$$

yaitu respons bergantung pada field laten Gaussian x lewat link g, dan parameter hiper θ hanya muncul dalam presisi Q. INLA mengejar marginal posterior

$$\pi(x_i \mid y) \approx \int \tilde{\pi}_{\text{INLA}}(x_i \mid \theta)\, \tilde{\pi}(\theta \mid y)\, d\theta \tag{73}$$

lewat tiga aproksimasi bertumpuk (Laplace untuk $\pi(x_i \mid \theta)$ dan $\pi(\theta \mid y)$, plus simplifikasi strategi yang dipilih pengguna). Strategi dipilih lewat `inla_engine(strategy = )` dan sama untuk ketiga fungsi: `laplace` (default), `simplified.laplace`, `gaussian`. Rujukan buku untuk pemakaian praktis INLA dari R, termasuk cara membaca argumen `control.compute` yang dipakai bagian 10, adalah Gómez-Rubio [2020]. Alternatif MCMC untuk masalah ini dibahas Datta and Ghosh [1991] untuk prediksi Bayes dan Molina and Marhuenda [2015] untuk SAE tingkat daerah.

13.2 Gaussian Markov Random Field

Field laten x berdimensi n adalah GMRF [Rue and Held, 2005] bila

$$\pi(x) \propto \exp\!\Big(-\tfrac{1}{2}(x-\mu)' Q (x-\mu)\Big), \qquad Q \text{ sparse}, \tag{74}$$

di mana sparsitas Q mencerminkan *conditional independence*: $x_i \perp x_j \mid x_{-i,j}$ bila $Q_{ij} = 0$. Struktur Q diambil dari graf ketetanggaan, dan inversi dilakukan lewat Cholesky sparse (dalam INLA, format supernode). Untuk SAE, n = jumlah domain dan graf-nya adalah ketetanggaan spasial atau rantai waktu.

13.3 Struktur presisi: CAR, ICAR, BYM, dan varian lain

Definisi dasar. Misalkan W matriks ketetanggaan, D matriks derajat, dan $C = W$ atau $C = D^{-1}W$ menentukan skala yang dipakai. Tabel 28 merangkum pilihan yang tersedia di INLA beserta padanannya di `hb_area()`.

Tabel 28: Struktur presisi spasial dan pemakainya di fastsae. Sumber: penyusunan `f(...)` di `R/hb_area.R:570-649` dan `R/hb_unit.R:280-294`, `R/hb_twofold.R:250-266`.

Pilihan	Presisi Q	Catatan
`none`	$\mathbf{I}$ (i.i.d.)	tanpa struktur; hanya bila tidak ada interaksi lain
`besag` (ICAR)	$D - W$	intrinsik: tak terbatas pada konstanta; diperbaiki lewat konstraint dan `scale.model`
`bym` (BYM)	$Q_{\text{ICAR}} + \mathbf{I}$, dua presisi	BYM klasik [Besag et al., 1991]
`bym2` (BYM2)	$s\,Q_{\text{ICAR}} + (1-s)\mathbf{I}$	satu presisi + fraksi spasial s [Riebler et al., 2016]
`generic1`	$\mathbf{I} - (\beta/\lambda_{\max})C$, $\beta \in [0,1)$	presisi terikat graf
`slm`	melalui $(\mathbf{I} - \rho\mathbf{W})$	model lag spasial; ρ dipetakan ke $[0,1]$

Konversi bobot ke graf: `.convert_spatial_weights`. Sebelum masuk `INLA::inla`, matriks ketetanggaan harus berupa `inla.graph` (graf biner simetris tanpa bobot). Fungsi `R/inla_utils.R:48-109` menerima `listw` (spdep), `nb`, `matrix`, atau `inla.graph` langsung; bobot apa pun dibakar menjadi $w_{ij} > 0 \Rightarrow$ simpul (i,j), simetrisasi $A \leftarrow \max(A, A')$, lalu ditulis sebagai baris teks graf. Untuk `spatial = "slm"` bobot mentah dipakai apa adanya (bukan graf) karena `args.slm$W` memang matriks bobot — konsekuensinya berbeda pada kasus `W` berupa `inla.graph` (HB-20).

CAR proper vs ICAR. CAR proper memakai

$$Q = \tau(D - \rho\, C), \qquad \rho \in (1/\omega_{\min},\, 1/\omega_{\max}) \tag{75}$$

dengan ω eigen C — di luar rentang itu Q tidak positif definitif. ICAR adalah kasus batas tanpa ρ:

$$Q_{\text{ICAR}} = D - W, \qquad \pi(x) \propto \exp\!\left(-\tfrac{1}{2}\tau\, x'(D-W)x\right), \tag{76}$$

yang *tidak terintegrasi* pada konstanta (presisi punya satu nol) sehingga perlu konstraint jumlah-nol atau referensi; INLA menambahkannya otomatis untuk `bym2/bym/besag/rw1/rw2` (konfirmasi lewat `inla.models()`). Kedua kasus diperkenalkan Besag [1974] dan dibahas Banerjee et al. [2014].

BYM dan BYM2. BYM [Besag et al., 1991] menggabungkan dua komponen $u = u^s + u^u$ dengan dua presisi terpisah; parameter sulit diinterpretasikan dan korelasinya tinggi. BYM2 [Riebler et al., 2016] merapikannya menjadi satu presisi τ dan fraksi spasial

$$\varphi = \frac{\operatorname{Var}(u^s)}{\operatorname{Var}(u^s) + \operatorname{Var}(u^u)} \in [0,1], \qquad u \sim N\!\left(0,\ \tau^{-1}\left[\varphi\,\tilde{Q}_{\text{ICAR}} + (1-\varphi)\mathbf{I}\right]^{-1}\right), \tag{77}$$

dengan $\tilde{Q}_{\text{ICAR}}$ yang telah diskalakan [Sørbye and Rue, 2014] (`scale.model = TRUE` pada `hb_area()`). Karena presisi terikat, prior PC untuk φ menjadi interpetabel (bagian 13.4).

Implementation note. Kode `generic1` membangun $R = \text{diag}(\text{baris sum}) - W$, $C = \mathbf{I} - R$ (`hb_area.R:600-602`), sehingga C bernilai dua kali derajat pada diagonal — inilah $C = \mathbf{I} - R$ yang diminta INLA untuk `generic1`, dan ρ terkait melalui $\beta/\lambda_{\max}$. Untuk `slm`, rentang ρ dihitung dari akar eigen matriks row-standar lalu diklem ke $[-0.999, 0.999]$ (baris 612-649), dan prior default-nya `logitbeta(1,1)` (bukan PC).

13.4 Prior PC dan log-gamma

Prior *penalised complexity* [Simpson et al., 2017] mencabut kompleksitas dari prior dasar (keseragaman untuk varian, ketiadaan korelasi untuk ρ). Untuk standar deviasi:

$$\Pr(\sigma > u_0) = \alpha \quad \Longrightarrow \quad \pi(\sigma) = \lambda e^{-\lambda\sigma}, \qquad \lambda = \frac{-\log\alpha}{u_0}, \tag{78}$$

ditulis INLA sebagai `pc.prec` dengan `param = (u0, alpha)`. Untuk fraksi spasial BYM2 dan korelasi lain:

$$\Pr(|\varphi| > u_0) = \alpha \quad \Longrightarrow \quad \pi(\varphi) = \frac{\lambda}{2} e^{-\lambda|\varphi|}, \tag{79}$$

ditulis `pc` dengan `param = (u0, alpha)`. Prior log-gamma $\pi(\tau) = \text{loggamma}(a, b)$ lama dipakai untuk presisi dan tetap menjadi default INLA untuk presisi likelihood Gaussian (`loggamma(1, 5e-5)`).

Tabel 29 mencatat prior yang *sebenarnya* dipakai tiap fungsi. Perbedaan `hb_unit()` disengaja (HB-8): `pc.prec(1, 0.01)` mengandaikan simpangan baku efek area berorde 1 (proporsi, rate, skala log), sedangkan respons unit-level sering berskala mentah, sehingga dipakai prior log-gamma yang samar.

Tabel 29: Prior default di tiga fungsi Bayes (baca: "argumen bawaan yang didefinisikan dalam signature"). Sumber: signature fungsi dan pemakaian `hyper` di `R/hb_area.R:559-649`, `R/hb_unit.R:137-144`, `R/hb_twofold.R:155-171`.

Fungsi	Prior presisi komponen acak	Prior korelasi
`hb_area`	`pc.prec(1, 0.01)`	`pc(0.5, 0.5)`
`hb_unit`	`loggamma(0.01, 0.01)`	`pc(0.5, 0.5)`
`hb_twofold`	`pc.prec(1, 0.01)` per tingkat (area, sub-area)	`pc(0.5, 0.5)`

Implementation note. Prior tetap (intercept, koefisien regresi) tidak diset fastsae kecuali untuk *self-benchmarking* WFQ di `hb_area()` (prior difus, presisi 10^{-6}), sehingga umumnya berlaku default INLA: rata-rata 0, presisi 0.001 (intersep: presisi 0 = prior rata di $\mathbb{R}$). Nilai ini, beserta prior lain pada tabel di atas, adalah prior yang dipakai seluruh hasil Bayes di laporan ini (HB-21). Selain itu `prior_phi` pada `hb_area()` juga dipakai ulang untuk `beta generic1` dan `rho slm` bila nilainya bukan `"pc"` dan `prior_rho` tidak diberikan (HB-7; `prior_rho` didahulukan), dan untuk `bym2` + `prior_phi = "pc"` prior jatuh ke default INLA `pc(0.5,0.5)`.

13.5 Likelihood, link, dan parameterisasi `scale`

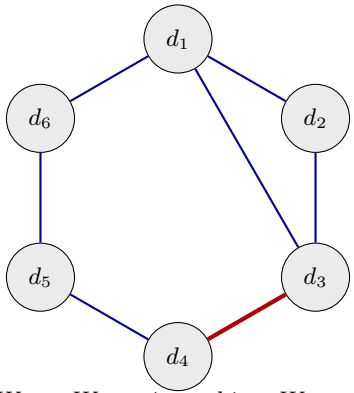
Tabel 30 memetakan family ke link dan ke apa yang benar-benar dikirim paket ke INLA. Ini penting karena *kolom* ***`link`*** pada argumen `hb_area` tidak pernah dipakai (HB-6): selalu `control.predictor$link = 1` (link default family).

Tabel 30: Family likelihood dan parameterisasi yang dikirim fastsae ke INLA. Sumber: `R/hb_area.R:785-841`, `R/hb_unit.R:303-311`, `R/hb_twofold.R:286-296`.

Family	Link yang dipakai	Apa yang benar-benar dikirim ke INLA
`gaussian` + `vardir`	identitas	`scale = 1/vardir`, presisi likelihood diklem tetap (`prec = list(initial = 0, fixed = TRUE)`)
`beta` + `vardir`	logit	`scale = y(1-y)/vardir - 1` diklem ≥ 1, hiperparameter `theta` tetap
`beta` + `trials`	logit	`scale = trials - 1` diklem ≥ 1
`gamma` + `vardir`	log	`scale = y^2/vardir`
`binomial` + `trials`	logit	`Ntrials = trials`
`poisson`, `nbinomial` + `exposure`	log	`E = exposure`
`hb_unit` (semua family)	link = 1	tanpa `scale`/`Ntrials`/`E`; penyesuaian hanya lewat `...`
`hb_twofold`	link = 1	per tingkat: `scale` (gaussian), `Ntrials` (binomial), `E` (poisson)

Parametrisasi `scale` pada baris beta, gamma, dan gaussian bukan pilihan gaya: INLA memakai `scale` sebagai presisi hipotetis (*scale*-inverse-dispersion) sehingga varians per observasi menjadi $1/$`scale`. Untuk beta, `phi_dir` diturunkan dari $\mathrm{Var}(y) = \phi^{-1} y(1-y)$ lalu diklem ke ≥ 1; untuk gamma, `scale` $= y^2/\psi$ meniru bentuk varians $\phi\mu^2$. Rujukan parametrisasi beta area-level ada di Janicki [2020].

Implementation note. Argumen `link` pada `hb_area()` diabaikan (HB-6): ketiga fungsi menyetel `control.predictor$link = 1` secara eksplisit, jadi link selalu link default family. Konsekuensinya, family non-Gaussian yang dipanggil dengan link bukan bawaan tidak akan menerima link yang diminta pengguna; kolom "Link yang dipakai" pada tabel di atas adalah link bawaan itulah.



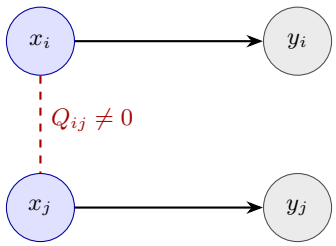
merah: $W_{34} = W_{43} = 1$; biru: $W_{13} = W_{31} = 1$

$D = \mathrm{diag}(3,2,3,2,2,2)$ (derajat); $Q_{\mathrm{ICAR}} = D - W$

$$Q_{\mathrm{proper}} = \tau\big(D - \rho\,(D^{-1}W)\big)$$

Gambar 7: Graf tetangga enam domain beserta matriks yang diturunkannya. Derajat dibaca dari graf: d_1 dan d_3 bertiga, sisanya bertua. Matriks bobot apa pun (matrix, `nb`, `listw`) dikonversi ke graf biner simetris oleh `.convert_spatial_weights` sebelum masuk `inla.graph`. Sumber: `R/inla_utils.R:48-109` dan pemakaian Q di `R/hb_area.R:570-649`.

Peta ke bab model. Tiga pemakaian praktis kerangka di atas terlihat pada bagian 10 (pilihan `f(...)`, prior, dan argumen `hyper`), bagian 11 (field laten per unit observasi), dan



biru = laten (x, GMRF); hitam = diamati (y)
putus-putus = ketergantungan struktural

Gambar 8: Struktur ketergantungan pada field laten GMRF: hanya pasangan dengan $Q_{ij} \neq 0$ yang bersambung (markov blanket), sementara y_i bergantung pada x_i lewat link. Struktur Q inilah yang diisi oleh CAR/ICAR/BYM/BYM2 pada Tabel 28. Sumber: (74) dan `R/hb_area.R:570-649`.

bagian 12 (dua tingkat ketetanggaan); perbedaan perilaku antar ketiganya terkumpul di `DISCREPANCIES.md` bagian F (ID `HB-1`–`HB-21`).

14 Lapisan utilitas: benchmarking, diagnostik, perbandingan, ekspor, peta, pra-pemodelan

Kelima kelompok fungsi di bawah ini bekerja pada objek apa pun berkelas `fastsae` (atau turunannya) dan membentuk "alur kerja" setelah sebuah model selesai dipasang pada data: *kalibrasi* → *diagnosis* → *perbandingan* → *pelaporan/pemetaan*. Semua rumus adalah rumus persis yang dikodekan; selisih terhadap rujukan ditandai *implementation note* dan dikumpulkan di `DISCREPANCIES.md` bagian G (ID `UT-1`–`UT-16`).

14.1 Benchmarking: `benchmark_sae`

Definisi. Agregat per grup dihitung dari bobot yang sudah disterilkan oleh `type`:

$$w_{ig}^{\text{calc}} = \begin{cases} w_{ig} / \sum_{k \in g} w_{ik} & \texttt{type = "mean"} \\ w_{ig} & \texttt{type = "total"} \end{cases} \qquad A_g = \sum_{i \in g} w_{ig}^{\text{calc}}\, y_i, \tag{80}$$

dan target T_g dipaksa tercapai lewat empat metode (`R/benchmark.R:452-531`):

$$\textbf{ratio:} \quad \lambda_g = \frac{T_g}{A_g}, \qquad \tilde{y}_i = y_i\, \lambda_g; \tag{81}$$

$$\textbf{difference:} \quad \delta_g = T_g - A_g, \qquad \tilde{y}_i = y_i + \frac{\delta_g}{\sum_i w_{ig}^{\text{calc}}}; \tag{82}$$

$$\textbf{optimal:} \quad \tilde{y}_i = y_i + \frac{w_{ig}^{\text{calc}}\, \tilde{m}_i}{\sum_{j \in g} (w_{jg}^{\text{calc}})^2\, \tilde{m}_j}\, \delta_g; \tag{83}$$

$$\textbf{logit:} \quad \text{cari } \alpha \text{ dari } \sum_i w_{ig}^{\text{calc}} \operatorname{logit}^{-1}(\operatorname{logit}(y_i) + \alpha) = T_g,$$
$$\tilde{y}_i = \operatorname{logit}^{-1}(\operatorname{logit}(y_i) + \hat{\alpha}), \tag{84}$$

dengan $\tilde{m}_i = \max(\text{mse}_i, 10^{-8})$; galat `ratio` dicatat bila $|A_g| <$ `double.eps`. Persamaan (84) dicari lewat `uniroot` pada bracket $[-20, 20]$, diperluas ke $[-50, 50]$ bila tandanya sama, dengan `tol = 1e-9`; persyaratan $0 < y < 1$ (dan $0 < T_g < 1$ untuk `type = "mean"`) diperiksa lebih dulu. Kepatuhan constraint $\sum_i w_{ig}^{\text{calc}} \tilde{y}_i = T_g$ terverifikasi aljabar untuk keempat metode.

Implementation note. Metode `optimal` kini persis solusi KKT dari $\min \sum_i (\tilde{y}_i - y_i)^2/\text{mse}_i$ s.t. $\sum_i w_i \tilde{y}_i = T_g$ (bobot kerugian $\phi_i = 1/\text{mse}_i$, Datta et al., 2011), yaitu $\tilde{y}_i - y_i \propto w_i \text{mse}_i$. Versi sebelum audit 12 Okt 2026 memakai $(\text{mse}_i/w_i)\lambda$ — constraint tercapai, tetapi alokasinya salah ketika bobot tidak seragam (UT-6, kini diperbaiki). Tahap 1 hierarkis memakai bentuk yang sama pada tingkat grup.

MSE estimasi terbenchmark. Setiap aturan menggeser estimasi sebesar $\text{adj}_i = k_i(T - \sum_j w_j \hat{\theta}_j)$ dengan $\sum_i w_i k_i = 1$ (*difference*: $k_i = 1/\sum w$; *ratio*: $k_i = \hat{\theta}_i/\sum w\hat{\theta}$; *optimal*: $k_i = w_i \text{mse}_i/\sum w^2 \text{mse}$; *logit*: $k_i = \hat{\theta}_i(1-\hat{\theta}_i)/\sum w\hat{\theta}(1-\hat{\theta})$). Dengan galat e, $\mathbb{E}[ee'] = M$, MSE orde pertama estimasi terbenchmark adalah

$$\text{MSE}_{bm,i} \approx \text{mse}_i - 2k_i(Mw)_i + \text{adj}_i^2 \tag{85}$$

(`R/benchmark.R:532-586`, kolom `mse_benchmarked`). M adalah matriks MSE *penuh* antar area — galat prediksi berkorelasi karena $\hat{\beta}$ bersama dan efek acak spasial/temporal — yang tidak pernah dibentuk: `.mse_operator` (baris 790-893) mengembalikan $v \mapsto Mv$ dengan $M = SCS$, C matriks korelasi $g_1 + g_2$ dan $S = \text{diag}(\sqrt{\text{mse}})$, untuk FH (skala identitas), SFH, STFH, dan HB area-level (dari sampel posterior INLA atau draw MCMC). Model lain memakai fallback diagonal $(Mw)_i = w_i \text{mse}_i$. MSE agregat grup pada tahap 1 juga memakai $w'Mw$, bukan $\sum w^2 \text{mse}$.

Dua tahap hierarkis. Aktif bila `national_target` diberikan dan ada > 1 grup (`R/benchmark.R:316`): STAGE 1 menyetel target grup T_g agar agregatnya menyamai target nasional (empat metode yang sama, `W_share`=porsi bobot grup), STAGE 2 menyetel estimasi level kecil ke T_g; keduanya dicatat `stage_1_summary` dan diverifikasi ulang.

Algorithm 10: `benchmark_sae` — alur sebenarnya (`R/benchmark.R:120-633`)

Masukan *objek model (atau vektor numerik),* ***`target`, `weight`, `method`*** $\in$*{ratio, difference, optimal, logit},* ***`group`, `type`, `national_target`, `outer_target`, `mse`***
Keluaran *data frame kelas* ***`fastsae_benchmark`*** *+ atribut verifikasi* estimasi $\leftarrow$ `df_hb %||% df_ebp %||% df_eblup`; $y \leftarrow$ `hb/ebp/eblup/est`; mse $\leftarrow$ `mse` atau sd^2 (146-153);
$M \leftarrow$ `.mse_operator(object)` (atau `NULL`: fallback diagonal);
bobot $\leftarrow$ nama kolom / vektor / seragam 1; grup $\leftarrow$ nama kolom / vektor / `"All"` (157-200);
$A_g, T_g \leftarrow$ (80) + parsing target (316-400);
if ***`national_target`*** $\neq$ *NULL* ***dan*** > 1 *grup* **then**
  STAGE 1: setel T_g agar $\sum_g W_{\text{share},g} T_g = T_{\text{nas}}$ (402-450);
STAGE 2: $\tilde{y} \leftarrow$ (81)–(84) per grup (452-531);
$\text{MSE}_{bm} \leftarrow$ (85) (532-586);
susun `domain, weight, original, benchmarked, adjustment, rel_adjustment_pct, target` (+ `mse`, `mse_benchmarked`) (570-586);
verifikasi $|\sum w^{\text{calc}} \tilde{y} - T_g|$; status `CONSISTENT` bila $< 10^{-5}$ (588-631);

Keluaran. `data.frame` kelas `c("fastsae_benchmark","data.frame")` dengan atribut `method`, `type`, `hierarchical`, `stage1_summary`, `verification`, `national_verification`; metode S3: `print` (status konsistensi per grup), `summary`, `autoplot` (sebaran `original` vs `benchmarked` dengan garis 1:1), dan `plot` yang bila diberi `sf_geom` meneruskan ke `map_sae()`.

Rujukan yang memverifikasi pendekatan restrisi/benchmarking di tingkat estimasi: Rao and Molina [2015] (bab 10), You and Rao [2002] dan Wang et al. [2008].

14.2 Diagnostik: `diagnose`

`diagnose()` adalah fungsi biasa (bukan generic S3) dan mensyaratkan objek `fastsae` (`R/diagnose.R:73-80`). Kuantitas dasar (`R/diagnose.R:104-150`):

$$\mathrm{RSE}_d^{\mathrm{direct}} = 100\frac{\sqrt{\psi_d}}{|y_d|}, \qquad \mathrm{eff}_d = \frac{\psi_d}{\mathrm{mse}_d}, \qquad r_d = \frac{y_d - \hat{\theta}_d}{\sqrt{\psi_d - \mathrm{mse}_d}}, \qquad \widehat{\Pr}(\mathrm{RSE} < T) = \frac{\#\{\mathrm{RSE} < T\}}{n}, \tag{86}$$

dengan ambang default $T = 25\,\%$. Residual terstandar memakai $\mathrm{Var}(y_d - \hat{\theta}_d) \approx \psi_d - \mathrm{mse}_d$ (galat prediksi berkorelasi positif dengan e_d; untuk EBLUP $\psi_d(1-\gamma_d)$), sehingga $r_d \approx N(0,1)$ di bawah model yang benar (sd 1,01 pada simulasi; pembagi lama $\sqrt{\psi_d}$ memberi sd 0,76 dan menyembunyikan pencilan). Area tanpa sampel dihitung terpisahnya (`N_sampled`/`N_unsampled`).

Diagnostik Brown et al. (2001). Untuk minimal lima pasang $(y, \hat{\theta})$ ter-sampel (`R/diagnose.R:153-201`):

$$W = \sum_d \frac{(y_d - \hat{\theta}_d)^2}{\psi_d + \mathrm{mse}_d}, \quad p = \Pr(\chi^2_{n_w} \geq W), \tag{87}$$

$$y_d = \alpha + \beta\hat{\theta}_d + e_d \quad \text{(OLS, deskriptif)}. \tag{88}$$

`is_good_fit` $= p \geq \alpha$ (ekor atas saja, seperti `emdi::compare.fh`): hanya W yang besar menandakan estimasi model berbeda dari estimasi langsung melebihi ketidakpastiannya. W konservatif karena mengabaikan korelasi positif y_d dan $\hat{\theta}_d$; di bawah model FH yang benar $W/n_w \approx 0{,}4$ pada simulasi. Regresi (88) hanya dilaporkan ($\hat{\alpha}, \hat{\beta}$, galat baku, R^2), tanpa uji: untuk estimator penyusutan kemiringannya melebihi 1 walau model benar (sekitar $1/\gamma_d$ untuk variasi yang tidak dijelaskan kovariat).

Implementation note. Sebelum 14 Okt 2026 kode menguji $\alpha = 0, \beta = 1$ dengan uji F OLS dan memutuskan W dengan interval dua sisi. Simulasi 300 replikasi model FH yang benar ($D = 100$) memberi tingkat penolakan 93 % dan 100 % (nominal 5 %); keduanya kini diperbaiki (UT-1–UT-3). Rujukan Brown et al. adalah laporan internal ONS yang tidak dapat diverifikasi (lihat `TODO_REFERENCES.md`); acuan implementasinya `emdi::compare.fh`.

Moran's I residual (`R/diagnose.R:203-256`) dihitung pada residual mentah area ter-sampel (bukan terstandarisasi), W dipotong ke baris ter-sampel, minimal lima area:

$$I = \frac{n}{S_0} \cdot \frac{\sum_{ij} w_{ij} z_i z_j}{\sum_i z_i^2}, \tag{89}$$

$$\mathrm{Var}(I) = \frac{n\big[(n^2-3n+3)S_1 - nS_2 + 3S_0^2\big] - \kappa\big[(n^2-n)S_1 - 2nS_2 + 6S_0^2\big]}{(n-1)(n-2)(n-3)S_0^2} - (\mathbb{E}I)^2, \tag{90}$$

dengan $S_0 = \sum_{ij} w_{ij}$, $z = r - \bar{r}$, $S_1 = \frac{1}{2}\sum(w_{ij} + w_{ji})^2$, $S_2 = \sum_i(\sum_j w_{ij} + \sum_j w_{ji})^2$, $\kappa = n\sum z^4/(\sum z^2)^2$, $\mathbb{E}I = -1/(n-1)$, dan $\mathrm{Var} \geq 10^{-8}$; z-statistik dua sisi $2\Phi(-|z|)$.

Metrik simulasi dan diagnostik Bayes. Bila `truth` diberikan: rerata $\mathrm{rb}_d = 100(\hat{\theta}_d - \theta_d)/\theta_d$, rerata $|\mathrm{rb}_d|$, RRMSE lintas domain $100\sqrt{\overline{(\hat{\theta}-\theta)^2}}/\overline{|\theta|}$ (UT-4 diperbaiki), dan `coverage` dari

kolom CI apa adanya (UT-5). Untuk model Bayes, DIC/WAIC/marginal likelihood diambil dari `goodness` (vektor bernama pada `hb_area()`, list pada `hb_unit()`/`hb_twofold()`), CPO/PIT dari `fit$cpo` hanya bila barisnya per domain, jumlah `failure`, lalu uji KS `ks.test(pit, "punif")`.

Status akhir berurutan: `PASS` bila uji W lolos dan proporsi RSE andal $\geq 80\,\%$, `CAUTION` bila uji W menolak, lalu `WARNING` bila proporsi RSE$\geq$ ambang terlalu besar. Bila suatu tes tidak tersedia dianggap lolos.

Algorithm 11: `diagnose` — alur sebenarnya (`R/diagnose.R:73-410`)

Masukan *objek* ***fastsae****,* ***W****,* ***truth****,* ***rse_threshold*** $= 25$*,* ***alpha_level*** $= 0.05$
Keluaran *list kelas* ***fastsae_diagnose*** *+ string* ***status*** ekstrak `df`, y, $\hat{\theta}$, ψ, mse (82-102);
(86): RSE direct/SAE, efisiensi, residual, proporsi andal (104-150);
if $n_{pasang} \geq 5$ **then**
  regresi deskriptif (88); GOF (87) ekor atas (153-201)
;
if ***W*** *tersedia* ***dan*** $n \geq 5$ **then**
  Moran's I (89)–(90) pada residual terpotong (203-256)
;
if ***truth*** *diberikan* **then**
  rb, rrmse, coverage (258-285)
;
if *objek Bayes* **then**
  DIC/WAIC/CPO/PIT + uji KS (287-358)
;
`status` ← aturan berurutan pada teks (360-371);
susun `df_diag` (`domain, direct_y, sae_pred, vardir, mse, direct_rse, sae_rse, eff_ratio, residuals, std_residuals, is_sampled`[, `cpo, pit`]) (373-390);

Keluaran dicetak `print.fastsae_diagnose` dalam lima bagian (efisiensi, kalibrasi Brown, Moran's I, simulasi, Bayes) plus penilaian akhir; grafik lewat `autoplot` dengan tipe `all`, `calibration`, `rse`, `residuals`, `qq`, `pit`, `cpo`. Tidak ada `summary` maupun `plot` untuk kelas ini (UT-13).

14.3 Perbandingan dua model: `compare_sae`

`compare_sae(model1, model2, names, thresholds = c(25,50))` hanya memiliki method `.default` (UT-14); kunci gabung adalah `domain`, atau `c(domain, subarea)`, atau `c(domain, time)` (inner join; error bila tumpang tindih nol). Metrik yang dihitung (`R/compare_sae.R:112-176`):

$$r = \mathrm{cor}(\hat{\theta}_1, \hat{\theta}_2), \quad \rho = \text{Spearman}, \quad \mathrm{MAE} = \overline{|d|}, \quad \mathrm{RMSD} = \sqrt{\overline{d^2}}, \quad d = \hat{\theta}_2 - \hat{\theta}_1, \tag{91}$$

plus rasio $\mathrm{mse}_1/\mathrm{mse}_2$, jumlah domain dengan $\mathrm{mse}_2 < \mathrm{mse}_1$, dan $\Delta\overline{\mathrm{RSE}}$. *Tidak ada* argumen `truth`, sehingga RMSD/MAE di sini adalah selisih antarmodel, bukan galat terhadap kebenaran. `thresholds` hanya dipakai sebagai garis horizontal pada plot `rse` (UT-14). Baris diberi label

unik (domain, domain/sub-area, atau domain:waktu) sehingga data panel dan sub-area tidak lagi menumpuk atau gagal. Lima tipe `autoplot`: `scatter` (garis 1:1), `comparison` (default, dodged), `difference` (lollipop), `mse` (bar dodged), `rse` (garis + ambang).

14.4 Flag reliabilitas dan ekspor: `export_sae`

`add_reliability_flags` memotong RSE (helper `.reliability_tier`, dipakai juga oleh `export_sae()` dan `map_sae()`) menjadi tiga tingkat dengan kriteria BPS sebagai default: *Reliable* (RSE $\leq 25\%$), *Use with caution* ($25\% <$ RSE $\leq 50\%$), *Unreliable* (RSE $> 50\%$); bila kolom RSE tidak ada, ia menghitung $100\sqrt{\text{mse}}/\left|\hat{\theta}\right|$ dan mengembalikan tabel estimasi (data frame).

Implementation note. Sebelum 14 Okt 2026 ambang default 20/30 % dengan klaim "standar BPS", padahal publikasi BPS memakai 25 dan 50 %; kini disesuaikan (UT-9 juga dibereskan: dokumentasi menyatakan yang dikembalikan data frame).

`export_sae(object, file, benchmark, thresholds, overwrite)` hanya menerima ekstensi `xlsx`/`csv`; sheet yang ditulis: `Estimates`, `Model_Summary`, `Benchmarked` dan `Benchmarked_Groups` (bila hierarkis) — hingga empat, sesuai dokumentasi (UT-10 diperbaiki). Penulisan xlsx memakai writexl, fallback openxlsx, fallback terakhir CSV berisi `Estimates` saja; sheet `Benchmarked` tidak pernah ikut pada mode csv. Sheet `Estimates` memuat SE $= \sqrt{\text{mse}}$, $\text{CI}_{95\%} = \hat{\theta} \pm 1.96\,\text{SE}$ (bila kolom CI tidak ada), serta flag reliabilitas; sheet `Model_Summary` berupa pasangan `Parameter/Value` (paket, tipe model, method, konvergensi, komponen ragam, goodness of fit, koefisien + SE + p).

14.5 Peta choropleth: `map_sae`

Empat method: `.fastsae`, `.fastsae_benchmark`, `.list`, `.default`; semuanya mensyaratkan sf dan menghasilkan `ggplot`. Lima tipe `type`:

- `estimate` — `geom_sf` + viridis;
- `rse` — `option = "magma"`, arah terbalik;
- `reliability` — tiga warna manual `#198754` / `#ffc107` / `#dc3545` sesuai tingkat §14.4;
- `comparison` — facet Direct Survey vs SAE (otomatis jatuh ke `estimate` bila kolom direct seluruhnya `NA`);
- `difference` — $\hat{\theta} - y$ dengan `gradient2` `#2b83ba`/`#ffffbf`/`#d7191c`.

Pencocokan kunci otomatis memakai daftar kandidat `domain, area, id, code, kd_kab, kode, kabupaten, provinsi, district, region` dan mengadopsi pasangan dengan tumpang tindih terbesar *bila mencapai* 40% jumlah domain; gagal → error.

`thresholds` (default `c(25, 50)`) kini benar-benar menentukan tingkat pada `type = "reliability"` (UT-12), geometri diambil dengan `sf::st_geometry` sehingga nama kolom geometri bebas (UT-16), label model mengikuti `object$model`, dan tabel dengan domain berulang (panel atau sub-area) ditolak dengan pesan yang jelas karena satu poligon hanya bisa menampilkan satu nilai.

14.6 Pra-pemodelan: `create_weights`, `spatial_test`, `gvf_smooth`

Tiga fungsi ini dipakai *sebelum* model dipasang.

`create_weights` (`R/create_weights.R`) membangun matriks bobot spasial W ($m \times m$, kelas `fastsae_weights`) dari poligon/titik `sf`, matriks koordinat, atau objek `nb`/`listw`. Metode: kontiguitas `queen`/`rook`, `knn` (k tetangga terdekat antar-sentroid), `distance` (pita jarak; `d_max` otomatis sebesar jarak terkecil yang membuat graf terhubung), dan `idw` ($w_{ij} = d_{ij}^{-\alpha}$). Gaya normalisasi: `W` (row-standar, untuk SAR/SFH), `B` (biner, untuk BYM2/Besag di INLA), `C` (standar global), `U` ($1/m$), dan `minmax` (dibagi nilai eigen terbesar). Domain tanpa tetangga (pulau, mis. kabupaten kepulauan) dihubungkan ke `k_fallback` tetangga terdekatnya bila `island_fallback = TRUE` (default), sehingga graf terhubung dan tidak menimbulkan singularitas. Fungsi ini juga dipanggil otomatis oleh `eblup_sfh()` (queen, `"W"`), `hb_area()` (queen, `"B"`), dan `spatial_test()` bila `data` berkelas `sf` dan `W` tidak diberikan. Dalam implementasinya jarak dihitung per blok dan matriks diisi dalam satu lintasan, sehingga tidak ada matriks jarak $m \times m$ penuh untuk metode berbasis jarak.

`spatial_test` (`R/spatial_test.R`) menilai apakah model spasial diperlukan, pada domain tersampel saja:

1. Moran's I global pada respons y dan pada residual OLS $y \sim \mathbf{X}$, dengan $\mathbb{E}[I] = -1/(n-1)$ dan varians di bawah asumsi randomisasi (memakai kurtosis sampel);
2. uji Lagrange Multiplier Anselin [1988] pada residual OLS e: $\mathrm{LM}_{\mathrm{err}} = \left(e'We/\hat{s}^2\right)^2/T$ dan $\mathrm{LM}_{\mathrm{lag}} = \left(e'Wy/\hat{s}^2\right)^2/D_{\mathrm{lag}}$, $T = \mathrm{tr}(W'W + W^2)$, $D_{\mathrm{lag}} = T + \|MW\mathbf{X}\hat{\beta}\|^2/\hat{s}^2$ (M proyektor residual OLS lewat QR), masing-masing χ^2_1;
3. bila `vardir` diberikan (termasuk objek `gvf_smooth()`): `eblup_fh()` dan `eblup_sfh()` (REML) dipasang dan dibandingkan lewat log-likelihood, AIC, BIC, dan statistik $2(\ell_{\mathrm{SFH}} - \ell_{\mathrm{FH}})$ terhadap χ^2_1.

Kedua model pada butir 3 dipasang dengan ML, sehingga $2(\ell_{\mathrm{SFH}} - \ell_{\mathrm{FH}})$ adalah uji rasio likelihood untuk $\rho = 0$ (titik interior, χ^2_1). Rekomendasi (`spatial_recommended`) bersandar pada *satu* uji pada taraf `alpha` (`decision_test`): uji rasio likelihood bila `vardir` ada — yang memperhitungkan varians sampling — dan uji LM-error bila tidak; statistik lain dilaporkan sebagai informasi. Moran's I residual, LM-error, dan LM-lag identik dengan `spdep` (`lm.morantest`, `lm.RStests`).

Implementation note. Sebelum 14 Okt 2026 kedua model dipasang dengan REML (log-likelihood ML dievaluasi di estimasi REML) dan rekomendasi diberikan bila salah satu dari tiga uji signifikan; pada data non-spasial itu merekomendasikan model spasial pada 9 % replikasi (nominal 5 %), kini 5,3 %. Kuasa uji rasio likelihood terhadap SAR $\rho = 0{,}7$ ($D = 100$): 93 %.

`gvf_smooth` (`R/gvf.R`) menghaluskan varians sampling langsung D_i yang tidak stabil dengan *generalized variance function* [Wolter, 2007]:

- `"log_linear"` (default): $\log D_i = \alpha_0 + \alpha_1 \log y_i + \alpha_2 \log n_i + \varepsilon_i$ lewat `lm`, lalu $\tilde{D}_i = \exp(\hat{\mu}_i + \hat{s}^2/2)$ (koreksi bias log-normal, dapat dimatikan);
- `"gamma_glm"`: GLM Gamma dengan link log pada D_i, tanpa bias transformasi balik;
- `"cv_power"`: model pangkat pada $\log \mathrm{CV}_i^2$ (membutuhkan y), lalu $\tilde{D}_i = \widetilde{\mathrm{CV}}_i^2 y_i^2$.

Hasilnya (kelas `gvf_smooth`) dapat diberikan langsung sebagai `vardir` ke `eblup_fh()` dan `spatial_test()`, yang memakai `smooth_vardir`-nya.

14.7 Metode S3 dan grafik

Tabel 31 merangkum metode yang terdaftar di `NAMESPACE:23-52`.

Tabel 31: Metode S3 utama beserta file sumbernya.

Generic	Objek	Perilaku ringkas
`print`	`fastsae`	Call, konvergensi + `n_iter`, tipe model, komponen ragam, `printCoefmat(estcoef)`, `head(df,6)`
`summary`	`fastsae`	list 20 komponen; dicetak `print.summary.fastsae`
`coef`, `fitted`, `residuals`	`fastsae`	`estcoef$beta`; `hb/ebp/eblup`; $y - \hat{\theta}$
`plot`, `autoplot`	`fastsae`	7 tipe: `comparison`, `ribbon`, `mse`, `estimates`, `scatter`, `rse`, `map`
`autoplot`	`list fastsae`	`comparison`, `mse`, `scatter` (list tanpa nama diberi nama "Model k" — UT-15)
`autoplot`	`fastsae_diagnose`	`all`, `calibration`, `rse`, `residuals`, `qq`, `pit`, `cpo`
`print`, `summary`, `autoplot`, `plot`	`fastsae_benchmark`	lihat §14.1
`print`, `summary`, `autoplot`, `plot`	`fastsae_comparison`	lihat §14.3

Pada `autoplot` model tunggal, bila kolom CI belum ada, pita dibuat $\hat{\theta} \pm 1.96\sqrt{\text{mse}}$; tipe `rse` membandingkan RSE SAE vs RSE direct dengan dua ambang `c(20,30)`.

14.8 Data simulasi

Tiga pembangkit tersedia, semuanya menghasilkan matriks ketetanggaan lewat `sim_spatial_weights()` (tipe `knn/grid/ring`, gaya `B` biner atau `W` row-standar; kNN disimetrisasi $A \leftarrow \max(A, A')$).

DGP area-level (`sim_area_data`, `R/sim_area_data.R:63-166`):

$$v = (\mathbf{I} - \rho W_{\text{std}})^{-1}\varepsilon, \quad \tilde{v} = \text{standarisasi}(v), \quad \tilde{u} = \text{standarisasi}(N(0,1)), \tag{92}$$

$$u = \sigma_u\big(\sqrt{\varphi}\,\tilde{v} + \sqrt{1-\varphi}\,\tilde{u}\big), \qquad \eta = x'\beta + u, \tag{93}$$

$$y \mid \eta \sim \begin{cases} \eta + \mathcal{N}(0, \psi) & \psi \sim U(0.04,\ 0.16) \\ \text{Poisson}\big(\text{exposure} \cdot e^{\text{clip}(\eta/2 - 0.5,\ -4,\ 4)}\big) & \text{offset} \\ \text{Binomial}\big(\text{trials},\ \text{logit}^{-1}(\eta - 1)\big) & \text{peluang} \\ \text{NB}\big(\theta = 3,\ \mu = 15\, e^{\text{clip}(\eta/2,\ -3,\ 4)}\big) & \text{gamma-Poisson} \\ \text{Beta}\big(\mu\kappa,\ (1-\mu)\kappa\big) & \kappa = 25 \\ \text{Gamma}\big(k = 5,\ \mu/k\big) & k = 5 \end{cases} \tag{94}$$

dengan $x_1 \sim N(2,1)$, $x_2 \sim U(0,5)$, parameter dipotong $\rho \in [-0.95, 0.95]$, $\varphi \in [0,1]$, dan `n_unsampled` domain diset `NA`; `exposure` $\sim$ round $U(80,400)$ dan `trials` $\sim$ round $U(50,250)$. Rata-rata pada Beta adalah $\mu = \text{clip}\big(\text{logit}^{-1}(\eta - 1),\ 0.02,\ 0.98\big)$ (dan y dipotong ke $[0.001, 0.999]$), pada Gamma $\mu = \exp\big(\text{clip}(\eta/2,\ -3,\ 3)\big)$, dan pada NB $\mu = 15\exp\big(\text{clip}(\eta/2,\ -3,\ 4)\big)$. (93) adalah campuran gaya BYM2 yang sama dengan (77).

DGP panel (`sim_series_data`) menambah efek spasial $u_1 = \sigma_s\,\text{standar}\big((\mathbf{I} - \rho_s W_{\text{std}})^{-1}\varepsilon\big)$, proses AR(1) per domain $u_2[d,t] = \rho_t u_2[d,t-1] + \mathcal{N}(0, \sigma_t\sqrt{1-\rho_t^2})$ (varian stasioner σ_t^2), dan tren

deterministik (`linear`/`random_walk`/`ar1`/`none`); pengurutan baris `domain-major` memenuhi syarat `eblup_stfh()`.

Implementation note. Dataset terlampir berbeda dari keluaran pembangkit: `sim_area` memakai kolom `area` dengan 14 kolom tanpa `truth_gaussian`, sedangkan `sim_area_data()` menghasilkan 15 kolom dengan `domain` dan `truth_gaussian` (UT-11); hal yang sama berlaku untuk `sim_panel`.

14.9 Rujukan lapisan utilitas

Blok roxygen `R/benchmark.R:79-93` memuat lima rujukan dan `R/diagnose.R` memuat tiga; yang berhasil diverifikasi dipakai di sini: Rao and Molina [2015] (bab benchmarking), You and Rao [2002], Datta et al. [2010] (benchmarking Bayes), dan Berg and Fuller [2014]. Rujukan yang tidak dapat diverifikasi (Steorts et al. 2014; Brown et al. 2001; Cliff & Ord 1981) tidak dikutip sebagai referensi bibliografis dan didaftarkan di `TODO_REFERENCES.md` bagian 5, termasuk dua koreksi judul/tahun terhadap teks kode.

A Derivasi identitas Woodbury dan penerapannya

Identitas. Untuk matriks yang saling kompatibel,

$$(\mathbf{V}_A + \mathbf{Z}\mathbf{G}\mathbf{Z}')^{-1} = \mathbf{V}_A^{-1} - \mathbf{V}_A^{-1}\mathbf{Z}(\mathbf{G}^{-1} + \mathbf{Z}'\mathbf{V}_A^{-1}\mathbf{Z})^{-1}\mathbf{Z}'\mathbf{V}_A^{-1}, \tag{95}$$

dan log-determinannya

$$\log|\mathbf{V}_A + \mathbf{Z}\mathbf{G}\mathbf{Z}'| = \log|\mathbf{V}_A| + \log|\mathbf{G}| + \log\left|\mathbf{G}^{-1} + \mathbf{Z}'\mathbf{V}_A^{-1}\mathbf{Z}\right|. \tag{96}$$

Bukti singkat: tulis $\mathbf{V}_A + \mathbf{Z}\mathbf{G}\mathbf{Z}' = [\mathbf{I} \;\; \mathbf{Z}] \begin{pmatrix} \mathbf{V}_A & 0 \\ 0 & \mathbf{G} \end{pmatrix} \begin{pmatrix} \mathbf{I} \\ \mathbf{Z}' \end{pmatrix}$ dan pakai rumus invers blok ((95) adalah kasus $C = \mathbf{G}$, $U = \mathbf{Z}$); untuk (96) pakai determinan matriks blok pada representasi yang sama. Kasus khusus $\mathbf{G} = \gamma\mathbf{I}$ berkurang ke Sherman–Morrison,

$$(\mathbf{V}_A + \gamma zz')^{-1} = \mathbf{V}_A^{-1} - \frac{\gamma\,\mathbf{V}_A^{-1}zz'\mathbf{V}_A^{-1}}{1 + \gamma z'\mathbf{V}_A^{-1}z}. \tag{97}$$

Penerapan pada `fastsae`. Tabel 32 mencatat persis di mana identitas ini dipakai.

- **STFH.** Dengan $\mathbf{V}_A$ blok diagonal per domain, $\mathbf{V}_A^{-1}Z_1$ dan $Z_1'\mathbf{V}_A^{-1}Z_1$ dihitung per blok, lalu (95) dengan $\mathbf{G} = \sigma_1^2\Omega_1$ memberi $\mathbf{V}^{-1}$ dengan biaya $O(D\,T^2) + O(D^3)$ alih-alih $O((DT)^3)$; (96) menurunkan $\log|\mathbf{V}|$ dari $O((DT)^3)$ menjadi $O(D\,T^3) + O(D^3)$. Catatan: $\mathbf{C}$ yang muncul di kode adalah $\mathbf{C} = \sigma_1^{-2}\Omega_1^{-1} + \mathrm{diag}(Z_1'\mathbf{V}_A^{-1}Z_1)$, ekuivalen dengan (95) setelah kedua sisi dikalikan σ_1^2.
- **Two-fold.** Karena $\sigma_v^2\mathbf{1}\mathbf{1}'$ berderajat 1, (97) dipakai per area dengan $d = 1/(\sigma_u^2 + \psi_d)$, $s = \sum_i d_i$ dan $c = \sigma_v^2/(1 + \sigma_v^2 s)$ (kode `dd`, `s`, `c`), sehingga $V_d^{-1}a = d \odot a - c\,d(d'a)$ — biaya $O(n_d)$ per blok.
- **SFH.** (95) dengan $\mathbf{G} = \sigma^2\Sigma$ dan $\mathbf{Z} = E$ memberi $\mathbf{V}^{-1}$ lewat *solve* dengan M; log-determinan dari lemma determinan $\log|\mathbf{V}| = \log|\Psi| + \log|M| - \log|A|$.
- **Memori.** STFH tidak menyimpan $\mathbf{V}^{-1}$ maupun P secara penuh (hanya blok $T \times T$, $D \times D$, dan faktor $M \times (D + p)$), dan SFH hanya faktor sparse serta blok $D \times m$; versi sebelum Okt 2026 menyimpan matriks $M \times M$ penuh sehingga jejak memorinya $O(M^2)$.

Tabel 32: Pemakaian identitas Woodbury di dalam paket. Sumber: baris kode yang dikutip pada kolom terakhir.

Fungsi	Bentuk pemakaian	Lokasi
`eblup_stfh`	$\mathbf{V}_A$ = diag blok $T \times T$ per domain, $\mathbf{G} = \sigma_1^2 \Omega_1$, $\mathbf{Z} = Z_1$; $\mathbf{C}$ berdimensi $D \times D$; P dalam bentuk peringkat-rendah $A^{-1} - UW_mU'$	`eblup_stfh_core.cpp:71-410`
`eblup_sfh`	$\mathbf{V} = \Psi + \sigma^2 E\Sigma E'$, $\Sigma = A^{-1}$: $\mathbf{V}^{-1} = \Psi^{-1} - \sigma^2 \Psi^{-1} E M^{-1} E' \Psi^{-1}$, $M = A + \sigma^2 E' \Psi^{-1} E$, dengan A dan M difaktorkan sparse	`sar_engine.h`, `eblup_sfh_core.cpp:74-160`
`eblup_twofold`	per blok area: $V_d = \sigma_v^2 \mathbf{1}\mathbf{1}' + \mathrm{diag}(\sigma_u^2 + \psi_d)$, kasus (97) dengan $z = \mathbf{1}$	`eblup_twofold.cpp:102, 375-377`
`hb_*`	— (diatasi INLA lewat Cholesky sparse)	—
`eblup_fh`	tidak perlu: $\mathbf{V}$ diagonal	`eblup_fh.cpp`

B Derivasi skor dan informasi Fisher (ML dan REML)

Misalkan $y \sim N(\mathbf{Z}\beta, \mathbf{V}(\theta))$ dengan $\theta \in \{\sigma^2, \rho, \ldots\}$ dan $D_a = \partial \mathbf{V}/\partial \theta_a$. Dari $\ell = -\frac{1}{2}\big[n \log 2\pi + \log|\mathbf{V}| + (\beta - \beta)'\mathbf{X}'\mathbf{V}^{-1}\mathbf{X}(\ldots) + r'\mathbf{V}^{-1}r\big]$ dengan $r = y - \mathbf{X}\hat{\beta}$ dan $\hat{\beta} = (\mathbf{X}'\mathbf{V}^{-1}\mathbf{X})^{-1}\mathbf{X}'\mathbf{V}^{-1}y$, turunan parsial memakai $\partial \mathbf{V}^{-1}/\partial\theta_a = -\mathbf{V}^{-1}D_a\mathbf{V}^{-1}$ memberi

$$s_a = \frac{\partial \ell}{\partial \theta_a} = -\tfrac{1}{2}\,\mathrm{tr}(\mathbf{V}^{-1}D_a) + \tfrac{1}{2} r'\mathbf{V}^{-1}D_a\mathbf{V}^{-1}r \qquad \text{(ML)}. \tag{98}$$

Estimasi terbatas (REML) memakai residual berbentuk Pr dengan $P = \mathbf{V}^{-1} - \mathbf{V}^{-1}\mathbf{X}Q\mathbf{X}'\mathbf{V}^{-1}$ (yang membatalkan komponen desain), sehingga

$$s_a^{\mathrm{REML}} = -\tfrac{1}{2}\,\mathrm{tr}(PD_a) + \tfrac{1}{2} y'PD_aPy, \qquad \mathcal{I}_{ab}^{\mathrm{REML}} = \tfrac{1}{2}\,\mathrm{tr}(PD_aPD_b), \tag{99}$$

dan $\mathcal{I}_{ab}^{\mathrm{ML}} = \frac{1}{2}\,\mathrm{tr}(\mathbf{V}^{-1}D_a\mathbf{V}^{-1}D_b)$. Yang dikodekan: untuk `eblup_sfh` dan `eblup_stfh`, (99) dipakai apa adanya bila `method = "REML"`; untuk `method = "ML"` pada SFH, baris trace dan informasinya memakai $\mathbf{V}^{-1}$, sedangkan kuadratiknya $y'PDPy$; karena $Py = \mathbf{V}^{-1}(y - \mathbf{X}\hat{\beta})$, kuadratik itu sama dengan $(y - \mathbf{X}\hat{\beta})'\mathbf{V}^{-1}D_a\mathbf{V}^{-1}(y - \mathbf{X}\hat{\beta})$, yaitu skor ML terprofil (SFH-1). Untuk `eblup_twofold` kedua bentuk ditulis terpisah dalam (55) dan (56).

Langkah iterasi. Karena ℓ tidak konkav linear pada θ, digunakan langkah Newton–Fisher $\Delta = \mathcal{I}^{-1}s$ (dengan pembakuan `solve` dan fallback `pinv` bila singular), diikuti pembatasan domain parameter (ρ diklem ± 0.999, $\sigma^2 \geq 0$) dan pemberhentian berdasarkan galat relatif $\max_a |\Delta_a/(\theta_a + 10^{-12})| \leq 10^{-4}$. Galat baku komponen ragam diambil dari $\sqrt{\mathrm{diag}(\mathcal{I}^{-1})}$ pada informasi final.

Turunan D_a yang dipakai. Untuk SFH/STFH dengan $A = (\mathbf{I} - \rho\mathbf{W}')'(\mathbf{I} - \rho\mathbf{W})$,

$$\frac{\partial A}{\partial \rho} = -(\mathbf{W} + \mathbf{W}') + 2\rho\mathbf{W}'\mathbf{W}, \qquad \frac{\partial G}{\partial \rho} = -\sigma^2 A^{-1}\frac{\partial A}{\partial \rho}A^{-1}, \qquad \frac{\partial G}{\partial \sigma^2} = A^{-1}, \tag{100}$$

dan pada dua-fold dua turunan hanya terhadap σ_v^2 dan σ_u^2 lewat $\partial V_d/\partial\sigma_v^2 = \mathbf{1}\mathbf{1}'$, $\partial V_d/\partial\sigma_u^2 = \mathbf{I}$ (matriks identitas pada blok d).

C Derivasi suku-suku MSE (g1, g2, g3, g4)

Dekomposisi galat prediktor. Dengan $G = \sigma^2 A^{-1}$, $\mathbf{V} = G + \Psi$ ($\Psi = \mathrm{diag}(\psi)$), dan prediktor $\hat{\theta} = \mathbf{Z}\hat{\beta} + G\mathbf{V}^{-1}(y - \mathbf{Z}\hat{\beta})$, tulis $y = \mathbf{Z}\beta + \theta + e$. Menggunakan $\mathbf{V}^{-1}\Psi = \mathbf{I} - \mathbf{V}^{-1}G$

dan $\Psi\mathbf{V}^{-1} = \mathbf{I} - G\mathbf{V}^{-1}$:

$$\hat{\theta} - \theta = \underbrace{(\mathbf{I} - G\mathbf{V}^{-1})\mathbf{Z}(\hat{\beta} - \beta)}_{\text{galat } \beta} + \underbrace{G\mathbf{V}^{-1}e - \Psi\mathbf{V}^{-1}\theta}_{\text{galat } u,e}. \tag{101}$$

g_1. Dari suku kedua (kosinus nol karena $e \perp \theta$):

$$\begin{aligned}
\mathrm{Var}(G\mathbf{V}^{-1}e - \Psi\mathbf{V}^{-1}\theta) &= G\mathbf{V}^{-1}\Psi\mathbf{V}^{-1}G + \Psi\mathbf{V}^{-1}G\mathbf{V}^{-1}\Psi \\
&= \underbrace{G\mathbf{V}^{-1}\Psi\mathbf{V}^{-1}G}_{A\Psi A',\ A=G\mathbf{V}^{-1}} + \underbrace{(\mathbf{I}-A)G(\mathbf{I}-A)'}_{\text{perluasan}} \\
&= G\mathbf{V}^{-1}G - G\mathbf{V}^{-1}G\mathbf{V}^{-1}G + G - 2G\mathbf{V}^{-1}G + G\mathbf{V}^{-1}G\mathbf{V}^{-1}G \\
&= G - G\mathbf{V}^{-1}G,
\end{aligned} \tag{102}$$

sehingga $g_1 = \mathrm{diag}(G - G\mathbf{V}^{-1}G)$ — bentuk yang persis dikodekan (`Ga = G - GVi * G`).

g_2. Dari suku pertama (101) dengan $\mathrm{Var}(\hat{\beta} - \beta) = Q = (\mathbf{X}'\mathbf{V}^{-1}\mathbf{X})^{-1}$:

$$g_2 = \mathrm{diag}(R\,Q\,R'), \qquad R = (\mathbf{I} - G\mathbf{V}^{-1})\mathbf{Z} \quad (\text{kode: } R = X - GV^{-1}X). \tag{103}$$

g_3 **(ketidakpastian parameter ragam).** Karena G bergantung pada θ, tulis $\hat{\theta}_i(\hat{\theta}) \approx \hat{\theta}_i(\theta) + \xi_i'(\hat{\theta}_{\text{param}} - \theta)$ dengan $\mathrm{Var}(\hat{\theta}_{\text{param}}) \approx \mathcal{I}^{-1}$, sehingga

$$g_3(i) = \xi_i'\mathcal{I}^{-1}\xi_i = \sum_{a,b}(\mathcal{I}^{-1})_{ab}\,\xi_{ia}\xi_{ib}. \tag{104}$$

Kode SFH mengekspresikan bentuk ini lewat $l_1 = \mathbf{V}^{-1}D_1 - \sigma^2\mathbf{V}^{-1}D_1\mathbf{V}^{-1}D_1$ dan $l_2 = \mathbf{V}^{-1}D_2 - \sigma^2\mathbf{V}^{-1}D_2\mathbf{V}^{-1}D_1$ ($D_1 = \mathbf{I}$, $D_2 = \partial\mathbf{V}/\partial\rho$): dengan L_i matriks $2 \times m$ yang barisnya adalah l_{1i}' dan l_{2i}',

$$g_3(i) = \mathrm{tr}(L_i\,\mathbf{V}\,L_i'\mathcal{I}^{-1}) = \sum_{a,b}(\mathcal{I}^{-1})_{ab}\,l_{ai}'\mathbf{V}l_{bi}, \tag{105}$$

identik dengan (104) melalui $\xi_a{}'\xi_b = l_a'\mathbf{V}l_b$. Untuk FH (satu parameter) ini meluruh menjadi $g_3 = \mathrm{Var}(\hat{\sigma}^2) \cdot$ (derivative-squared) seperti ditulis pada (14).

g_4 **(koreksi orde keempat, hanya SFH).** Suku ini berasal dari estimasi Ψ (varian direktor) yang juga tidak diketahui; kode membangun $D = (\Psi\mathbf{V}^{-1}D_{12})(\mathcal{I}_{01}^{-1} + \mathcal{I}_{10}^{-1}) + (\Psi\mathbf{V}^{-1}D_{22})\mathcal{I}_{11}^{-1}$ dengan $D_{12} = -\partial^2 A/\partial\rho^2$-faktor dan D_{22} orde kedua pada ρ, lalu $g_4(i) = \frac{1}{2}D_{ii}$. Totalnya (34): $g_1 + g_2 + 2g_3 - g_4$; STFH memakai bentuk yang sama ((44)); untuk dua-fold (kovarians linear dalam σ_v^2, σ_u^2, sehingga $g_4 = 0$) estimatornya $g_1 + g_2 + 2g_3$ ((60)).

Domain tanpa sampel. Pada FH prediktor sintetis $x_{ns}'\hat{\beta}$ tidak bergantung pada θ selain lewat $\hat{\beta}$, sehingga

$$\widehat{\mathrm{MSE}}(\hat{\theta}_{ns}) = g_{1,ns} + g_{2,ns} = \hat{\sigma}^2 + x_{ns}'Qx_{ns} \quad (\text{FH}). \tag{106}$$

Pada SFH prediktor kriging bergantung pada θ, sehingga dipakai MSE orde kedua penuh (turunan eksak prediktor kriging dan koreksi bias g_1 plug-in; bagian 6.5). Dua-fold untuk sub-area tak-sampel pada area tersampel memakai $g_1^* + g_2^* + 2g_3^*$ (dikurangi suku bias ML bila ML; `eblup_twofold.cpp:500-545`).

Catatan dua-fold. Komentar kode `eblup_twofold.cpp:19-24` menyatakan bahwa pendekatan g_3 matriks menggantikan rumus tertutup Torabi and Rao [2014] (persamaan 3.4) karena suku $[A][B]$ dianggap semu; pendekatan itu divalidasi dengan turunan beda pemisah (korelasi 0,97/0,96/0,74 terhadap uji numerik). Laporan ini memperlakukannya sebagai perbedaan implementasi TFH-1, bukan sebagai kekeliruan yang diverifikasi ulang di sini.

D Glosarium

Tabel 33 menautkan istilah, simbol, dan nama di kode.

Tabel 33: Glosarium tiga arah: istilah — simbol — nama di kode.

Istilah	Simbol	Nama di kode
Estimasi BLUP empirik	$\hat{\theta}_d$	`eblup`, `hb`, `ebp`
Varians direktor	ψ_d	`vardir`
Fungsi presisi spasial	A	`A` (`sar_engine.h`, `SarEngine::factor`)
Matriks proyeksi	P	`P = Vi - XtVi.t()*Q*XtVi`
Matriks info Fisher	$\mathcal{I}$	`Idev` (SFH), `F` (STFH), `I` (FH)
Korelasi spasial (SAR)	ρ	`rho`, `rho_fix`
Fraksi spasial BYM2	φ	`phi` (`hyperpar`)
Fraksi sampel	$f_d = n_d/N_d$	`f`, `ratio` (BHF)
Suku g MSE	$g_1, \ldots, g_4$	`g1d`, `g2d`, `g3d`, `g4d`
Galat mutlak relatif	RSE	`rse` ($100\sqrt{\text{mse}}/\left\lvert\hat{\theta}\right\rvert$)
Domain tanpa sampel	I_{ns}	`find_nonfinite(y)`, `unsampled_domains`
Inversi Woodbury	(95)	variabel `C` (STFH), `M` (SFH), `dd/s/c` (two-fold)
Aproksimasi Laplace	—	`strategy` pada `INLA::inla`
Prior PC	(78)	`prior_prec = list(prior="pc.prec", ...)`
Graf ketetanggaan	W, Q	`W`, `graph` (`inla.graph`)
Indeks domain (0-based)	—	`domain_idx`, `area_idx`
Kelas hasil	—	`"fastsae"`, `"fastsae_hb"`, `"fastsae_benchmark"`

E Reproduksi angka dan gambar dalam laporan ini

Seluruh tabel dan gambar di bagian 3 dihasilkan ulang dari data mentah di `inst/extdata` oleh dua skrip; tidak ada angka benchmark yang diketik manual.

Listing 19: Perintah regenerasi (dikerjakan dari direktori `technical-report/`).

```
# 1. tabel (.tex berisi lingkungan float lengkap) -> technical-report/tables/
Rscript scripts/01_tables.R

# 2. gambar log-scale -> technical-report/figures/
Rscript scripts/02_figures.R

# 3. laporan
latexmk -pdf main.tex          # atau: latexmk -pdf -jobname=fastsae_technical_report main.tex
```

Setiap `\caption` hasil skrip menyertakan baris *Sumber:* yang menunjuk berkas `.rds` sumber, dan setiap angka diformat `formatC(format="g", digits=6)`, sehingga perbedaan pembulatan antara laporan ini dan README dapat ditelusuri ke skrip, bukan ke pengetikan.

Lingkungan eksekusi. Kutipan `sessionInfo()` yang berlaku untuk pembuatan laporan ini tersimpan di `scripts/sessionInfo.txt`; ringkasannya:

- Apple M4 (10 core), R 4.6.1 (2026-06-24), `aarch64-apple-darwin23`, macOS Tahoe 26.6.2, zona waktu `Asia/Makassar`;
- paket: `fastsae` 1.0.0, `sae` 1.3, `emdi` 2.2.3, `tipsae` 1.0.3, `hbsae` 1.2, `INLA` 26.8.7, `Matrix` 1.7.5, `Rcpp` 1.1.2, `RcppArmadillo` 15.4.2.1, `RcppEigen` 0.3.4.0.2, `bench` 1.1.4, `ggplot2` 4.0.3, `sf` 1.1.1, `spdep` 1.4.2, `cli` 3.6.6, `testthat` 3.3.2, `covr` 3.6.5, `lme4` 2.0.1 (hanya pembanding di tes).

Yang tidak dapat direproduksi dari repo. Berdasarkan pemeriksaan terhadap `inst/extdata` dan `benchmarks/` (bagian 3, temuan DOC-11):

- baris sae/emdi pada `eblup_benchmark.rds`, `seblup_benchmark.rds`, `steblup_benchmark.rds`, serta kolom tipsae pada `beta_estimates_n1000.rds` tidak memiliki skrip pembuat di repo, sehingga tidak dapat dibangkitkan ulang (kolom fastsae-nya dapat, lihat bagian 3.1); baris fastsae di semua berkas benchmark dapat dibangkitkan ulang dengan `benchmarks/bench_frequentist_fastsae.R` dan `benchmarks/bench_bayes_fastsae.R`;
- jumlah iterasi nyata sebagian besar run tidak tercatat di dalam berkas (hanya `n_iter` hasil `bench::mark`);
- jenis CPU, jumlah thread, dan `OMP_NUM_THREADS` tidak direkam di dalam berkas (pengukuran ulang fastsae: lihat `scripts/sessionInfo.txt`), sehingga angka waktu baris pembanding hanya dapat dibandingkan *di dalam* satu berkas yang sama;
- memori yang dilaporkan adalah total alokasi `bench::mark()$mem_alloc`, bukan puncak RSS (README, beranda situs, dan vignette kini melabelinya demikian).

Untuk pengujian unit dan cakupan kode, perintah yang dipakai pada pengembangan paket adalah:

```
# unit test (membutuhkan GCC/Homebrew untuk gcov bila ingin cakupan)
R CMD build . && R CMD check fastsae_*.tar.gz
# cakupan: hasilnya disimpan sebagai cov_res.rds, lalu
# covr::tally_coverage(readRDS("cov_res.rds"))
```

Referensi yang tidak dapat diverifikasi. Daftar lengkap entri yang ada di blok `@references` kode tetapi tidak dapat diverifikasi (Steorts et al. 2014; Brown et al. 2001; Cliff & Ord 1981) beserta dua koreksi judul/tahun terhadap teks kode terdapat di `TODO_REFERENCES.md`; seluruh perbedaan implementasi terkumpul di `DISCREPANCIES.md`.

Pustaka

Luc Anselin. *Spatial Econometrics: Methods and Models*. Springer Netherlands, Dordrecht, 1988. doi: 10.1007/978-94-015-7799-1.

Sudipto Banerjee, Bradley P. Carlin, and Alan E. Gelfand. *Hierarchical Modeling and Analysis for Spatial Data*. Chapman & Hall/CRC Monographs on Statistics and Applied Probability. Chapman & Hall/CRC, Boca Raton, FL, 2nd edition, 2014. ISBN 978-1-4398-1917-3. doi: 10.1201/b17115.

Douglas Bates, Martin Mächler, Ben Bolker, and Steve Walker. Fitting linear mixed-effects models using lme4. *Journal of Statistical Software*, 67(1), 2015. doi: 10.18637/jss.v067.i01.

George E. Battese, R. M. Harter, and Wayne A. Fuller. An error-components model for prediction of county crop areas using survey and satellite data. *Journal of the American Statistical Association*, 83:28–36, 1988. doi: 10.1080/01621459.1988.10478561.

Roberto Benedetti, Federica Piersimoni, Monica Pratesi, Nicola Salvati, and Thomas Suesse. Handling out-of-sample areas to estimate the unemployment rate at local labour market areas in Italy. *International Statistical Review*, 94(1):140–162, 2026. doi: 10.1111/insr.12596. First published online 2024.

Emily J. Berg and Wayne A. Fuller. Small area prediction of proportions with applications to the Canadian Labour Force Survey. *Journal of Survey Statistics and Methodology*, 2(3): 227–256, 2014. doi: 10.1093/jssam/smu011.

Julian Besag. Spatial interaction and the statistical analysis of lattice systems. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 36(2):192–236, 1974. doi: 10.1111/j.2517-6161.1974.tb00999.x.

Julian Besag, Jeremy York, and Annie Mollié. Bayesian image restoration, with two applications in spatial statistics. *Annals of the Institute of Statistical Mathematics*, 43(1):1–20, 1991. doi: 10.1007/BF00116466.

G. S. Datta, M. Ghosh, R. Steorts, and J. Maples. Bayesian benchmarking with applications to small area estimation. *TEST*, 20(3):574–588, 2010. doi: 10.1007/s11749-010-0218-y.

Gauri S. Datta, Malay Ghosh, Rebecca Steorts, and Jerry Maples. Bayesian benchmarking with applications to small area estimation. *TEST*, 20(3):574–588, 2011. doi: 10.1007/s11749-010-0218-y.

Gauri Sankar Datta and Malay Ghosh. Bayesian prediction in linear models: Applications to small area estimation. *The Annals of Statistics*, 19(4):1748–1770, 1991. doi: 10.1214/aos/1176348369.

Gauri Sankar Datta and P. Lahiri. A unified measure of uncertainty of estimated best linear unbiased predictors in small area estimation problems. *Statistica Sinica*, 10:613–627, 2000.

Silvia De Nicolò and Aldo Gardini. The R package tipsae: Tools for mapping proportions and indicators on the unit interval. *Journal of Statistical Software*, 108(1):1–36, 2024. doi: 10.18637/jss.v108.i01.

Silvia De Nicolò, Maria Rosaria Ferrante, and Silvia Pacei. Small area estimation of inequality measures using mixtures of Beta. *Journal of the Royal Statistical Society: Series A (Statistics in Society)*, 187(1):85–109, 2024. doi: 10.1093/jrsssa/qnad083.

María Dolores Esteban, María José Lombardía, Esther López-Vizcaíno, Domingo Morales, and Agustín Pérez. Small area estimation of proportions under area-level compositional mixed models. *TEST*, 29(3):793–818, 2019. doi: 10.1007/s11749-019-00688-w.

Robert E. Fay and Roger A. Herriot. Estimates of income for small places: An application of James-Stein procedures to census data. *Journal of the American Statistical Association*, 74: 269–277, 1979. doi: 10.1080/01621459.1979.10482505.

Silvia L. P. Ferrari and Francisco Cribari-Neto. Beta regression for modelling rates and proportions. *Journal of Applied Statistics*, 31(7):799–815, 2004. doi: 10.1080/0266476042000214501.

Virgilio Gómez-Rubio. *Bayesian Inference with INLA*. Chapman & Hall/CRC, Boca Raton, FL, 2020. ISBN 978-1-138-03987-2. doi: 10.1201/9781315175584.

Wenceslao González-Manteiga, María José Lombardía, Isabel Molina, Domingo Morales, and Laura Santamaría. Bootstrap mean squared error of a small-area EBLUP. *Journal of Statistical Computation and Simulation*, 78(5):443–462, 2008. doi: 10.1080/00949650601141811.

Ryan Janicki. Properties of the beta regression model for small area estimation of proportions and application to estimation of poverty rates. *Communications in Statistics – Theory and Methods*, 49(9):2264–2284, 2020. doi: 10.1080/03610926.2019.1570266.

R. N. Kackar and David A. Harville. Approximations for standard errors of estimators of fixed and random effects in mixed linear models. *Journal of the American Statistical Association*, 79:853–862, 1984. doi: 10.1080/01621459.1984.10477102.

Ann-Kristin Kreutzmann, Sören Pannier, Natalia Rojas-Perilla, Timo Schmid, Matthias Templ, and Nikos Tzavidis. The R package emdi for estimating and mapping regionally disaggregated indicators. *Journal of Statistical Software*, 91(7):1–33, 2019. doi: 10.18637/jss.v091.i07.

Finn Lindgren and Håvard Rue. Bayesian spatial modelling with R-INLA. *Journal of Statistical Software*, 63(19), 2015. doi: 10.18637/jss.v063.i19.

Eduardo Marhuenda, Isabel Molina, and Domingo Morales. Small area estimation with spatio-temporal Fay–Herriot models. *Computational Statistics & Data Analysis*, 58:308–325, 2013. doi: 10.1016/j.csda.2012.09.002.

Isabel Molina and Yolanda Marhuenda. sae: An R package for small area estimation. *The R Journal*, 7(1):81–98, 2015. doi: 10.32614/RJ-2015-007.

Isabel Molina and J. N. K. Rao. Small area estimation of poverty indicators. *Canadian Journal of Statistics*, 38(3):369–385, 2010. doi: 10.1002/cjs.10051.

Isabel Molina, Nicola Salvati, and Monica Pratesi. Bootstrap for estimating the MSE of the spatial EBLUP. *Computational Statistics*, 24(3):441–458, 2009. doi: 10.1007/s00180-008-0138-4.

Danny Pfeffermann. New important developments in small area estimation. *Statistical Science*, 28(1):40–68, 2013. doi: 10.1214/12-STS395.

N. G. N. Prasad and J. N. K. Rao. The estimation of the mean squared error of small-area estimators. *Journal of the American Statistical Association*, 85:163–171, 1990. doi: 10.1080/01621459.1990.10475320.

Monica Pratesi and Nicola Salvati. Small area estimation: the EBLUP estimator based on spatially correlated random area effects. *Statistical Methods and Applications*, 17(1):113–141, 2008. doi: 10.1007/s10260-007-0061-9.

J. N. K. Rao and Isabel Molina. *Small Area Estimation*. John Wiley & Sons, Hoboken, NJ, 2nd edition, 2015. ISBN 9781118735787. doi: 10.1002/9781118735855.

J. N. K. Rao and M. Yu. Small-area estimation by combining time-series and cross-sectional data. *Canadian Journal of Statistics*, 22:511–528, 1994. doi: 10.2307/3315407.

Andrea Riebler, Sigrunn H. Sørbye, Daniel Simpson, and Håvard Rue. An intuitive Bayesian spatial model for disease mapping that accounts for scaling. *Statistical Methods in Medical Research*, 25(4):1145–1165, 2016. doi: 10.1177/0962280216660421.

Håvard Rue and Leonhard Held. *Gaussian Markov Random Fields: Theory and Applications*, volume 104 of *Monographs on Statistics and Applied Probability*. Chapman & Hall/CRC, Boca Raton, FL, 2005. ISBN 978-1-58488-432-3. doi: 10.1201/9780203492024.

Håvard Rue, Sara Martino, and Nicolas Chopin. Approximate Bayesian inference for latent Gaussian models by using integrated nested Laplace approximations. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 71(2):319–392, 2009. doi: 10.1111/j.1467-9868.2008.00700.x.

Ayoub Saei and Ray Chambers. Out of sample estimation for small areas using area level data. Methodology Working Paper M05/11, Southampton Statistical Sciences Research Institute, University of Southampton, 2005.

Daniel Simpson, Håvard Rue, Andrea Riebler, Thiago G. Martins, and Sigrunn H. Sørbye. Penalising Model Component Complexity: A principled, practical approach to constructing priors. *Statistical Science*, 32(1):1–28, 2017. doi: 10.1214/16-STS576.

Eric V. Slud and Tapabrata Maiti. Mean-squared error estimation in transformed Fay-Herriot models. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 68(5):833–850, 2006. doi: 10.1111/j.1467-9868.2006.00569.x.

Sigrunn H. Sørbye and Håvard Rue. Scaling intrinsic Gaussian Markov random field priors in spatial modelling. *Spatial Statistics*, 8:39–51, 2014. doi: 10.1016/j.spasta.2013.06.004.

Mahmoud Torabi and J. N. K. Rao. On small area estimation under a sub-area level model. *Journal of Multivariate Analysis*, 127:36–55, 2014. doi: 10.1016/j.jmva.2014.02.001.

Junyuan Wang, Wayne A. Fuller, and Yongming Qu. Small area estimation under a restriction. *Survey Methodology*, 34(1):29–36, 2008.

Kirk M. Wolter. *Introduction to Variance Estimation*. Springer, New York, 2 edition, 2007. doi: 10.1007/978-0-387-35099-8.

Yong You and J. N. K. Rao. A pseudo-empirical best linear unbiased prediction approach to small area estimation using survey weights. *Canadian Journal of Statistics*, 30:431–439, 2002. doi: 10.2307/3316146.